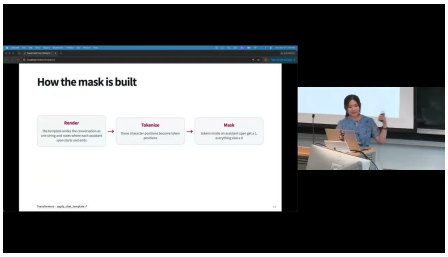
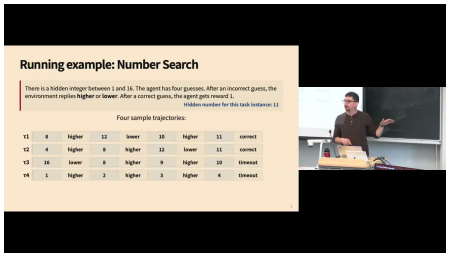
CMU 11-768 AI Agents（2026 秋）
训练模块精读：从 SFT 到异步 RL 系统，
以及 Assignment 3（scottyrl）复现指南

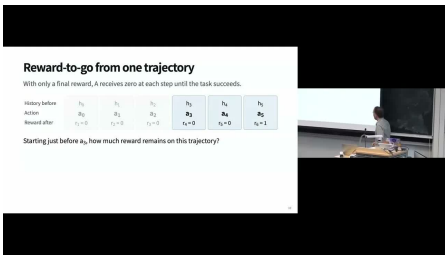
五道口纳什 & Claude Code　　2026 年 10 月 21 日



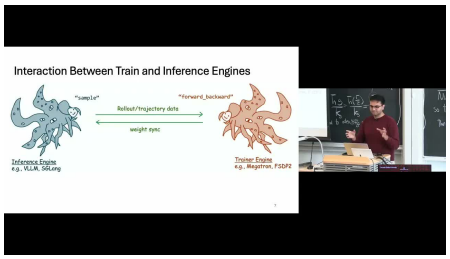
第 8 讲 · Supervised Fine-Tuning（Yueqi Song）



第 9 讲 · RL Basics（Daniel Fried）



第 11 讲 · Advanced RL Algorithms（Graham Neubig）



第 12 讲 · RL Systems（Apurva Gandhi）

视频作者/频道：Graham Neubig（YouTube），CMU LTI 课程 11-768，授课 Daniel Fried、Graham Neubig 及客座讲者
系列入口（用户给定）：`https://www.youtube.com/watch?v=UwfjzyLnvMg`（第 1 讲 What are Agents and How Do They Work?，2026-09-08 上传）
本笔记覆盖的 training 模块视频（上传日期 · 时长）：
第 8 讲 `youtu.be/O3HSU0AoILc`（2026-09-25 · 1:04:59）　第 9 讲 `youtu.be/paAcPaaYZGM`（2026-09-25 · 1:17:46）
第 11 讲 `youtu.be/jKx7_fOZRzg`（2026-10-12 · 1:08:15）　第 12 讲 `youtu.be/dwR6NP9nI-8`（2026-10-14 · 1:06:26）
课程主页 / 课件：`https://cmu-agents.com`（`/slides/lecture-08-sft.pdf` 等）
作业仓库：`https://github.com/cmu-agents/assignment-3`（commit `ef42c0d`，scottyrl）
PDF 制作 Skill：`https://github.com/wdkns/wdkns-skills`

章节索引（点击跳转）

1　课程背景与本笔记的范围　5
2　第 8 讲：智能体的监督微调（SFT）　7
3　第 9 讲：RL 基础——从 SFT 到策略梯度　23
4　第 11 讲：进阶 RL 算法——信用分配、异步与奖励可靠性　42
5　第 12 讲：RL 系统——两个引擎、智能体特有的坑与服务化设计　57
6　项目复现：Assignment 3 · scottyrl 训练智能体　71
7　总结与延伸　86

目录

1 课程背景与本笔记的范围 **5**

1.1 这门课与它的 training 模块 . . . 5

1.2 四讲之间的逻辑与本笔记的读法 . . . 6

1.3 本章小结 . . . 6

2 第 8 讲：智能体的监督微调（SFT） **7**

2.1 背景：把能力写进权重 . . . 7

2.1.1 第三种更新智能体的方式 . . . 7

2.1.2 训练流水线与冷启动 . . . 8

2.2 轨迹作为 token 序列 . . . 9

2.2.1 chat template 与损失掩码 . . . 9

2.2.2 回合结束符与推理块 . . . 11

2.2.3 token 级加权与样本—token 失衡 . . . 12

2.3 选择轨迹 . . . 14

2.3.1 轨迹从哪里来 . . . 14

2.3.2 成功的 run 里也有坏习惯 . . . 15

2.3.3 OpenThoughts-Agent 的四组消融 . . . 15

2.4 统一多个数据集的格式：Agent Data Protocol . . . 17

2.5 运行训练 . . . 19

2.5.1 MoE 的专家路由、打包与模板漂移 . . . 19

2.6 怎样知道训练有效 . . . 20

2.7 交给 RL：SFT 该训练到什么程度 . . . 21

2.8 本章小结 . . . 22

2.9 拓展阅读 . . . 23

3 第 9 讲：RL 基础——从 SFT 到策略梯度 **23**

3.1 贯穿示例与 SFT 的三种失配 . . . 23

3.1.1 Number Search . . . 23

3.1.2 SFT 的目标与它的三种失配 . . . 24

3.2 把交互写成轨迹 . . . 26

3.2.1 目标：最大化期望奖励 . . . 27

3.3 专家迭代：ReST . . . 28

3.4 从奖励到梯度：REINFORCE . . . 29

3.4.1 为什么不能直接反向传播 . . . 29

3.4.2 推导 . . . 30

3.4.3 实现：对 SFT 损失做掩码与缩放 . . . 31
3.5 基线与优势：比预期好多少 . . . 32
3.5.1 基线不引入偏差 . . . 33
3.5.2 两臂老虎机：把方差算出来 . . . 34
3.6 组内比较：GRPO 与 Dr.GRPO . . . 37
3.6.1 Dr.GRPO 的两处修改 . . . 38
3.6.2 退化组 . . . 40
3.7 总结：所有方法都在给自己的动作加权 . . . 41
3.8 本章小结 . . . 41
3.9 拓展阅读 . . . 42

4 第 11 讲：进阶 RL 算法——信用分配、异步与奖励可靠性 42
4.1 贯穿示例：修一个 bug . . . 43
4.2 信用分配与稠密反馈 . . . 43
4.2.1 从组基线到逐回合信用 . . . 43
4.2.2 价值函数与 critic . . . 44
4.2.3 TD 误差与 GAE . . . 45
4.3 同步与异步 RL . . . 46
4.4 稳定的策略更新 . . . 46
4.4.1 过期 rollout 与重要性采样 . . . 46
4.4.2 长轨迹让比率失控 . . . 47
4.4.3 裁剪 . . . 48
4.4.4 KL 惩罚与熵奖励 . . . 50
4.4.5 算法总表 . . . 51
4.5 任务选择与奖励可靠性 . . . 52
4.5.1 零优势组与全失败的三种原因 . . . 52
4.5.2 假阴性：verifier 拒绝了正确解 . . . 52
4.5.3 假阳性与 reward hacking . . . 53
4.5.4 部分奖励与辅助奖励 . . . 54
4.6 从教师学习 . . . 55
4.7 本章小结 . . . 56
4.8 拓展阅读 . . . 57

5 第 12 讲：RL 系统——两个引擎、智能体特有的坑与服务化设计 57
5.1 为什么 RL 框架由两个引擎组成 . . . 58
5.2 推理侧的优化 . . . 60

5.3 训练侧的优化与并行 . . . 61
5.4 训练与推理之间的交互 . . . 63
5.5 智能体 RL 的坑 . . . 64
5.5.1 前缀扩展 . . . 64
5.5.2 Token-In-Token-Out . . . 65
5.5.3 异步 RL 与在途权重更新 . . . 66
5.6 系统设计选择 . . . 67
5.7 讲义概念与作业实现的对照 . . . 70
5.8 本章小结 . . . 70
5.9 拓展阅读 . . . 71

6 项目复现：Assignment 3 · scottyrl 训练智能体 71
6.1 项目背景：要做的是什么，为什么用 Tinker 接口 . . . 71
6.2 环境、数据与任务 . . . 72
6.2.1 Number Search：猜数字 . . . 72
6.2.2 TextCraft-Synth：按配方树合成物品 . . . 73
6.3 模型与提示格式 . . . 74
6.4 框架结构与阶段划分 . . . 75
6.5 第 1 阶段：从 episode 到训练序列 . . . 76
6.5.1 目标函数与掩码 . . . 76
6.5.2 拼接算法与差一位对齐 . . . 77
6.5.3 组内优势 . . . 78
6.6 第 2 阶段：同步训练循环 . . . 78
6.6.1 一个训练步做了什么 . . . 78
6.6.2 三种 token 级损失 . . . 79
6.6.3 唯一一个后端相关的学生函数：`submit_update` . . . 80
6.7 第 3 阶段：异步 GRPO 与过期数据 . . . 80
6.7.1 生产者—消费者结构 . . . 80
6.7.2 过期度能有多大：一个容易验证的上界 . . . 80
6.7.3 异步实现的几条硬性要求 . . . 81
6.8 第 4 阶段：三组消融 . . . 82
6.9 第 5 阶段：TextCraft-Synth hero run . . . 83
6.10 训练后端：SkyRL Tinker 服务部署在 Modal . . . 83
6.11 运行、监控与诊断 . . . 84
6.12 结果与结论：能期望看到什么 . . . 85
6.13 复现清单 . . . 85

6.14 本章小结 86

7 总结与延伸 **86**

7.1 讲者们的收尾 86

7.2 一张表看懂「给动作加权」 87

7.3 各讲引用案例速查 87

7.4 一条完整的智能体训练流水线 88

7.5 贯穿全模块的几条结论 89

7.6 开放问题与下一步 89

1 课程背景与本笔记的范围

1.1 这门课与它的 training 模块

用户给出的链接是 CMU 11-768「AI Agents」(2026 秋) 的第 1 讲。这是卡内基梅隆大学语言技术研究所 (LTI) 的研究生课程，主讲 Daniel Fried 与 Graham Neubig，课程主页 `cmu-agents.com` 把它描述为「关于基于 LLM 的智能体的研究生课程——工具使用、规划、记忆、训练、安全与交互」，面向「已有训练神经语言模型经验」的学生 (先修 11-667、11-711 或 10-202 一类课程)。课程共五个模块：

模块	主页上的主题描述
Agent Capabilities	指令遵循、工具使用、规划、记忆系统
Training Methods	**监督微调，以及强化学习的算法与系统**
Frameworks	OpenHands、LangGraph 等智能体开发框架
Safety	沙箱、凭据管理、对智能体的对抗攻击
Advanced Topics	人机交互、多智能体、基于搜索的推理

本笔记只覆盖 Training Methods 模块，即下表四讲，以及与之配套、占总成绩 15% 的 Assignment 3 · Training (截止 2026-10-29)：

讲次	日期	讲者	主题与本笔记章节
8	9 月 17 日	Yueqi Song	Training 1：Supervised Fine-Tuning (第 2 章)
9	9 月 22 日	Daniel Fried	Training 2：Reinforcement Learning Basics (第 3 章)
11	9 月 29 日	Graham Neubig	Training 3：Advanced RL Algorithms (第 4 章)
12	10 月 1 日	Apurva Gandhi	Training 4：RL Systems (第 5 章)
—	截止 10 月 29 日	Apurva Gandhi 等	Assignment 3：scottyrl (第 6 章)

第 10 讲 (Deep Research Agents) 夹在中间，与训练无关，不在范围内。四讲的课件 PDF 均在课程主页公开 (`/slides/lecture-08-sft.pdf` 等)，作业代码在 GitHub 公开。本笔记以视频为主线，用课件核对公式与数字，用作业代码给出可执行的细节。

1.2　四讲之间的逻辑与本笔记的读法

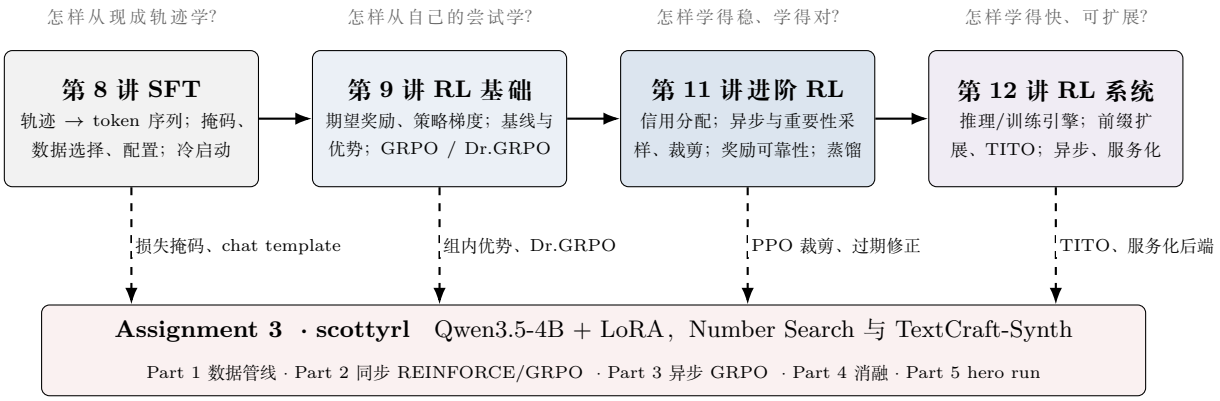


图 1: 四讲的问题链与作业的对应关系。上方灰字是每讲要回答的问题，虚线标出每讲在作业中落地的具体机制。

四讲沿一条线推进。第 8 讲回答「已有一批好的轨迹，怎样把它们写进权重」，并指出 SFT 的上限：示范可能错误、狭窄或用尽，且模型从未见过自己出错后的状态。第 9 讲用同一个猜数字例子说明 RL 如何同时化解三种失配（任务目标、数据质量、暴露偏差），推导出策略梯度，并逐步加入基线、组内优势，得到作业所用的 GRPO / Dr.GRPO。第 11 讲处理把 RL 用到智能体上会遇到的三类问题：信用分配稀疏、rollout 太慢需要异步（因而产生过期数据与重要性采样）、奖励不可靠。第 12 讲转向系统：RL 框架是推理引擎与训练引擎两种完全不同工作负载的组合，智能体场景还有前缀扩展、Token-In-Token-Out 等特有的坑。

贯穿四讲的统一视角

第 9 讲结尾的总结表可以作为全篇的索引：本模块里的所有方法都在给智能体采取过的动作的对数概率加权，差别只在权重 w_t。

$$\nabla_\theta J = \sum_t w_t \, \nabla_\theta \log \pi_\theta(a_t \mid h_t)$$

- h_t：第 t 步之前的历史（任务、之前的动作与观测）；a_t：第 t 步的动作（一段 token）；
- SFT：动作来自示范，$w_t = 1$；ReST/专家迭代：自采样后只保留成功轨迹，成功为 1、其余为 0；
- REINFORCE：$w_t = R(\tau)$；GRPO / Dr.GRPO：$w_t = \hat{A}_j = R(\tau_j) - \bar{R}$（GRPO 原版再除以组内标准差）；
- PPO 类：在 $\hat{A}$ 之外再乘以裁剪过的重要性比率；蒸馏：权重换成「教师与学生分布之差」的稠密信号。

1.3　本章小结

本章确定了范围：CMU 11-768 的 Training Methods 模块四讲（SFT、RL 基础、进阶 RL、RL 系统）与 Assignment 3。四讲依次回答「从示范学 → 从自己的尝试学 → 学得稳且对 → 学得快且可

扩展」，而作业把后三讲的核心机制压缩进一个约 5,600 行（含测试与脚本）的教学框架。视频未展开但值得先记住的一点是：上面的统一公式说明 SFT 与策略梯度在实现上共享同一段代码（对被掩码选中的 token 求对数似然），区别只在于「数据从哪里来」与「每个 token 乘以什么权重」；第 6 章的 `to_tinker_datum` 正是用 `advantages` 或 `weights` 一个字段承载了这个权重。

2　第 8 讲：智能体的监督微调（SFT）

本讲由 Yueqi Song（CMU LTI，Graham Neubig 组；她在课上介绍 Agent Data Protocol 是自己与 Graham Neubig 的工作）主讲（1:04:59），副标题是「把录制好的轨迹变成权重」。她给出的核心立场是：智能体 SFT 是带教师强制的模仿学习[1]，它会复制数据里的一切，所以数据的选择比调超参数更影响结果。全讲沿七个问题展开：为什么要改权重、轨迹怎样变成 token 序列、损失算在哪些 token 上、轨迹从哪里来、多个数据集怎样统一格式、真实训练怎样配置、怎样判断训练有效，最后说明 SFT 作为 RL 的冷启动该训练到什么程度。

2.1　背景：把能力写进权重

2.1.1　第三种更新智能体的方式

本课程此前两讲讨论过另外两种「更新智能体」的位置：上下文窗口（9 月 1 日，长上下文）与外部工件（9 月 3 日，记忆与技能）。本讲是第三种：模型权重。

更新位置	优点	缺点
上下文窗口	对已发生内容保真度高	成本高、噪声大，不决定什么重要
外部工件	可检查、可编辑、可检索、可移植	必须被归纳、筛选和维护
模型权重	推理更快，行为改变范围广	更新慢（每次都要一次训练）、不透明、绑定具体模型

讲者的概括是 “change the weight once and every call after that gets a new behavior with nothing extra in the prompt”（00:02:51）；代价是无法打开权重查看改了什么，也不能把改动迁移到另一个模型。她用 SWE-Gym 的一条样例轨迹贯穿全讲：智能体收到一个 issue 和沙箱里的两个工具（bash 与文件编辑器），产生思考与命令、读取命令返回的观测，最后由仓库自带的单元测试判定是否成功；这条轨迹里只有三条 assistant 消息。

这些轨迹会教给模型四件事：怎样写出当前 harness 能真正执行的工具调用；对话变长时怎样继续（因为训练轨迹本身就长）；工具结果长什么样，所以模型会等待工具返回，而不是自己编造一个观测；以及数据里的其他一切——多余的评论、重复的命令、在多种解法都可行时只会其中一种。最后一条是全讲的伏笔。

[1] 教师强制（teacher forcing）：训练时每一步都以录制好的前缀为条件预测下一个 token，而不是以模型自己生成的前缀为条件；模仿学习（imitation learning）指直接拟合示范者的动作。两者合起来的含义是：SFT 只学「在示范走过的状态下该做什么」，从不经历自己走偏后的状态。

2.1.2　训练流水线与冷启动

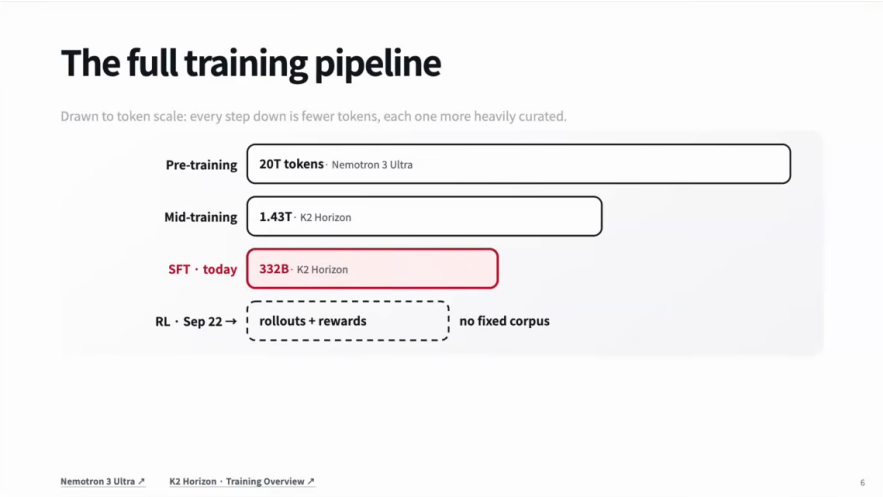


图 2: 按 token 量画的训练阶梯：预训练 20T（Nemotron 3 Ultra）、中期训练 1.43T（K2 Horizon）、SFT 332B（K2 Horizon），RL 没有固定语料。每往下一级 token 更少、筛选更重。注意三个数字来自两个不同的模型。[2]

讲者补充：中期训练（mid-training）的定义各家不同，泛指预训练与后训练之间的一切，甚至有人把 SFT 也算作中期训练；预训练、中期训练、SFT 都有固定语料，RL 没有，它在模型自己的 rollout 上学习。

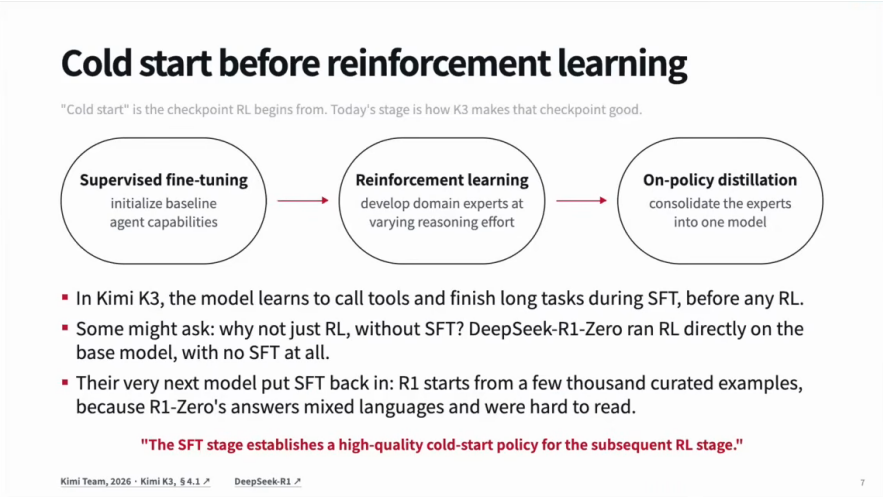


图 3: Kimi K3 的后训练三段：SFT（建立基础智能体能力）→ RL（在不同推理强度下培养领域专家）→ on-policy 蒸馏（把专家合并成一个模型）。下方是 DeepSeek-R1-Zero 与 R1 的对比。[3]

冷启动（cold start）指 RL 起步时的 checkpoint，本讲这一阶段决定它的质量。K3 在 SFT 阶段——任何 RL 之前——就学会调用工具和完成长任务。对「为什么不直接做 RL」这个问题，讲义用 DeepSeek 的经历作答：R1-Zero 直接在基座模型上做 RL、没有任何 SFT；下一个模型 R1 又把 SFT 加了回来，从几千条精选样本开始，因为 R1-Zero 的回答混杂多种语言、难以阅读。K3 技术报告的原话是 “The SFT stage establishes a high-quality cold-start policy for the subsequent RL stage.”

[2] 视频画面时间区间：第 8 讲 00:04:07–00:05:19（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。
[3] 视频画面时间区间：第 8 讲 00:05:20–00:07:15（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

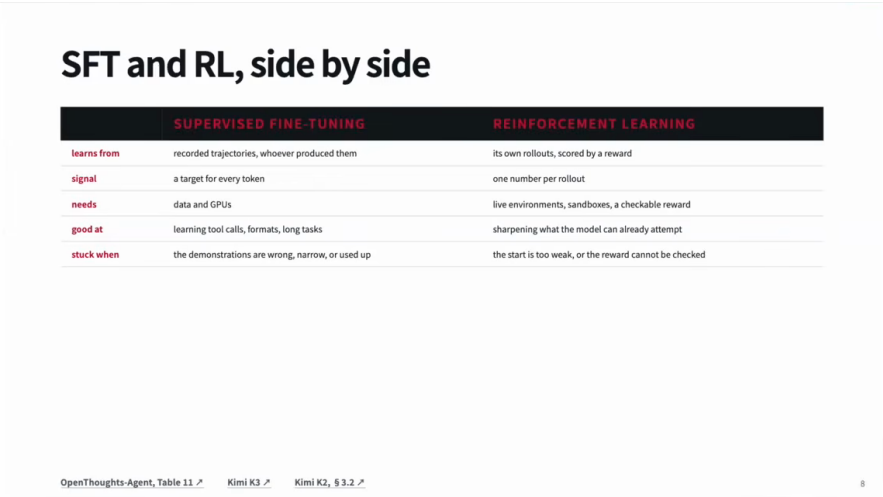


图 4: SFT 与 RL 的五维对比：学什么（录制轨迹 vs 自己的 rollout）、信号（每个 token 一个目标 vs 每条 rollout 一个数）、需要什么（数据与 GPU vs 在线环境、沙箱、可校验的奖励）、擅长什么、什么时候卡住。[4]

2.2　轨迹作为 token 序列

2.2.1　chat template 与损失掩码

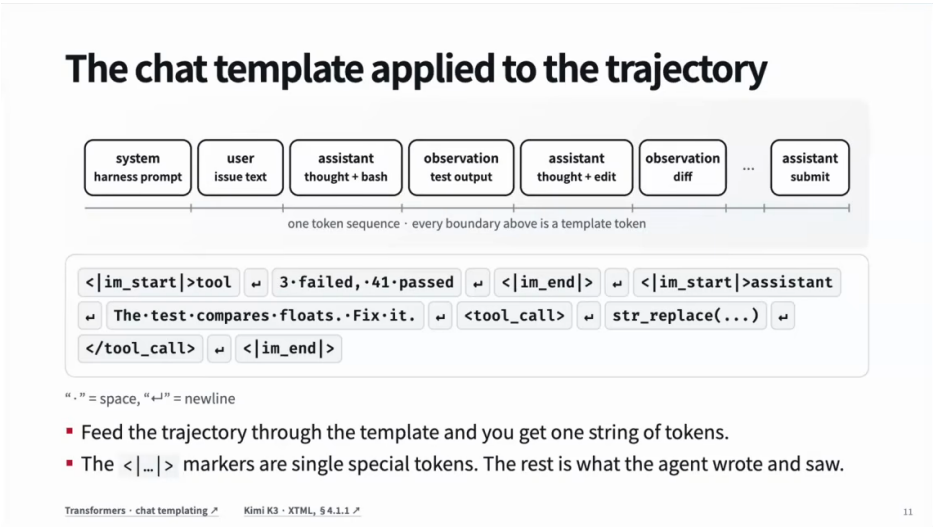


图 5: 把轨迹送进 chat template 得到一条 token 串。<|…|> 标记是单个特殊 token，其余是智能体写下和看到的内容（“·”为空格，“↵”为换行）。[5]

每个模型的 chat template 不同；一条包含十次工具调用、最后把答案交给 verifier 的轨迹，整条被渲染成一个字符串送进训练框架。与普通语言模型训练不同，智能体 SFT 不在整条轨迹上算损失：

[4] 视频画面时间区间：第 8 讲 00:07:16–00:08:24（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。
[5] 视频画面时间区间：第 8 讲 00:08:39–00:09:51（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

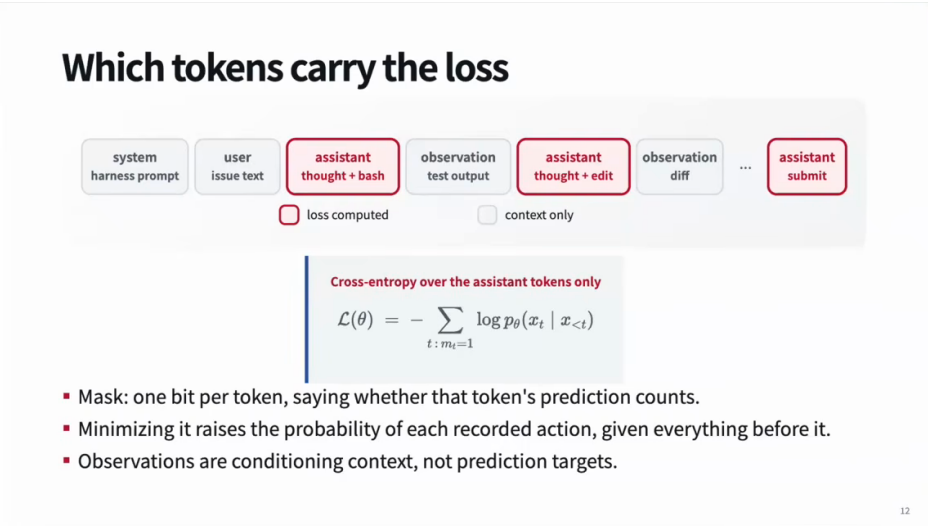


图 6: 哪些 token 计入损失：只有 assistant 消息（红框）计入，system、user、observation（灰框）只作上下文。[6]

$$\mathcal{L}(\theta) = -\sum_{t:\, m_t=1} \log p_\theta(x_t \mid x_{<t}) = -\sum_{t=1}^{T} m_t \log p_\theta(x_t \mid x_{<t})$$

- $x_{1:T}$：整条轨迹经 chat template 渲染、分词后的 token 序列（system、user、observation、assistant 全在其中）；
- $x_{<t}$：位置 t 之前的全部 token，作为条件；p_θ：参数为 θ 的语言模型；
- $m_t \in \{0, 1\}$：损失掩码，「每个 token 一个比特，表示该 token 的预测是否计入损失」；assistant 片段内（含工具调用与停止 token、不含角色头）为 1，其余为 0。

在 HF 风格的训练代码里，这对应 `labels = input_ids.clone(); labels[m == 0] = -100`，模型内部把 `logits[:, :-1]` 与 `labels[:, 1:]` 对齐，`CrossEntropyLoss(ignore_index=-100)` 跳过 -100 的位置；框架默认对 batch 内所有 $m_t = 1$ 的 token 取平均。观测是条件上下文，不是预测目标。

课堂问答：为什么不在观测上也训练？(00:11:08)

讲者回答：只在 assistant 消息上训练是智能体 SFT 的经验法则，因为观测由用户或环境给出，不是 assistant 要预测的；但确实有论文在观测上训练并发现有助于预测世界状态，这里讲的是通用做法，具体场景应自己做消融。在场的另一位讲师补充了 Meta 的 Code World Model：训练模型预测代码执行的结果，在需要理解执行语义的任务上很有帮助，但会占用模型容量，小模型可能因此损害其他能力。第 11 讲结尾的问答又回到这个问题（第 4 章）。

掩码的生成分三步：Render（模板把对话写成一个字符串，同时记录每个 assistant 片段的字符起止位置）→ Tokenize（字符位置映射到 token 位置）→ Mask（assistant 片段内的 token 记 1，其余记 0）。讲者的总结是 “The trainer sees the string and the bits and nothing else”（00:13:40）。在 HF Transformers 中，对应 `apply_chat_template(..., return_assistant_tokens_mask=True)`，

[6] 视频画面时间区间：第 8 讲 00:09:52–00:12:52（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

要求模板用 `{% generation %}...{% endgeneration %}` 标出 assistant 片段。三个常用框架的开关不同，但做的是同一个决定：

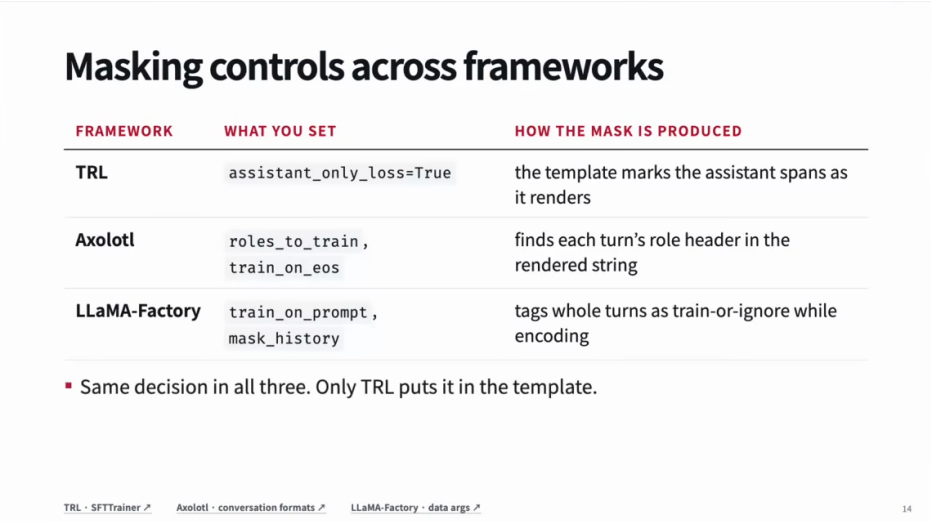


图 7：三个框架的掩码开关：TRL 的 `assistant_only_loss=True`（模板渲染时标出 assistant 片段）、Axolotl 的 `roles_to_train` 与 `train_on_eos`（在渲染后的字符串里找角色头）、LLaMA-Factory 的 `train_on_prompt` 与 `mask_history`（编码时把整轮标为训练或忽略）。[7]

讲者本人做智能体 SFT 时偏好 LLaMA-Factory，并当场展示了它的 GitHub 主页：生成 rollout 或合成轨迹 → 放进框架 → 指定要训练的模型 → 开始训练，可以选择训练整条轨迹、在某处截断、或也在上下文上训练。讲义备用页（视频未展示）给出了各开关的精确语义：TRL 的 `assistant_only_loss` 需要模板含 `{% generation %}` 标记；Axolotl 默认 `roles_to_train: ["assistant"]`，`train_on_eos: turn` 让每个被训练回合的 EOS 计入损失；LLaMA-Factory 默认 `train_on_prompt: false`，`mask_history: true` 只训练最后一轮。

2.2.2　回合结束符与推理块

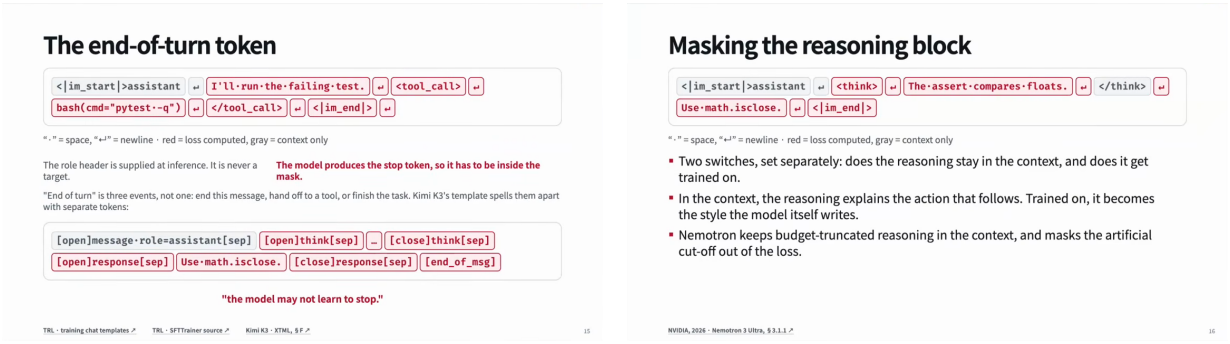
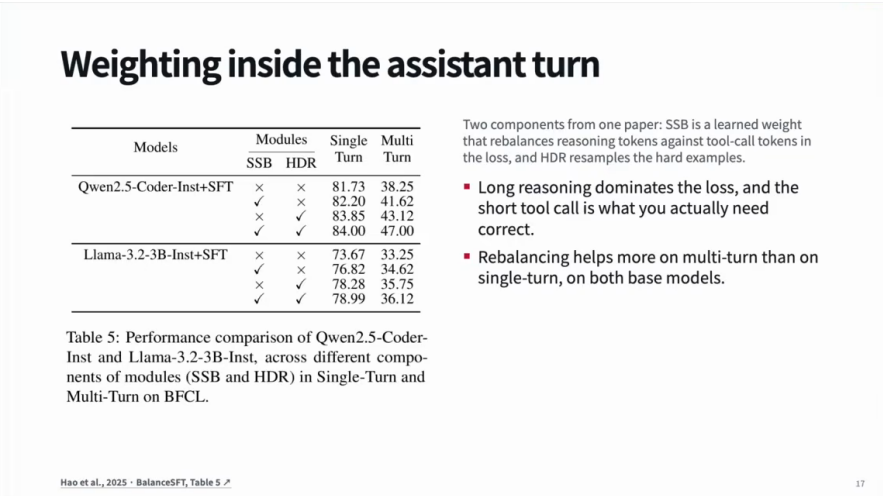


图 8：左：回合结束 token 必须在掩码内——角色头 `<|im_start|>assistant` 在推理时由系统提供，永远不是训练目标；停止 token 由模型自己生成。Kimi K3 的模板用不同 token 区分「结束这条消息」「交给工具」「完成任务」三种回合结束。右：推理块的掩码有两个独立开关：推理是否留在上下文里、是否在推理上训练。[8]

[7] 视频画面时间区间：第 8 讲 00:13:44–00:15:09（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。
[8] 视频画面时间区间：第 8 讲 00:15:10–00:18:10（均为幻灯片区域裁剪图）。

TRL 文档对漏掉停止 token 的后果写得很直白："the model may not learn to stop." 讲者补充：Qwen 的回合结束符是 `<|im_end|>`，Kimi、Llama 各不相同，训练前要查看目标模型的 chat template；她观察到的最常见错误是混淆「计入损失」和「仅上下文」——assistant 角色头不应训练。推理块留在上下文里时，它解释其后的动作；被训练时，它成为模型自己的写作风格。Nemotron 3 Ultra 把按预算截断的推理保留在上下文中，并把人为截断处从损失里掩掉。

2.2.3　token 级加权与样本—token 失衡



Models	Modules		Single Turn	Multi Turn
	SSB	HDR		
Qwen2.5-Coder-Inst+SFT	×	×	81.73	38.25
	✓	×	82.20	41.62
	×	✓	83.85	43.12
	✓	✓	84.00	47.00
Llama-3.2-3B-Inst+SFT	×	×	73.67	33.25
	✓	×	76.82	34.62
	×	✓	78.28	35.75
	✓	✓	78.99	36.12

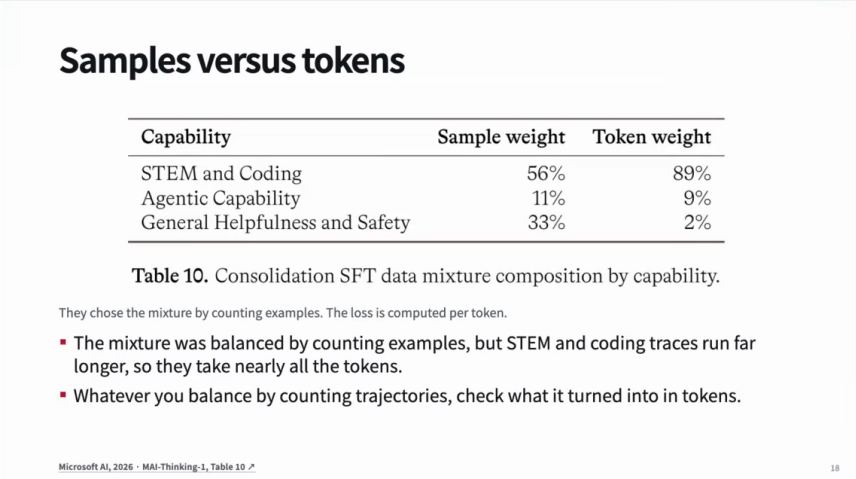
Table 5: Performance comparison of Qwen2.5-Coder-Inst and Llama-3.2-3B-Inst, across different components of modules (SSB and HDR) in Single-Turn and Multi-Turn on BFCL.

图 9: BalanceSFT（Hao 等，2025）表 5：SSB 是学到的权重，在损失中重新平衡推理 token 与工具调用 token；HDR 对困难样本重采样。BFCL 基准上，Qwen2.5-Coder 单轮 81.73→84.00、多轮 38.25→47.00；Llama-3.2-3B 单轮 73.67→78.99、多轮 33.25→36.12。[9]

长推理主导损失，而短的工具调用才是真正需要正确的部分。讲义结论「重平衡在多轮上的提升大于单轮」对 Qwen 成立（多轮 +8.75 vs 单轮 +2.27）；对 Llama 只在相对增幅上成立（绝对增幅单轮 +5.32 反而大于多轮 +2.87，相对增幅 7.2% vs 8.6%）。

同样的失衡也发生在数据集层面。MAI-Thinking-1 按样本数决定数据配比，但损失按 token 计算：

[9] 视频画面时间区间：第 8 讲 00:18:11–00:19:54（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。



Samples versus tokens

Capability	Sample weight	Token weight
STEM and Coding	56%	89%
Agentic Capability	11%	9%
General Helpfulness and Safety	33%	2%

Table 10. Consolidation SFT data mixture composition by capability.

They chose the mixture by counting examples. The loss is computed per token.

- The mixture was balanced by counting examples, but STEM and coding traces run far longer, so they take nearly all the tokens.
- Whatever you balance by counting trajectories, check what it turned into in tokens.

Microsoft AI, 2026 · MAI-Thinking-1, Table 10 ↗　18

图 10：MAI-Thinking-1 表 10：巩固阶段 SFT 数据配比。STEM 与代码占 56% 的样本、89% 的 token；智能体能力 11% / 9%；通用有用性与安全 33% / 2%。[10]

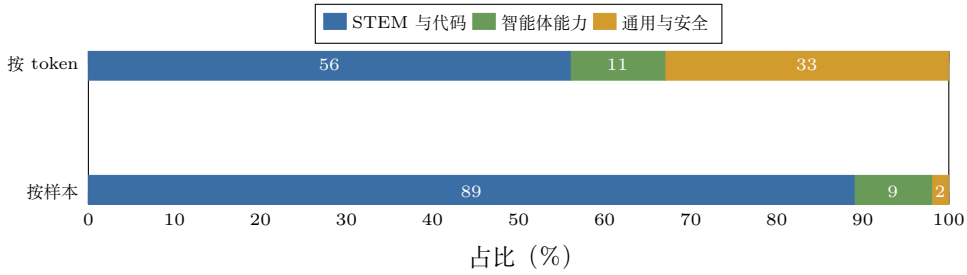


图 11：把 MAI 表 10 画成堆叠条（数据取自讲义）。由占比可反推各类平均每条样本的损失 token 数之比：STEM : 智能体: 通用 $\approx 89/56 : 9/11 : 2/33 = 1.59 : 0.82 : 0.061$，一条 STEM / 代码样本的 token 数约是通用样本的 26 倍（本笔记作者计算）。

这个比例的来源可以写成一个式子：按 token 平均的损失里，样本 i 对梯度的权重正比于它的损失 token 数 $n_i = \sum_t m_{i,t}$，所以类别 c 的 token 占比为

$$\tau_c = \frac{p_c \, \bar{n}_c}{\sum_{c'} p_{c'} \bar{n}_{c'}}$$

- p_c：类别 c 的样本占比（按条数配比的结果）；$\bar{n}_c$：该类样本的平均损失 token 数；
- τ_c：该类在梯度中的实际份额。讲义的建议是：无论按轨迹数平衡成什么样，都要检查它在 token 上变成了什么。

讲者在问答中回翻此页补充：MAI 的 56% 是通过实验确定的，一般做法是先在较小比例、较小模型上试，再扩大规模。

一个尚未有人做的实验

讲义最后一页列出的开放问题：没有找到公开实验在智能体 SFT 中改变掩码并测量效果；仔细的掩码实验都是单轮指令微调（WIT、Instruction Modelling）。唯一一个「取消 prompt 掩码

[10] 视频画面时间区间：第 8 讲 00:19:55–00:21:24（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

有帮助」的案例具备长 prompt、短回答、少数据三个条件，而智能体轨迹在这三点上全部相反。讲义原话："At 8B scale this experiment is cheap, and it is sitting there unrun."

2.3　选择轨迹

2.3.1　轨迹从哪里来

谁执行任务	全部保留	只保留通过的 run
更强的模型	普通蒸馏	SWE-Gym、Nemotron、OpenThoughts-Agent
模型自己	MAI（用自己的推理 run）	专家迭代（第 9 讲的 ReST）

大多数智能体 SFT 数据位于右上格：更强模型写出的轨迹优于学生模型能写的，而任务自带的测试让「只留好轨迹」的成本很低。下排是模型自己生成训练数据，循环迭代即专家迭代。

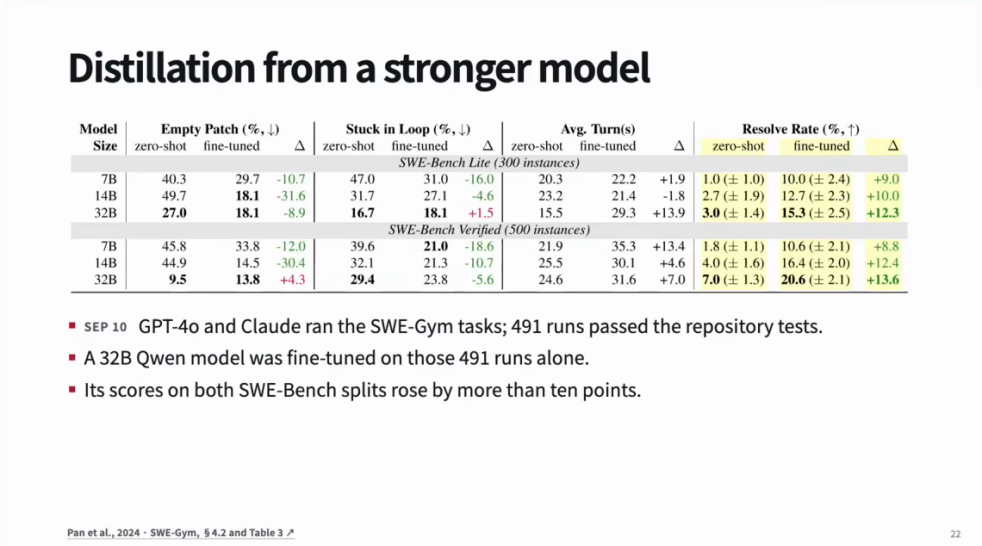


图 12: SWE-Gym（Pan 等，2024）表 3：GPT-4o 与 Claude 跑 SWE-Gym 任务，491 条 run 通过仓库测试；只用这 491 条微调 Qwen。32B 模型在 Lite 上解决率 3.0 → 15.3（+12.3），在 Verified 上 7.0 → 20.6（+13.6）。[11]

讲义说「两个 SWE-Bench 划分都提高了十分以上」，这只对 32B 成立（14B 为 +10.0/ + 12.4，7B 为 +9.0/ + 8.8）。表中另外两列也值得看：微调大幅降低了空补丁率与「陷入循环」的比例（例如 14B 在 Verified 上空补丁 44.9% → 14.5%），但 32B 的空补丁率（Verified）和陷入循环率（Lite）反而略有上升。

[11] 视频画面时间区间：第 8 讲 00:24:35–00:25:05（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

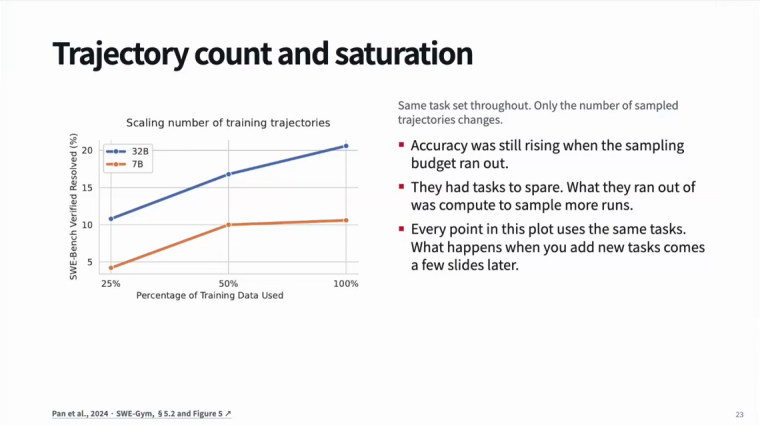


图 13: SWE-Gym 图 5：任务集不变，只改变采样轨迹数（25% / 50% / 100%）。32B 一直上升（约 10.8 → 16.8 → 20.6），7B 在 50% 后趋平（约 4.2 → 10.0 → 10.6）。[12]

讲义的解读：采样预算用完时 32B 的准确率仍在上升——他们不缺任务，缺的是继续采样的算力；讲者补充小模型更早饱和。

2.3.2　成功的 run 里也有坏习惯

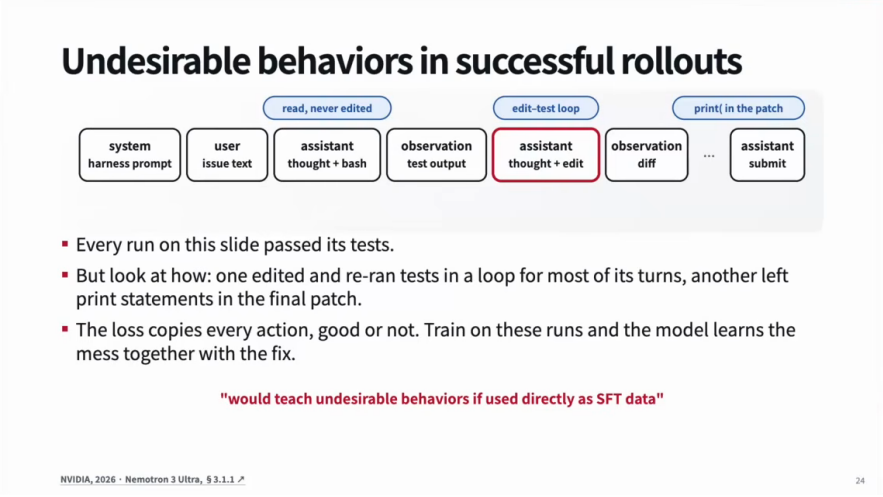


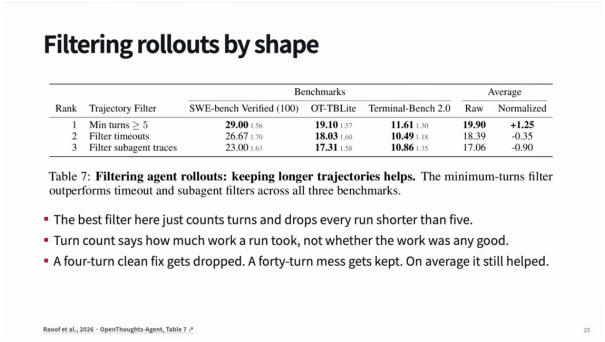
图 14: 本页每条 run 都通过了测试，但一条在大部分回合里反复「编辑—测试」，另一条在最终补丁里留下 `print` 语句。[13]

“The loss copies every action no matter whether it’s good or not”（00:27:13）：在这些 run 上训练，模型会把不良行为和修复一起学会。Nemotron 3 Ultra 技术报告的原话是 “would teach undesirable behaviors if used directly as SFT data”。

2.3.3　OpenThoughts-Agent 的四组消融

接下来四页都来自 OpenThoughts-Agent（Raoof 等，2026），在三个基准上评测：SWE-bench Verified 的 100 题子集、OT-TBLite、Terminal-Bench 2.0。

[12] 视频画面时间区间：第 8 讲 00:25:06–00:26:08（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。
[13] 视频画面时间区间：第 8 讲 00:26:09–00:27:23（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。



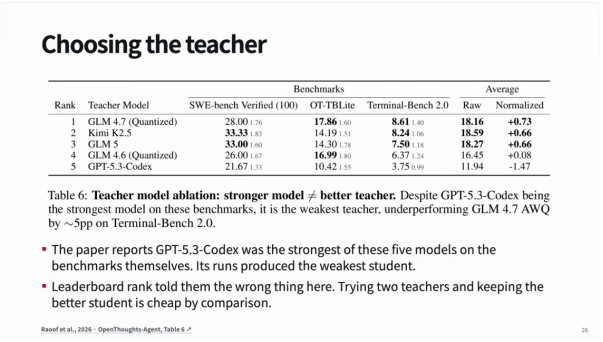
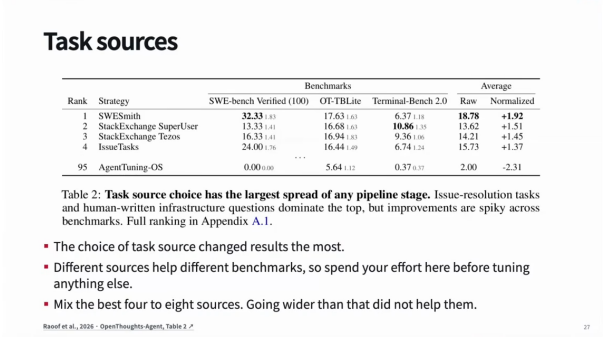


图 15: 左：表 7，轨迹过滤。最好的过滤器只是数回合数、丢掉少于 5 轮的 run（三基准 29.00 / 19.10 / 11.61，平均 19.90）。右：表 6，教师选择。GPT-5.3-Codex 在基准上最强，但用它的 run 训练出的学生最弱（平均 11.94，归一化 −1.47），比 GLM 4.7 在 Terminal-Bench 2.0 上低约 5 个点。[14]

按回合数过滤看起来粗糙：回合数反映 run 做了多少工作，不反映工作质量，4 轮的干净修复会被丢掉，40 轮的混乱 run 会被保留——但平均而言它仍然有帮助。讲者的解释是，短轨迹可能是一次就做对（学不到错误恢复）、轨迹本身错误、或生成时遇到错误提前结束。教师选择则说明排行榜名次给出了错误信号；有学生问原因，讲者答：强模型擅长自己解题，但它的思考方式和解题路径未必适合学生模仿，试两个教师、保留更好的学生，成本相对很低。



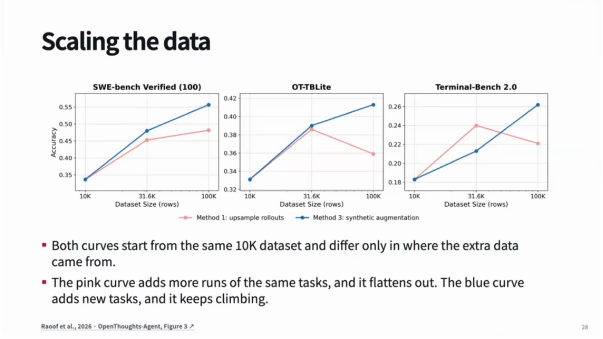


图 16: 左：表 2，任务来源是所有流水线环节里影响最大的（SWE-Smith 平均 18.78 排第一，AgentTuning-OS 只有 2.00 排第 95）。右：图 3，从同一个 10K 数据集出发：粉线对同一批任务多采 run，趋平甚至下降；蓝线加入合成的新任务，持续上升。[15]

讲义的建议是 “spend your effort here before tuning anything else”（00:32:07）：不同来源帮助不同基准（SWE-Smith 对 SWE-bench Verified 贡献 32.33，对 Terminal-Bench 只有 6.37，低于 StackExchange SuperUser 的 10.86），混合最好的 4–8 个来源，更宽并没有帮助。数据规模方面，从图上读出的 SWE-bench Verified 准确率：10K 时 0.335；100K 时「多采」0.482、「新任务」0.557——扩充数据时应扩充任务分布，而不是同一批任务的采样数。

课堂讨论：三条 run 留哪些？(00:34:06–00:37:29)

同一任务返回三条 run：A 四轮解决；B 做了错误编辑、看到测试失败、然后恢复；C 在 11 次无效尝试后解决。一位学生选 B，理由是错误恢复更有价值；讲者追问 B 的缺点——还可能学到错误编辑本身。另一位学生主张 A；讲者认为 A 从一开始就是正确行为，但学不到恢复。她

[14] 视频画面时间区间：第 8 讲 00:27:24–00:31:12（均为幻灯片区域裁剪图）。
[15] 视频画面时间区间：第 8 讲 00:31:13–00:34:05（均为幻灯片区域裁剪图）。

的结论是 “the best answer is to incorporate both A and B”。讲者没有回答第三问（最少回合过滤器会判错哪个），按表 7 的规则可以推出：≥ 5 轮的过滤器会丢掉干净的 A、保留 11 次无效尝试的 C——正是表 7 一页说的「干净修复被丢、混乱 run 被留」。问答中她还补充：可以在错误的中间步骤上监督（这是学会恢复的方式），但最终结果必须正确。

2.4　统一多个数据集的格式：Agent Data Protocol

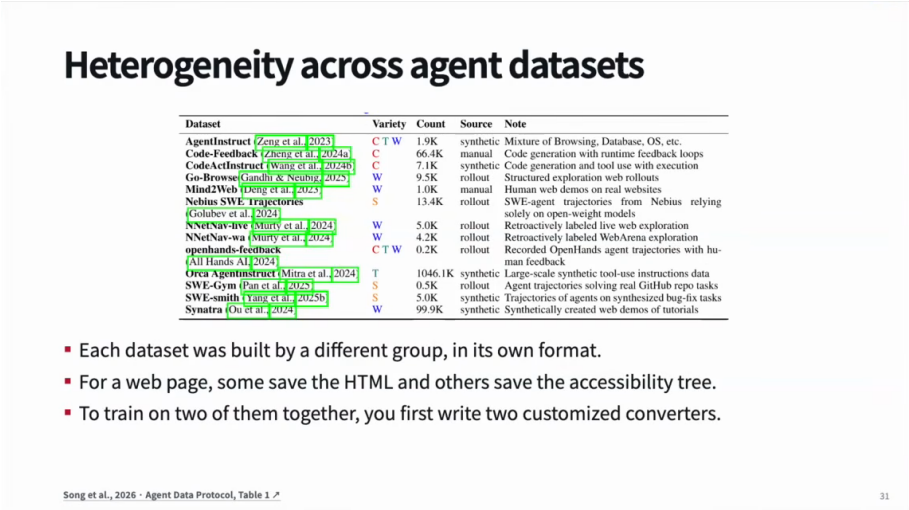


图 17: Agent Data Protocol 表 1：13 个智能体数据集各自由不同团队按不同格式构建（合计约 126 万条，其中 Orca AgentInstruct 占 83%）。网页观测有的存 HTML，有的存可访问性树。[16]

这是讲者本人与 Graham Neubig 的工作（ICLR oral）。她的例子：Go-Browse 用可访问性树，Mind2Web 用 HTML，要在两者上训练 OpenHands 智能体，需要写 Go-Browse→OpenHands 与 Mind2Web→OpenHands 两个转换器。ADP 的做法是每个数据集读一次，改写为类型化的动作与观测：

[16] 视频画面时间区间：第 8 讲 00:37:48–00:39:11（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

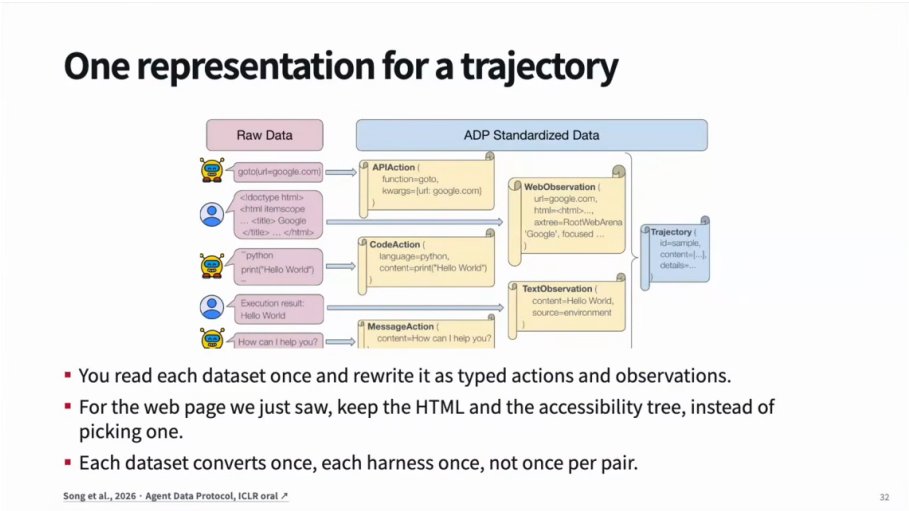


图 18: ADP 的统一表示：`APIAction`、`CodeAction`、`MessageAction` 等动作与 `WebObservation`（同时保留 HTML 与可访问性树）、`TextObservation` 等观测，汇总为一个 `Trajectory`。[17]

转换器数量因此从「数据集数 D × harness 数 H」降为 $D + H$：以表 1 的 13 个数据集和下一步提到的 3 个 harness 为例，是 39 个对 16 个。数据统一后还要写回目标 harness 的格式：OpenHands、SWE-Agent、AgentLab 的动作空间不同，同一条轨迹在三者中呈现不同；这一步也决定 system prompt 与上下文过长时的处理，最后检查工具调用能否解析、每次调用是否带思考、对话结尾是否规范。

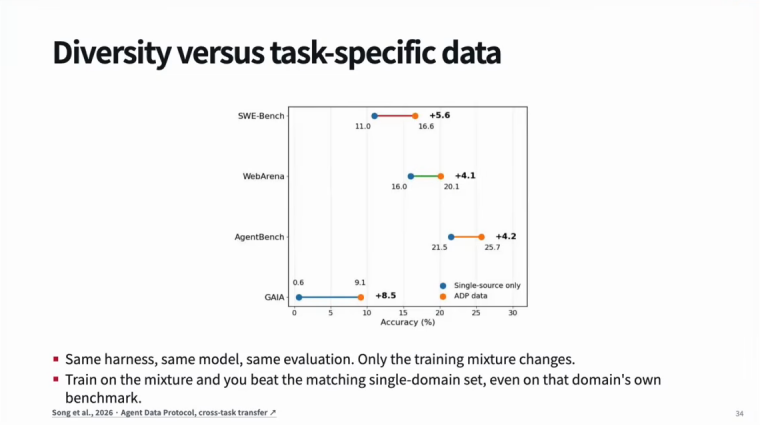


图 19: 同一 harness、同一模型、同一评测，只改训练混合：用 ADP 混合数据训练优于对应的单领域数据，即使在该领域自己的基准上（SWE-Bench 11.0 → 16.6，WebArena 16.0 → 20.1，AgentBench 21.5 → 25.7，GAIA 0.6 → 9.1）。[18]

讲者把它与前面 OpenThoughts-Agent 的结论对照：后者比较的是逐个任务来源单独训练的效果；ADP 比较的是多来源混合与单一来源，几乎每个智能体基准上多来源都更高。

[17] 视频画面时间区间：第 8 讲 00:39:12–00:40:32（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。
[18] 视频画面时间区间：第 8 讲 00:41:21–00:42:44（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

2.5　运行训练

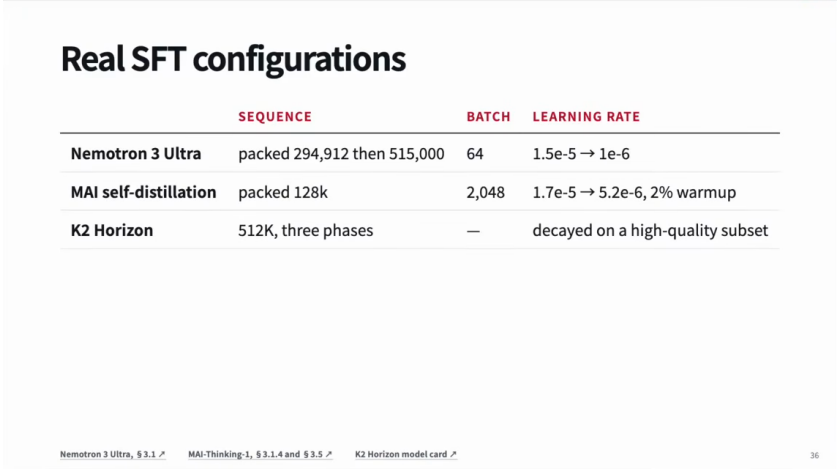
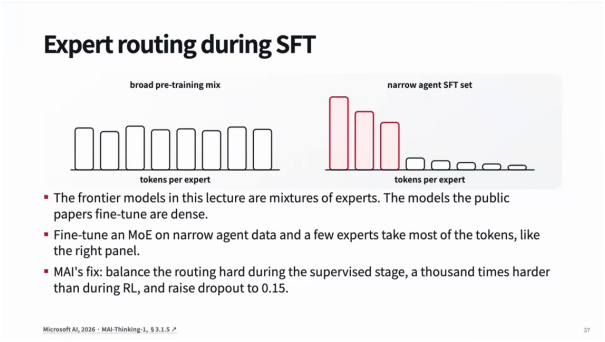


图 20: 三家前沿模型的真实 SFT 配置：Nemotron 3 Ultra 打包序列长 294,912、之后 515,000，batch 64，学习率 $1.5\times10^{-5}\to10^{-6}$；MAI 自蒸馏打包 128K，batch 2,048，学习率 $1.7\times10^{-5}\to5.2\times10^{-6}$、2% warmup；K2 Horizon 512K 序列、三个阶段、在高质量子集上衰减学习率。[19]

若把 batch 理解为每步打包序列数，Nemotron 每步约 $64\times294{,}912\approx1.9\times10^{7}$ 个 token，MAI 约 $2{,}048\times131{,}072\approx2.7\times10^{8}$ 个（本笔记作者计算，讲义未说明 batch 的确切含义）。有学生问这算不算参数高效微调，讲者答这是全参数训练，本质上与预训练相同，区别在于损失掩码；问答中她补充，资源充足的实验室偏好全参数微调，她本人为了效率也用过 LoRA[20]。

2.5.1　MoE 的专家路由、打包与模板漂移



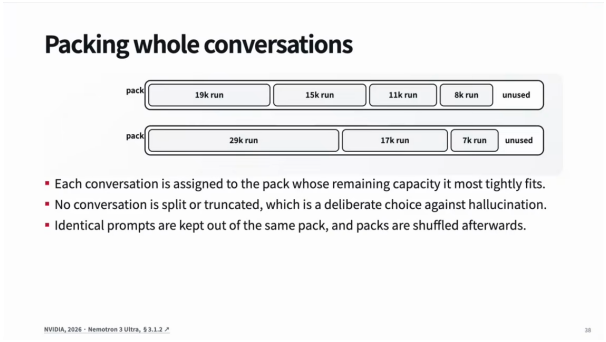


图 21: 左：在窄的智能体数据上微调混合专家模型，少数专家会吃下大部分 token（示意图）；MAI 的对策是在监督阶段把路由均衡强度提高到 RL 阶段的 1000 倍，并把 dropout 提到 0.15。右：Nemotron 的整对话打包——每条对话放进剩余容量最紧贴它的 pack，不切分、不截断，相同 prompt 不放进同一 pack，打包后再打乱。[21]

[19] 视频画面时间区间：第 8 讲 00:42:51–00:44:13（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

[20] LoRA（Low-Rank Adaptation）：冻结原权重 W，只训练低秩增量 $\Delta W=BA$（$B\in\mathbb{R}^{d\times r}$，$A\in\mathbb{R}^{r\times k}$，$r\ll d,k$）。rank $r=32$ 时 4B 模型的可训练参数只有千万量级。第 6 章的作业就用 LoRA rank 32 训练 Qwen3.5-4B。

[21] 视频画面时间区间：第 8 讲 00:44:14–00:46:18（均为幻灯片区域裁剪图）。

本讲涉及的前沿模型都是混合专家（MoE[22]）模型，而公开论文微调的多是 dense 模型。打包（packing）是把多条对话装进一个固定长度的训练序列，而不是把大部分空间填充（padding）掉；best-fit 规则写成式子是

$$j^\star = \arg\min_{j:\, r_j \geq \ell_i} (r_j - \ell_i)$$

- ℓ_i：第 i 条对话的 token 长度；r_j：第 j 个 pack 的剩余容量；没有能容纳的 pack 时新开一个；
- 讲义强调不切分、不截断任何对话是有意为之："a deliberate choice against hallucination"（00:46:03）——被截断的轨迹会教模型在不完整的上下文上继续生成。

模板漂移：训练模板必须与服务模板逐字一致

训练用的模板必须和服务时用的模板一致，精确到空白字符。同一条轨迹渲染两次——一次后面接 tool 消息、一次不接——可能得到两个不同的字符串：追加 tool 消息时，带条件判断的 thinking 块会改变前面那个 assistant 回合，前缀不再匹配。讲义引用 Transformers 文档："The chat template should always match the format the model was trained with." 讲者称这正是它在工具循环中致命的原因。第 12 讲的「前缀扩展」与作业里 `preserve_thinking_history` 开关讨论的是同一个现象（第 5、6 章）。

讲义最后推荐读公开的训练记录：K2 Horizon 的 model card 有逐阶段训练表（步数、token 数、序列长度、各阶段目的），链接了 Weights & Biases 的真实 loss 曲线，并发布了每个阶段的中间 checkpoint；Nemotron 发布了后训练 checkpoint、训练数据与配方。

2.6　怎样知道训练有效

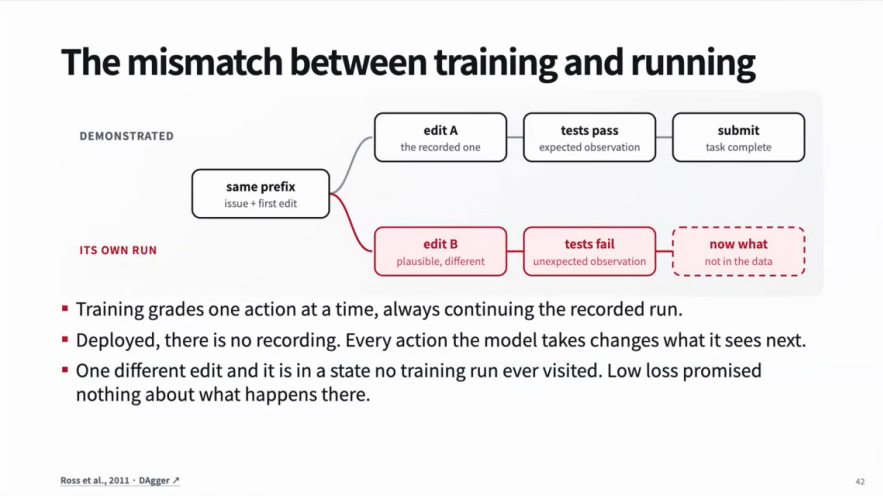


图 22：训练与运行的不一致：训练每次只评一个动作，并且总是接着录制好的 run 往下走；部署时只要一次不同的编辑（edit B），模型就进入任何训练 run 都没访问过的状态（"now what"）。[23]

[22] MoE（Mixture of Experts，混合专家）：每层有多个前馈「专家」，由路由器为每个 token 选择少数几个激活；参数总量大而每 token 计算量小。路由需要辅助的负载均衡损失，否则 token 会集中到少数专家。

[23] 视频画面时间区间：第 8 讲 00:48:48–00:50:35（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

讲者的说法是 “training continues the recording and deployment continues itself”；低 loss 对那些没见过的状态没有任何承诺。这就是第 9 讲所说的暴露偏差，讲义引用的经典处理是 DAgger[24]。随后几页给出评测纪律：

1. **先能过拟合 20 个样本**：loss 降不下来说明问题在上游（数据、掩码、模板）；
2. **在目标 harness 中、在结果可校验的任务上评测**；谨慎阅读已发表的数字，有时它们是多个 harness 中的最好值；
3. **划分匹配结论**：同一任务的所有 run 放在同一侧；要声称能泛化到新仓库，就按仓库划分；
4. **跨 harness 的鲁棒性**：在一个系统里收集的轨迹会连同任务一起教会这个系统的约定，用 OpenHands 收集的轨迹不一定能迁移到 Claude Code 或 Codex，所以实验室跨多个 harness 收集；
5. **目标领域之外的退化**：在一种能力上训练会改变其他能力，数据混合是控制手段，按样本平衡不等于按 token 平衡，所以也要评测不打算提升的能力——讲者的说法是不要只在一个领域上 “benchmaxing”。

2.7　交给 RL：SFT 该训练到什么程度

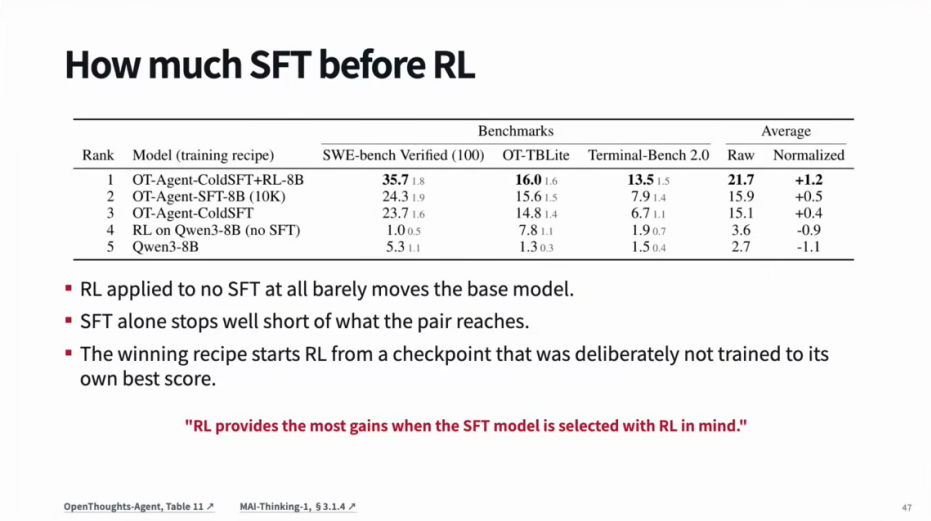


图 23: OpenThoughts-Agent 表 11：不做 SFT 直接 RL 几乎不动基座模型；只做 SFT 远不及 SFT + RL；最好的方案从一个有意没有训到自身最佳分数的 checkpoint 开始 RL。[25]

[24] DAgger（Dataset Aggregation，Ross 等，2011）：让当前策略自己运行、收集它实际访问到的状态，请专家在这些状态上标注正确动作，再并入训练集重新训练。理论上，纯行为克隆的累计误差上界随任务长度 T 以 $O(T^2\epsilon)$ 增长，DAgger 降为 $O(T\epsilon)$（ϵ 为单步模仿误差）。第 11 讲的 on-policy 蒸馏是它在 LLM 上的对应物。

[25] 视频画面时间区间：第 8 讲 00:53:00–00:54:37（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

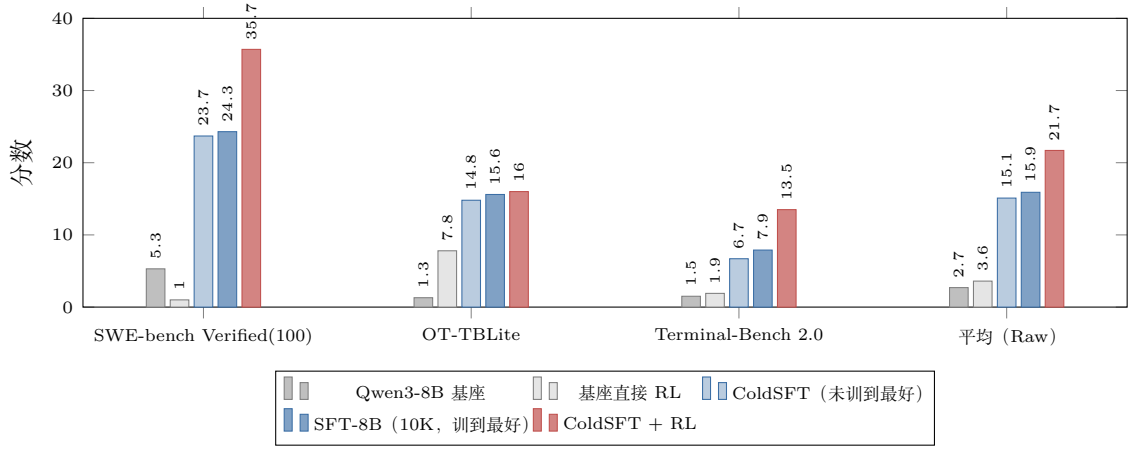


图 24: 把表 11 画成柱状图（数据取自讲义）。在 ColdSFT 上做 RL 带来 +12.0 / +1.2 / +6.8（平均 +6.6）；直接在基座上做 RL，平均只从 2.7 到 3.6，SWE-bench 反而从 5.3 降到 1.0。ColdSFT 单独看（15.1）不如训到最好的 SFT-8B（15.9），但它是更好的 RL 起点。

讲义引用 MAI 的总结："RL provides the most gains when the SFT model is selected with RL in mind." 讲者的观点是 SFT 是获得通用智能体能力最多的阶段，RL 用于获得特定领域能力或更好的推理，Nemotron 与 K2 Horizon 都有类似发现。问答中有学生问：如果没有更强的模型可以蒸馏怎么办？她答：MAI 选择自蒸馏；领先的闭源实验室更多依靠 RL 获得智能体能力，SFT 主要用于冷启动；"it's fundamentally limited by the teacher model's capability"（00:56:18）。另一位讲师补充了一个与第 9 讲直接相连的观点：RL 实际上在按正确性给样本加权——奖励越高梯度权重越大——后果是若模型犯错后恢复，RL 也会强化那些错误（"it's going to reinforce making those same mistakes again"，01:01:34）。

2.8　本章小结

- **机制**：轨迹经 chat template 变成一条 token 序列，交叉熵只算在 assistant 片段（含停止 token、不含角色头）上；推理块是否留在上下文、是否训练是两个独立开关。
- **数据**：大多数智能体 SFT 数据来自「更强模型执行 + 只留通过的 run」；成功的 run 也带坏习惯；任务来源的选择影响最大，扩充数据应扩充任务而非重复采样；更强的模型不一定是更好的教师；多来源混合优于单来源。
- **工程**：长序列打包、不截断；MoE 需要强力均衡路由；训练模板与服务模板必须逐字一致。
- **验证与交接**：先能过拟合 20 条，再在目标 harness 中评测；SFT 的价值最终由「它是不是好的 RL 起点」衡量。

视频未展开、但能把本讲与后续几讲连起来的几点：

1. **SFT 与策略梯度共用同一段代码**。本讲的 $\mathcal{L} = -\sum_t m_t \log p_\theta(x_t \mid x_{<t})$ 与第 9 讲的 $\mathcal{L}_{\text{PG}} = -R(\tau)\sum_t m_t \log \pi_\theta(u_t \mid u_{<t})$ 只差一个权重；作业里 `to_tinker_datum` 的 `cross_entropy` 分支直接把优势当作逐 token 权重传给 SFT 损失。因此本讲关于掩码、回合结束符、模板一致性的所有细节，在 RL 阶段原样成立，而且出错的代价更大：RL 训练时 token 必须与采样时逐个对齐。

2.「**按 token 平均」是一个隐含的加权决定**。MAI 的样本—token 失衡、BalanceSFT 的推理—工具调用失衡、Dr.GRPO 的长度归一化（第 9 讲）讨论的是同一件事：损失如何在序列之间、token 之间分配权重。作业选择「按整个 batch 的动作 token 数归一化」，就是在这个问题上的一个明确选择。

3. **SFT 为什么必须先于 RL**。第 9 讲指出全失败的组没有梯度；表 11 里「基座直接 RL」几乎不动，正是因为基座在这些任务上几乎拿不到正奖励。冷启动 SFT 的作用就是把成功率抬到 GRPO 能产生信号的区间（第 6.8 节的 P_{deg} 曲线）。

2.9 拓展阅读

- Raoof 等，OpenThoughts-Agent，`arXiv:2606.24855`（本讲表 2/6/7/11 与图 3 的出处）。
- Song 等，Agent Data Protocol，`arXiv:2510.24702`。

3 第 9 讲：RL 基础——从 SFT 到策略梯度

本讲由 Daniel Fried 主讲（1:17:46）。开场他把三讲 RL 的分工讲清楚：本讲讲最基础的策略梯度（policy gradient）方法，下一讲 Graham Neubig 讲异策略与稳定化技巧，再下一讲 Apurva Gandhi 讲 RL 系统与训练超参数；并直接说明 Assignment 3 会在两个环境上做 RL——一个是本讲的贯穿示例 Number Search，另一个是「有点像 Minecraft 的合成环境」，即 TextCraft-Synth。全讲的推进方式是：先指出 SFT 训练智能体时的三种失配，再把交互形式化为轨迹与期望奖励，然后依次给出三种「按权重重放智能体自己动作」的方法——ReST（过滤后做 SFT）、REINFORCE（按轨迹奖励加权）、GRPO / Dr.GRPO（按组内相对奖励加权），中间用一个单参数两臂老虎机精确演示基线为什么能降方差而不引入偏差。

3.1 贯穿示例与 SFT 的三种失配

3.1.1 Number Search

隐藏整数在 1–16 之间，智能体最多猜 4 次；猜错时环境回复 higher 或 lower，猜中得奖励 1，4 次用完仍未猜中（timeout）得 0。讲义中的四条样例轨迹都来自隐藏数为 11 的同一个任务实例（task instance），蓝色成功、红色失败，这一配色贯穿全讲。

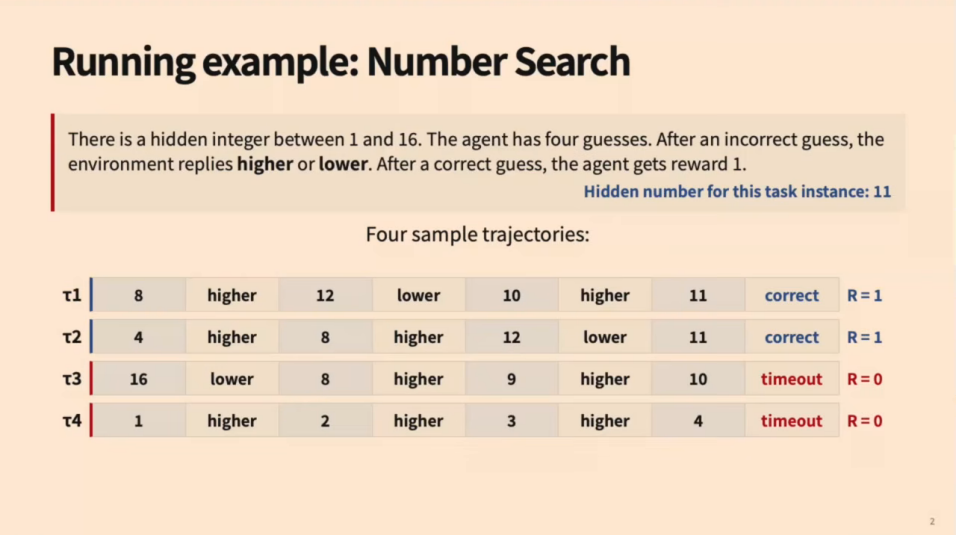


图 25: 贯穿示例 Number Search 的四条轨迹。τ_1 从最优首猜 8 开始二分；τ_2 首猜 4 次优，但仍利用反馈成功；τ_3 首猜 16 很差；τ_4 从 1 起逐个加 1。[26]

讲者把最优首猜 8 类比为 20 questions 或 Wordle 的「最优首猜」：按信息量把候选空间对半切。奖励是评价整条轨迹的标量；智能体场景里常见的是 SWE-bench 式的二值奖励（全部单元测试通过为 1），也可以是通过测试的比例。当前很多工作属于 RLVR[27]，奖励是 0/1，但本讲的方法对任意标量奖励都成立。

课堂版本与作业版本的差别

1–16 配 4 次猜测并不能保证必胜：4 次猜测最多区分 $2^4 - 1 = 15$ 个数，最优策略的胜率是 15/16（例如隐藏数为 16 时，$8 \to 12 \to 14 \to 15$ 之后已无机会）。作业版本把预算设为 $\lceil \log_2(\text{high} - \text{low} + 2) \rceil$，恰好保证完美二分必胜（第 6 章）。另外作业中的区间更大（例如 $[6, 988]$），动作写作 `<tool>guess(n)</tool>`，反馈为 “n is too low / too high”。

3.1.2　SFT 的目标与它的三种失配

SFT 训练智能体复制示范：在每个示范历史的条件下最大化示范动作 a_t^* 的概率。整段示范的对数概率等于各动作对数概率之和：

$$\mathcal{L}_{\text{SFT}} = -\sum_t \log \pi_\theta(a_t^* \mid h_t)$$

- π_θ：由语言模型参数 θ 驱动的策略；
- h_t：第 t 步之前的历史（prompt、之前的动作与观测）；
- a_t^*：示范给出的目标动作，可以来自人、更强的模型，或者——本讲的主题——智能体自己的成功轨迹。

[26] 视频画面时间区间：第 9 讲 00:02:29–00:06:10（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

[27] RLVR（Reinforcement Learning with Verifiable Rewards，可验证奖励的强化学习）：奖励由程序化的检查器（单元测试、答案比对等）给出，而不是由学习得到的奖励模型打分；优点是难以被「讨好」，缺点是只适用于结果可自动判定的任务。

讲者指出用 SFT 训练智能体有三种失配（mismatch）：

1. **任务失配（task mismatch）**：SFT 最大化的是示范动作的概率，我们真正想最大化的是完成任务的概率。两者相近但不同：完成任务的方式不止一种（τ_1 与 τ_2 都成功）；而且有些动作比其他动作更糟（在 “12 → lower” 之后绝不应该猜 13），SFT 却对所有「非示范动作」一视同仁。
2. **数据失配（data mismatch）**：希望也能从次优数据学。对 τ_3 这类超时轨迹，我们不想提高其中动作的概率，而是希望模型以后别这样走；SFT 没有这种机制。
3. **暴露偏差（exposure bias）**：SFT 在外部给定的历史上预测下一个动作，评估时历史却由模型自己生成。示范是 “8 higher 12 lower 10 higher 11”，生成时若走成 “8 higher **2**”（在 higher 之后猜了更小的数），模型就处在训练中从未见过的状态，可能「完全乱掉」。

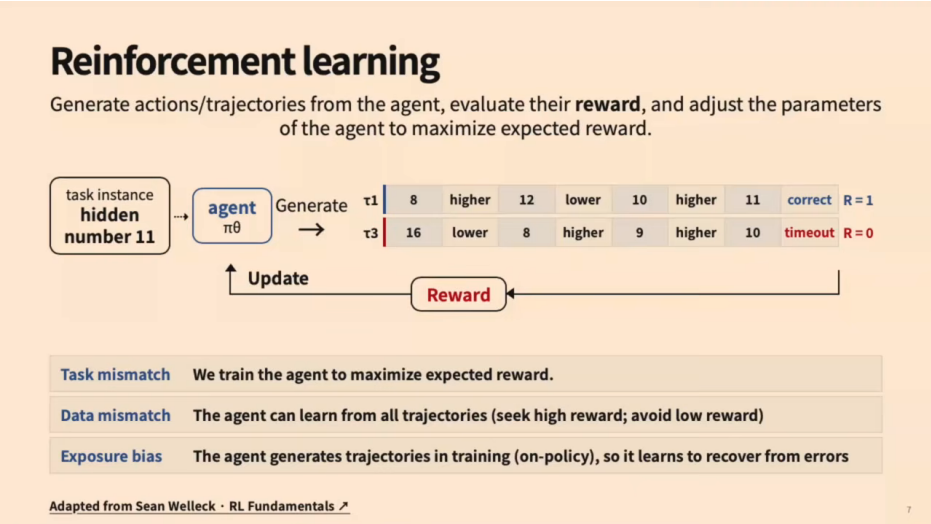


图 26: RL 的生成—评估—更新循环，以及它分别如何对应三种失配。讲者补充：几年前语言模型常见的重复、陷入循环等问题，在加入 RL 后训练阶段后大多得到缓解，因为模型学会了从自己的错误中恢复。[28]

RL 的思路是：让智能体像平时评估时那样生成多条轨迹，在环境中得到奖励，再调整参数使高奖励轨迹更可能、低奖励轨迹更不可能。它对三种失配的回答分别是：直接以期望奖励为目标；可以从所有轨迹学（趋近高奖励、远离低奖励）；训练轨迹由当前策略自己生成，因而学会从自身错误中恢复。最后一条性质有专门的名字，讲者原话是 “this property of having the model generate its own training data is called being on policy”（00:12:19）。on-policy（同策略）的代价是效率，第 11 讲会讨论放宽这一假设的异策略（off-policy）方法。

[28] 视频画面时间区间：第 9 讲 00:12:54–00:16:03（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

3.2　把交互写成轨迹

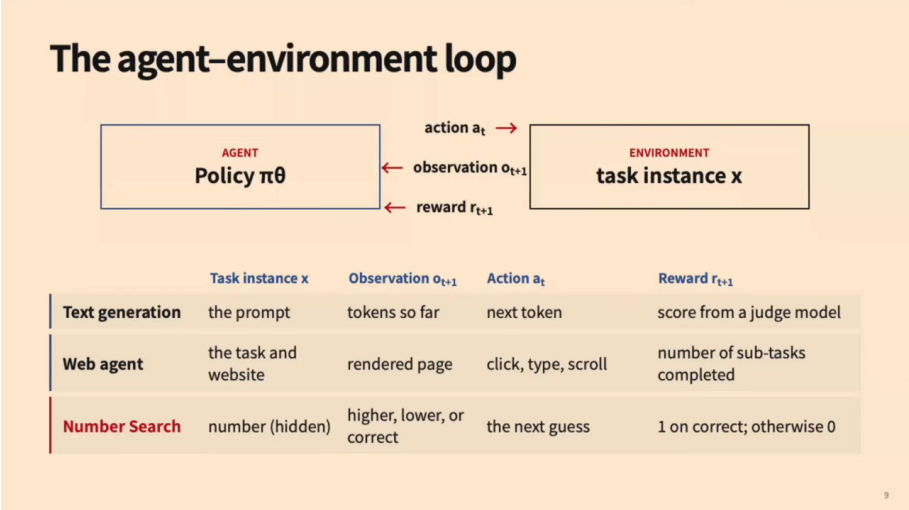


图 27: 智能体—环境循环与三种设置下的任务实例 x、观测 o_{t+1}、动作 a_t、奖励 r_{t+1}。Number Search 的隐藏数对智能体不可见，因此所有任务在开始时对智能体看起来都一样。[29]

讲者强调状态（state）与观测（observation）的区别：web 智能体的环境状态包括后台数据库等隐藏内容，智能体只能看到渲染出的页面；Number Search 的环境状态是隐藏数 11，智能体看不到。策略只以历史为条件：

$$h_t = (o_0, a_0, o_1, \ldots, o_t), \qquad \tau = (o_0, a_0, r_1, o_1, \ldots, a_{T-1}, r_T, o_T), \qquad R(\tau) = \sum_{t=1}^{T} r_t$$

- o_0：prompt；o_t：第 t 次观测（higher / lower / correct）；a_t：第 t 个动作（一次 `guess`）；
- r_t：动作后的奖励，Number Search 中 $r_1 = r_2 = r_3 = 0$、$r_4 = 1$（猜中时），所以 $R(\tau_1) = 1$；
- T：轨迹长度；$R(\tau)$：轨迹总奖励。

轨迹的概率由策略与环境两部分按链式法则逐步相乘：

$$p_\theta(\tau \mid x) = p(o_0 \mid x) \prod_{t=0}^{T-1} \pi_\theta(a_t \mid h_t)\, P(o_{t+1}, r_{t+1} \mid h_t, a_t, x)$$

- x：任务实例（如隐藏数 11）；P：环境的观测与奖励分布，未知，但与环境交互就是在从中采样；
- 讲义此页漏写了环境项前的 P，讲者在 00:21:04 与 00:23:56 两次口头更正，上式为补全后的形式。

[29] 视频画面时间区间：第 9 讲 00:16:18–00:18:51（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

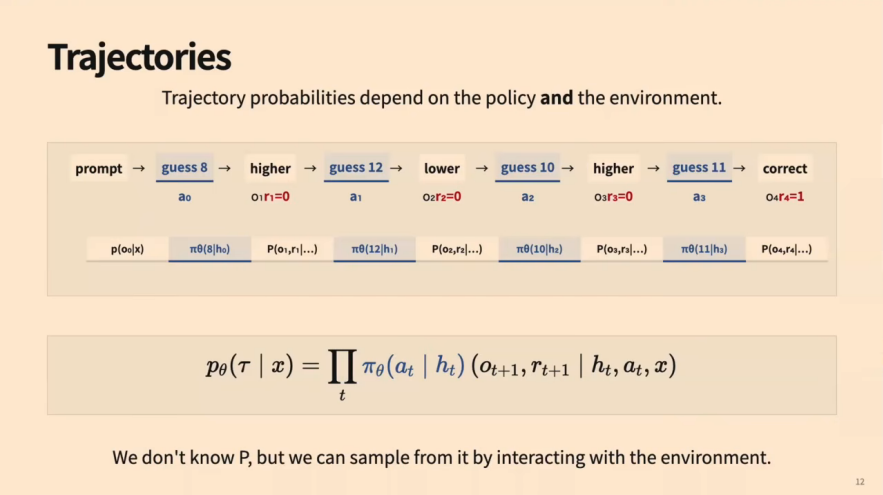


图 28: 轨迹概率的逐步分解（讲义原图，环境项前漏写 P）。[30]

有学生问：经典 RL 有马尔可夫假设，下一状态只依赖当前状态与动作，为什么这里要以整段历史为条件？讲者回答，这里处理的是部分可观测马尔可夫决策过程（POMDP[31]），底层状态不可见，所以下一个观测要以全部历史为条件。关键在于：RL 不需要知道 P——不必知道下一页 Amazon 页面以什么概率出现——只要能与环境交互采样。讲者说这 "is kind of magic in some sense, but it also makes reinforcement learning algorithms a bit slow because you kind of just have to like try stuff and see what happens"（00:22:30）。

3.2.1　目标：最大化期望奖励

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta(\cdot|x)}\big[R(\tau)\big] = \sum_\tau p_\theta(\tau \mid x)\, R(\tau)$$

- $J(\theta)$：当前策略在任务 x 上的期望奖励；二值奖励下就是成功率；
- 求和遍历所有可能轨迹，实际中组合爆炸，只能采样估计。

[30] 视频画面时间区间：第 9 讲 00:20:42–00:23:58（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

[31] POMDP（Partially Observable Markov Decision Process，部分可观测马尔可夫决策过程）：底层状态满足马尔可夫性，但智能体只能看到观测；因此智能体的决策与「下一个观测的分布」都必须以完整的观测—动作历史为条件。LLM 智能体天然是 POMDP：它看到的是 prompt 与工具返回的文本，而不是仓库、网页后台或隐藏答案。

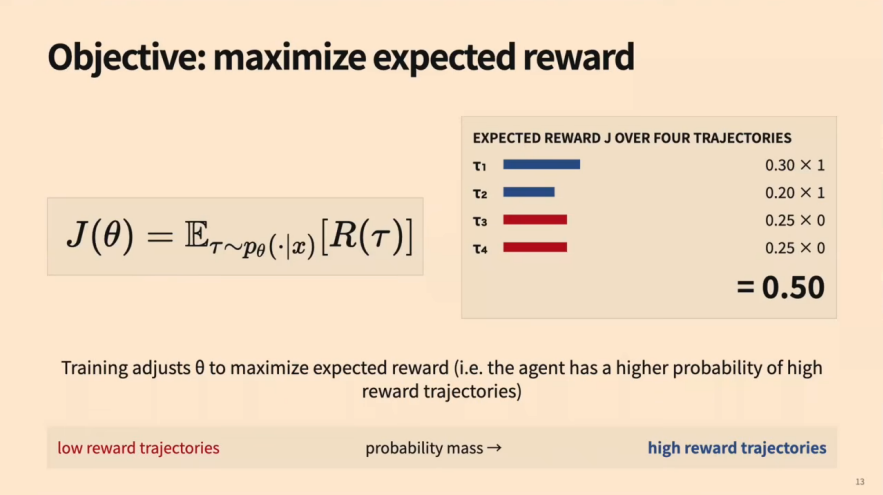


图 29：期望奖励的数例：假设策略只会产生这四条轨迹，概率 0.30 / 0.20 / 0.25 / 0.25，则 $J = 0.30 \times 1 + 0.20 \times 1 = 0.50$。训练要把概率质量从红色（低奖励）移到蓝色（高奖励）。[32]

3.3　专家迭代：ReST

在推导真正的 RL 之前，讲者先讲一个对 SFT 的简单改造：专家迭代（expert iteration）。ReST（Reinforced Self-Training，Gulcehre 等）的循环是：Grow——从当前策略采样轨迹并计算奖励；Improve——用奖励过滤，并入已有数据集，训练策略；Repeat——用更新后的策略继续。ReST-EM（Singh 等）因与 EM 算法[33]相似而得名。

$$\mathcal{L}_{\text{ReST}} = -\sum_{\tau:\, R(\tau)=1} \sum_{t} \log \pi_\theta(a_t \mid h_t)$$

- 外层求和只遍历奖励为 1 的轨迹（二值奖励下即按阈值 1 过滤）；
- 内层与 SFT 完全相同，只是动作来自模型自己的成功轨迹，而不是外部示范。

[32] 视频画面时间区间：第 9 讲 00:23:58–00:26:48（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

[33] EM 算法（Expectation–Maximization，期望最大化）：交替执行「在当前参数下推断隐变量的分布」（E 步）与「在该分布下最大化似然」（M 步）的迭代算法。ReST-EM 中，「按奖励过滤自采样轨迹」相当于 E 步，「在保留轨迹上做 SFT」相当于 M 步。

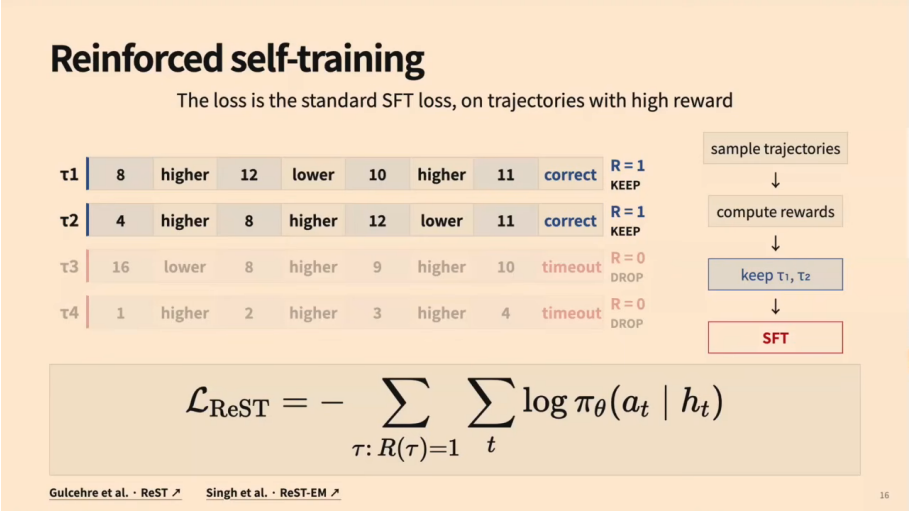


图 30: ReST：采样轨迹、计算奖励、保留 τ_1, τ_2、丢弃 τ_3, τ_4，然后在保留数据上做标准 SFT。[34]

讲者的评价是：严格说这不算 RL，但形态上与后面的 RL 方法非常相似——数据由模型生成，按奖励决定如何更新；它复用全部 SFT 机器，在二值奖励下是简单有效的基线方法。

课堂问答：如果一条正奖励轨迹都采不到？(00:30:31)

讲者回答：如果模型完全产生不了高奖励轨迹，RL 与 ReST 都不太适用，这是 RL 算法必须面对的大问题（“a really big kind of issue that RL algorithms have to grapple with”）。三种办法：(1) 先在好轨迹的示范上训练，例如 R1 等推理模型在 RL 之前用推理轨迹做 cold start SFT；(2) 课程学习，先挑模型能拿到奖励的题，变强后再做更难的；(3) 中间奖励，例如已知「先浏览代码仓库」有用，就为此给奖励，学会后可能解锁整个任务。另一个问题是 ReST 为何叫 self-training、与 self-distillation 有何不同：区别在于用奖励过滤；self-distillation 也可以指在模型产生的全部轨迹上训练。

3.4　从奖励到梯度：REINFORCE

3.4.1　为什么不能直接反向传播

计算链 $\theta \to \pi_\theta(a \mid h) \to$ 采样动作 $a \to$ 环境给出 (o, r) 有两处断开。其一，动作是离散的、采样得到的：θ 的微小变化改变的是动作概率，但采到的动作在跨过某个阶跃点之前不变，样本对参数的导数几乎处处为 0。其二，环境通常不可微，讲义原话是 “How do you take the derivative of a unit test w.r.t. code?”。通用解法是策略梯度（REINFORCE，Williams 1992）：利用 J 的定义做一点代数变换，得到只需要模型样本与常规反向传播的梯度估计。

[34] 视频画面时间区间：第 9 讲 00:28:36–00:32:16（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

3.4.2　推导

$$\begin{aligned}\nabla_\theta J &= \sum_\tau \nabla_\theta p_\theta(\tau \mid x)\, R(\tau) = \sum_\tau p_\theta(\tau \mid x)\, R(\tau)\, \frac{\nabla_\theta p_\theta(\tau \mid x)}{p_\theta(\tau \mid x)} \\ &= \sum_\tau p_\theta(\tau \mid x)\, R(\tau)\, \nabla_\theta \log p_\theta(\tau \mid x) = \mathbb{E}_{\tau \sim p_\theta(\cdot|x)}\big[R(\tau)\, \nabla_\theta \log p_\theta(\tau \mid x)\big]\end{aligned}$$

- 第一步：只有轨迹概率依赖 θ；讲者的说法是 “The parameters produce the trajectory. But then the reward is just a function of the trajectory.”；
- 第二步：乘除同一个 $p_\theta(\tau \mid x)$；
- 第三步：$\nabla p / p = \nabla \log p$（链式法则），这一技巧通称对数导数技巧（log-derivative trick）或得分函数估计（score function estimator）；
- 第四步：结果又成为一个期望，于是可以用采样的轨迹估计。

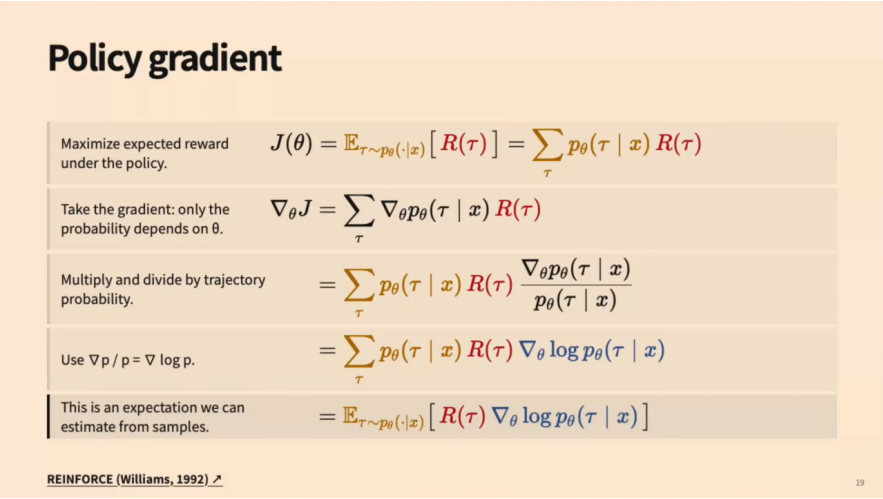


图 31：REINFORCE 推导的逐行展开。有学生评论这个推导对语言模型和环境都没有任何假设，讲者确认。[35]

再把轨迹对数概率展开：环境项与 θ 无关，求导时整体消失；一个动作可能由多个 token 组成（例如复杂的工具调用），动作的对数概率是其 token 对数概率之和：

$$\nabla_\theta \log p_\theta(\tau \mid x) = \sum_t \nabla_\theta \log \pi_\theta(a_t \mid h_t), \qquad \nabla_\theta \log \pi_\theta(a_t \mid h_t) = \sum_{k=1}^{m_t} \nabla_\theta \log \pi_\theta(u_{t,k} \mid h_t, u_{t,<k})$$

- $a_t = (u_{t,1}, \ldots, u_{t,m_t})$：动作 a_t 的 m_t 个 token；t 索引动作，k 索引动作内的 token。

所以单条轨迹的梯度估计是 $\widehat{\nabla_\theta J} = R(\tau) \sum_t \nabla_\theta \log \pi_\theta(a_t \mid h_t)$。三个组成部分分别来自：当前策略加环境采样出的轨迹（与运行智能体的过程相同）、环境给出的任务级评估 $R(\tau)$、与 SFT 完全相同的对数概率梯度——区别只在于这些 token 是智能体自己采样的，而不是示范。

[35] 视频画面时间区间：第 9 讲 00:34:11–00:37:37（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

3.4.3　实现：对 SFT 损失做掩码与缩放

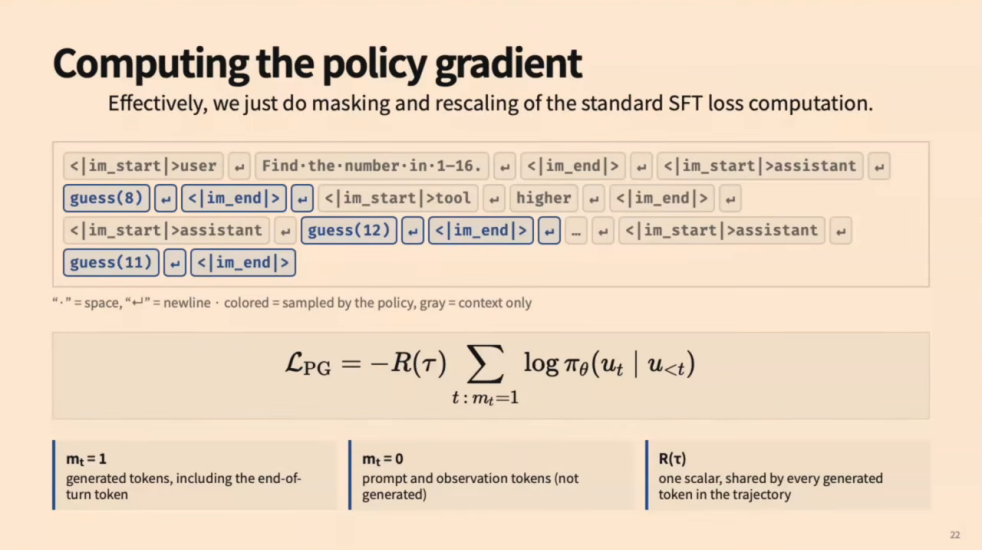


图 32: 在 chat template 渲染出的 token 序列上计算策略梯度：彩色为策略采样的 token（含回合结束符 `<|im_end|>`），灰色为 prompt 与观测 token；整条轨迹共享一个标量 $R(\tau)$。[36]

$$\mathcal{L}_{\mathrm{PG}} = -R(\tau) \sum_{t:\, m_t=1} \log \pi_\theta(u_t \mid u_{<t})$$

- u_t：渲染后整条序列的第 t 个 token；
- $m_t = 1$：u_t 由策略生成（包括回合结束 token）；$m_t = 0$：prompt 与观测 token，策略没有选择它们；
- $R(\tau)$：一个标量，由该轨迹中每个生成 token 共享。

讲者回忆第一次读到在复杂任务上实现 REINFORCE 的代码（Deal or No Deal 对话论文）时的惊讶："you just do basically an SFT gradient computation and then multiply that gradient by a reward or multiply the loss by a reward and take the gradient of that. But it works and the math works out."（00:42:57）。训练循环因此是：采样任务实例并在其上运行智能体 → 计算每条轨迹的奖励 → 用奖励加权动作对数概率 → 反向传播并更新，再用新策略采集下一批。标准 REINFORCE 每个任务实例一条轨迹，batch size 等于实例数（对应作业的 p2a：`group_size: 1`、`tasks_per_batch: 8`）。

二值奖励下 REINFORCE 与 ReST 形式相同

$$-\sum_i R(\tau_i) \sum_t \log \pi_\theta(a_{i,t} \mid h_{i,t}) = -\sum_{i:\, R(\tau_i)=1} \sum_t \log \pi_\theta(a_{i,t} \mid h_{i,t})$$

差别只在节奏：REINFORCE 采样后立刻更新（完全在线）；ReST 先积累大数据集、聚合历次数据，再做一个或多个 epoch。讲者随即指出 "it's very rare for people to just run standard reinforce policy gradient out of the box"——接下来要解决的就是它的问题。

[36] 视频画面时间区间：第 9 讲 00:41:27–00:43:31（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

3.5　基线与优势：比预期好多少

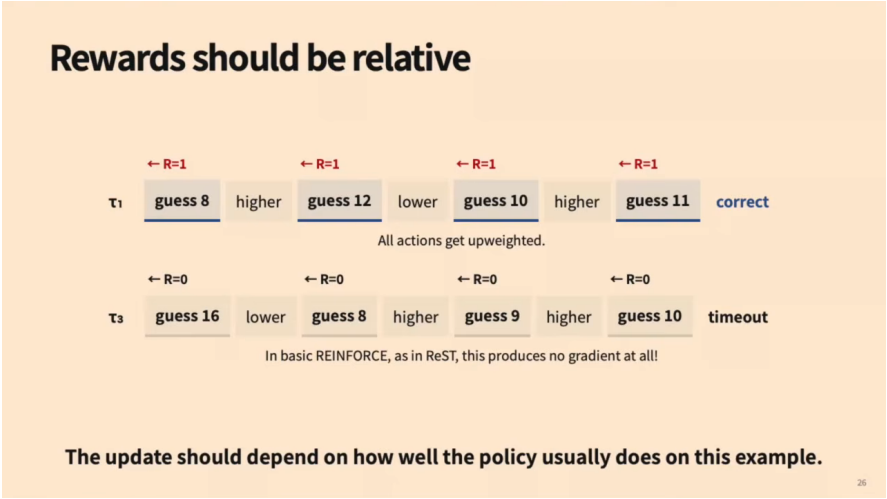


图 33: 奖励应当是相对的：成功轨迹的每个动作都被上调；失败轨迹每个动作的权重都是 0，在朴素 REINFORCE（与 ReST）中不产生任何梯度。[37]

失败轨迹不产生梯度是浪费：我们恰恰想学会不这样做。讲者的例子很直观："If 5% of the time you make a really dumb mistake and get a reward of zero, you should update very heavily against those costly but rare mistakes."（00:46:40）。更新应取决于策略在这个例子上「通常表现如何」，这引出优势（advantage）：

$$A^{\pi}(h,a) = Q^{\pi}(h,a) - V^{\pi}(h), \qquad \hat{A}_t = R(\tau) - b(h_t)$$

- $Q^{\pi}(h,a)$：在 h 先执行 a、之后按策略采样完成轨迹的期望奖励；
- $V^{\pi}(h)$：从 h 起（包括这一步）全部按策略采样的期望奖励，$V^{\pi}(h) = \mathbb{E}_{a\sim\pi}[Q^{\pi}(h,a)]$；
- $\hat{A}_t$：可计算的估计——Q 用实际采到的轨迹奖励 $R(\tau)$ 代替，V 用基线（baseline）函数 $b(h_t)$ 代替。

[37] 视频画面时间区间：第 9 讲 00:45:16–00:47:08（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

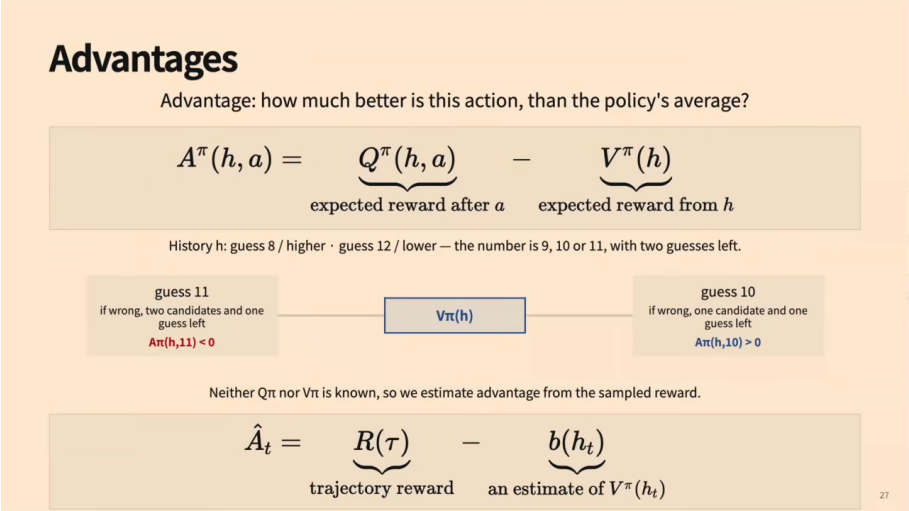


图 34: 优势的例子：历史为 guess 8 / higher · guess 12 / lower，隐藏数只能是 9、10、11，还剩两次。猜 10 若错只剩一个候选，$A^\pi(h,10) > 0$；猜 11 若错还剩两个候选，$A^\pi(h,11) < 0$。[38]

讲者口头估计这里的 $V^\pi(h)$「大概在 2/3 到 1 之间」。视频未给出精确值，但可以算出来：若策略在合法候选中均匀随机，$Q^\pi(h,10) = 1$，$Q^\pi(h,9) = Q^\pi(h,11) = \frac{1}{3} + \frac{2}{3}\cdot\frac{1}{2} = \frac{2}{3}$，于是 $V^\pi(h) = \frac{7}{9} \approx 0.78$，$A^\pi(h,10) = +\frac{2}{9}$，$A^\pi(h,11) = -\frac{1}{9}$。

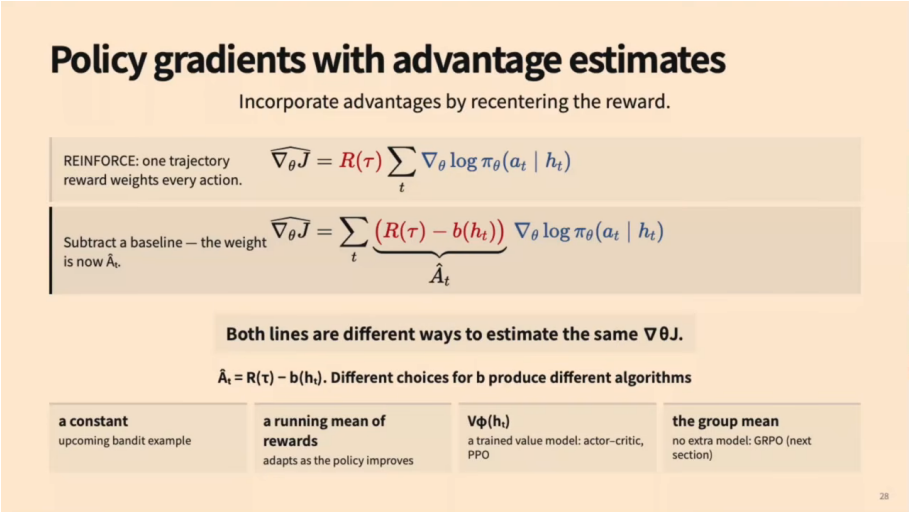


图 35: 带基线的策略梯度。两行在数值上不同，但都是同一 $\nabla_\theta J$ 的无偏估计；b 的不同选择对应不同算法：常数、历史奖励的滑动平均、训练出的价值模型 $V_\phi(h_t)$（actor–critic、PPO）、组均值（GRPO）。[39]

3.5.1 基线不引入偏差

先说清楚两个概念：估计量的偏差（bias）是它的均值偏离真值多少，方差（variance）是抽到的估计值有多分散。证明只需对每个 t 先对 a_t 取条件期望：

[38] 视频画面时间区间：第 9 讲 00:47:08–00:50:05（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。
[39] 视频画面时间区间：第 9 讲 00:50:05–00:52:13（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

$$\mathbb{E}_{a\sim\pi_\theta(\cdot|h)}\big[b(h)\nabla_\theta \log \pi_\theta(a \mid h)\big] = b(h)\sum_a \nabla_\theta \pi_\theta(a \mid h) = b(h)\,\nabla_\theta \sum_a \pi_\theta(a \mid h) = b(h)\,\nabla_\theta 1 = 0$$

- 第一步用了 $\pi\nabla \log \pi = \nabla\pi$，并把 $b(h)$ 提出求和号；
- 关键条件：b 只能依赖历史 h，不能依赖被评估的动作 a 本身，否则提不出求和号。

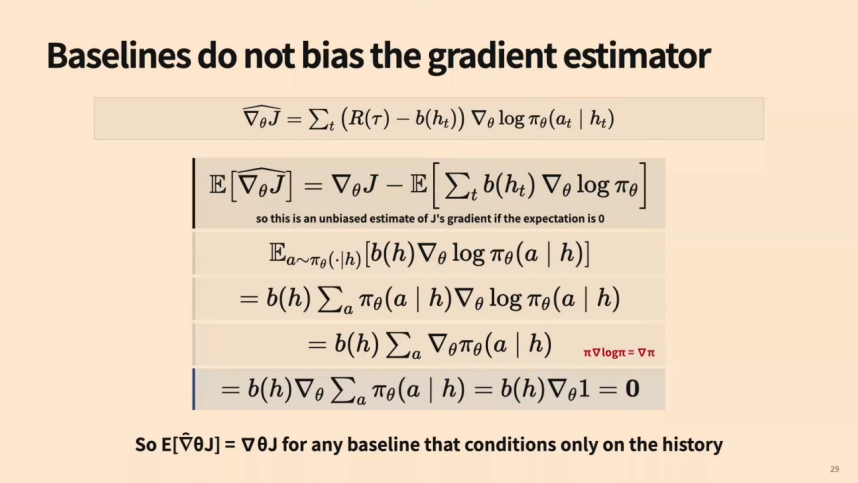


图 36: 基线无偏性的逐行证明：对任何只依赖历史的基线，$\mathbb{E}[\widehat{\nabla_\theta J}] = \nabla_\theta J$。[40]

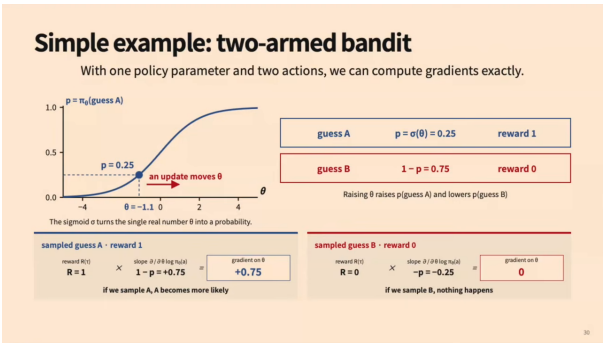
3.5.2　两臂老虎机：把方差算出来

讲者强调，RL 的大量工作都是在降低梯度估计的方差，使学习能承受策略采样与环境带来的随机性。为了让这一点可以精确计算，他用了一个单步、两个动作、一个参数的老虎机（bandit）：$p = \pi_\theta(\text{guess A}) = \sigma(\theta)$，当前 $\theta = -1.1$，$\sigma(-1.1) \approx 0.25$；guess A 奖励 1，guess B 奖励 0。

$$\partial_\theta \log \pi_\theta(A) = 1 - p = +0.75, \qquad \partial_\theta \log \pi_\theta(B) = -p = -0.25, \qquad \nabla_\theta J = p(1-p) = 0.1875$$

- σ：sigmoid，把单个实数 θ 变成概率；增大 θ 提高 p(A)、降低 p(B)；
- $\nabla_\theta J$：精确梯度，$J = p\cdot 1 + (1-p)\cdot 0 = \sigma(\theta)$，导数为 $p(1-p)$。

[40] 视频画面时间区间：第 9 讲 00:52:13–00:55:02（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。



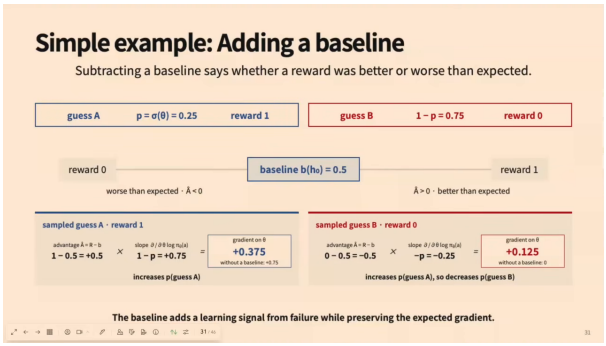
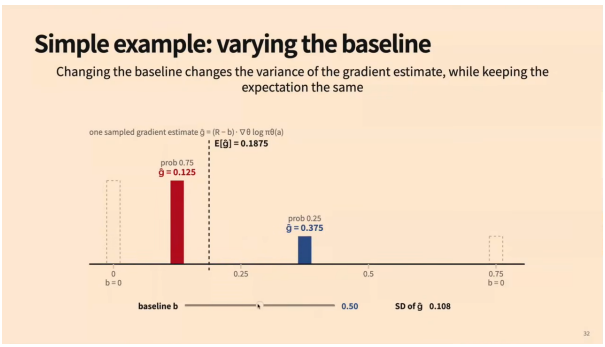


图 37: 左：无基线。采到 A 时单样本梯度 $1 \times 0.75 = +0.75$；采到 B 时 $0 \times (-0.25) = 0$，“nothing happens”。右：基线 $b = 0.5$。采到 A：$(1-0.5) \times 0.75 = +0.375$；采到 B：$(0-0.5) \times (-0.25) = +0.125$——失败样本也提高了 $p(\text{A})$。两种情况下期望都是 $0.25 \times 0.375 + 0.75 \times 0.125 = 0.1875$。[41]

接着讲者在一个交互页面上拖动基线滑块：单样本梯度 $\hat{g} = (R - b)\,\partial_\theta \log \pi_\theta(a)$ 只有两个取值，$\hat{g}_A = (1-b)(1-p)$（概率 0.25）与 $\hat{g}_B = bp$（概率 0.75），期望始终是 0.1875，而标准差随 b 变化。画面读数为：$b = 0$ 时 SD 0.325，$b = 0.5$ 时 0.108，$b = 0.75$ 时两根柱子重合于 0.188、SD 为 0。



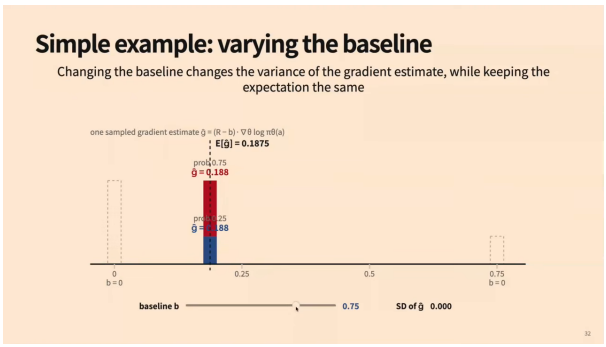


图 38: 交互滑块的两个状态。左：$b = 0.50$，$\hat{g} \in \{0.125, 0.375\}$，SD 0.108。右：$b = 0.75$，两种样本的梯度都等于 0.188，SD 为 0。[42]

把上面两个取值代入方差公式，可以得到一个闭式结果（视频只给了读数，下面的推导是本笔记补充）：

$$\mathrm{Var}[\hat{g}] = p(1-p)\,(1-p-b)^2, \qquad \mathrm{SD}[\hat{g}] = \sqrt{p(1-p)}\;|1-p-b|$$

- $p = 0.25$ 时 $\sqrt{p(1-p)} = 0.433$，于是 $b = 0 \to 0.325$，$b = 0.5 \to 0.108$，$b = 0.75 \to 0$，与画面一致；
- 方差最小的常数基线是 $b^\star = 1 - p = 0.75$，一般公式为 $b^\star = \mathbb{E}[R\,g^2]/\mathbb{E}[g^2]$（$g = \partial_\theta \log \pi$）。

[41] 视频画面时间区间：第 9 讲 00:55:02–01:00:34（均为幻灯片区域裁剪图）。
[42] 视频画面时间区间：第 9 讲 01:00:34–01:02:02（均为幻灯片区域裁剪图）。

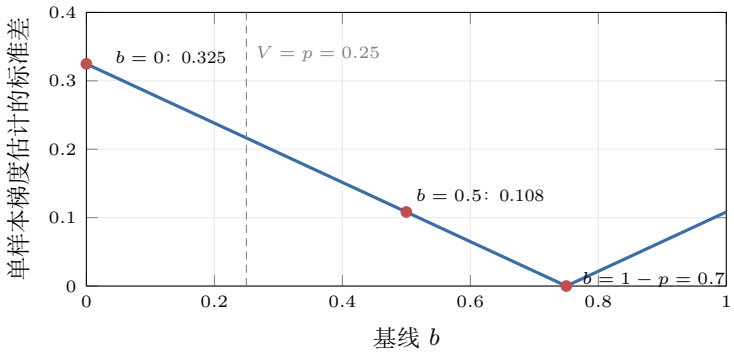


图 39: 两臂老虎机里梯度估计标准差随基线的变化：$\mathrm{SD}(\hat{g}) = \sqrt{p(1-p)}\,|1-p-b|$（由本笔记作者从讲义的设定解析推出，$p = 0.25$）。三个红点与课件演示中的数值一致。

方差最优的基线不是期望奖励

本例的期望奖励是 $V = p = 0.25$，取 $b = 0.25$ 时 SD 为 0.217，并不是最小值；方差最优的 $b^\star = 0.75$ 恰好让两种样本给出相同的梯度。「基线 = 期望奖励（价值函数）」是常用且通常不错的近似，GRPO 的组均值也是在估计它，但它不是方差最优解。这不影响后面的方法——任何只依赖历史的基线都无偏，差别只在方差。

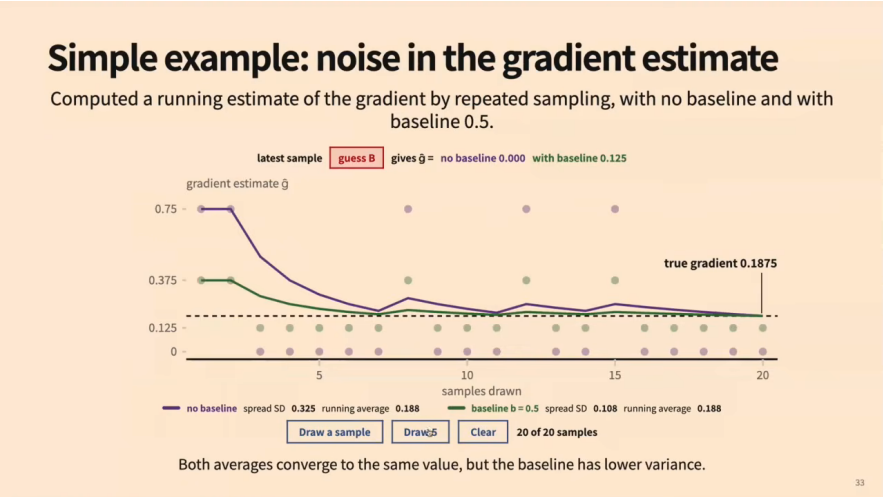


图 40: 反复采样时梯度估计的运行平均（预置的 20 次采样）：无基线时样本分布 SD 0.325，$b = 0.5$ 时 0.108；两条运行平均都收敛到真值 0.1875 附近。讲者随后现场逐个抽样，停在第 18 次时两者的 SD / 均值分别为 0.336 / 0.208 与 0.112 / 0.194。[43]

讲者坦言 “baselines have always been a little bit mysterious for me and I wanted to make these visualizations also to convince myself how they work”（01:03:23），并认为两臂老虎机的直觉可以推广到复杂的序列决策。随后他总结了基线的作用与局限：它能把一条轨迹标记为好于或差于预期、降低方差；但同一条轨迹内所有动作的优势仍然相同（没有逐步的信用分配），而估计 $V^\pi(h)$ 可能需要单独的价值模型——这两个问题留给第 11 讲。

若环境在中途给奖励，可以用 reward-to-go 代替整条轨迹的奖励（讲者基本跳过）：

[43] 视频画面时间区间：第 9 讲 01:01:29–01:03:42（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

$$R_t = \sum_{t'=t+1}^{T} r_{t'}, \qquad \widehat{\nabla_\theta J} = \sum_t R_t \, \nabla_\theta \log \pi_\theta(a_t \mid h_t)$$

- R_t：动作 a_t 之后获得的奖励之和；a_t 之前的奖励已包含在 h_t 中，相当于一个只依赖历史的基线，去掉它们降方差而不引入偏差；
- Number Search 只在结尾给一次奖励，所以每个动作的 $R_t = R(\tau)$。

3.6　组内比较：GRPO 与 Dr.GRPO

GRPO（Group Relative Policy Optimization）来自 DeepSeekMath 论文。此前把 RL 用于语言模型的工作常需要一个单独的模型预测基线，算力开销大；GRPO 去掉这个模型，用同一实例上的多次采样估计期望奖励。讲者的概括是："a good baseline is your expected reward under the policy. And a really good estimate of that is just running the policy multiple times and getting the reward taking the mean of those."（01:06:35）

$$\bar{R} = \frac{1}{G}\sum_{j=1}^{G} R(\tau_j), \qquad \sigma_R = \mathrm{std}\{R(\tau_1), \ldots, R(\tau_G)\}, \qquad \hat{A}_j = \frac{R(\tau_j) - \bar{R}}{\sigma_R}$$

- G：组大小，同一任务实例重置环境后完整采样 G 条轨迹；讲者说通常取 8 左右（作业 p2b 即 $G = 8$）；
- $\bar{R}$：组均值，作为组内所有 h_t 共用的基线；σ_R：组内奖励标准差（原版 GRPO 的标准化）。

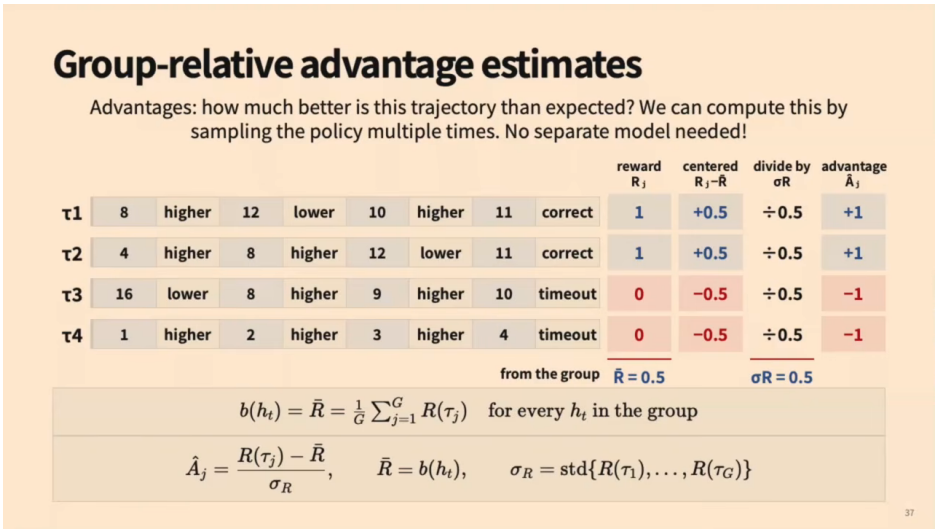


图 41：组相对优势：$R = (1, 1, 0, 0)$，$\bar{R} = 0.5$，$\sigma_R = 0.5$，$\hat{A} = (+1, +1, -1, -1)$。[44]

[44] 视频画面时间区间：第 9 讲 01:05:15–01:07:53（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

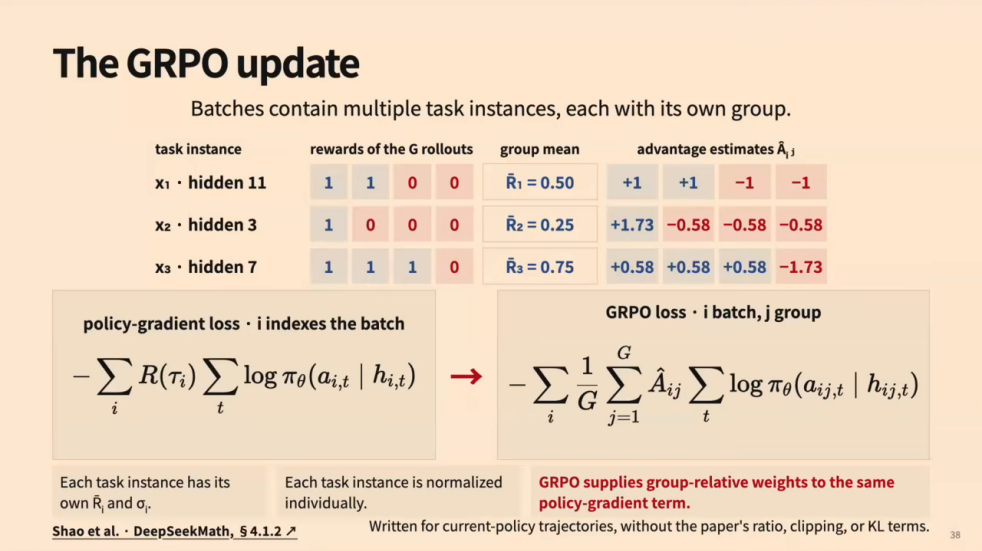


图 42：一个 batch 含多个任务实例，组均值只在实例内部计算。x_2 的奖励 $(1,0,0,0)$：$\bar{R}=0.25$，$\sigma=\sqrt{0.25\times 0.75}=0.433$，优势 $(+1.73,-0.58,-0.58,-0.58)$；$x_3$ 与之对称。下方是从策略梯度损失到 GRPO 损失的改写。[45]

$$\mathcal{L}_{\text{GRPO}}=-\sum_i \frac{1}{G}\sum_{j=1}^{G}\hat{A}_{ij}\sum_t \log \pi_\theta(a_{ij,t}\mid h_{ij,t}),\qquad \hat{A}_{ij}=\frac{R(\tau_{ij})-\bar{R}_i}{\sigma_i}$$

- i：batch 内的任务实例；j：组内 rollout；每个实例有自己的 $\bar{R}_i$ 与 σ_i；
- 讲义注明这是简化版：只写当前策略的轨迹，省去论文中的重要性比率、裁剪与 KL 项（第 11 讲展开）。讲者翻回这一页补充：裁剪防止一步走得太远，KL 正则让策略不偏离初始策略太多，二者都用于稳定训练。

两个容易对不上的细节

标准差的口径：讲义数值用的是总体标准差（ddof = 0）；若用 `torch.std` 的默认样本标准差（ddof = 1），x_1 的优势会变成 ± 0.866、x_2 变成 $+1.5/-0.5$。「**GRPO 无偏吗？**」：有学生这样问，讲者回答 “Yes”，理由是基线只依赖 prompt 与模型。更严格地说（本笔记补充）：组均值包含了被评估轨迹自身的奖励，期望梯度会整体乘上 $1-1/G$（方向不变）；用「留一均值」（leave-one-out）作基线才严格无偏；再除以依赖样本的 σ_R 则会引入真正的偏差——这正是下面 Dr.GRPO 去掉 σ_R 的理由之一。

3.6.1　Dr.GRPO 的两处修改

讲者特意点出：“Dr. GRPO makes two changes to GRPO. And you will also do these in your assignment.”（01:11:42）

[45] 视频画面时间区间：第 9 讲 01:07:53–01:11:32（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

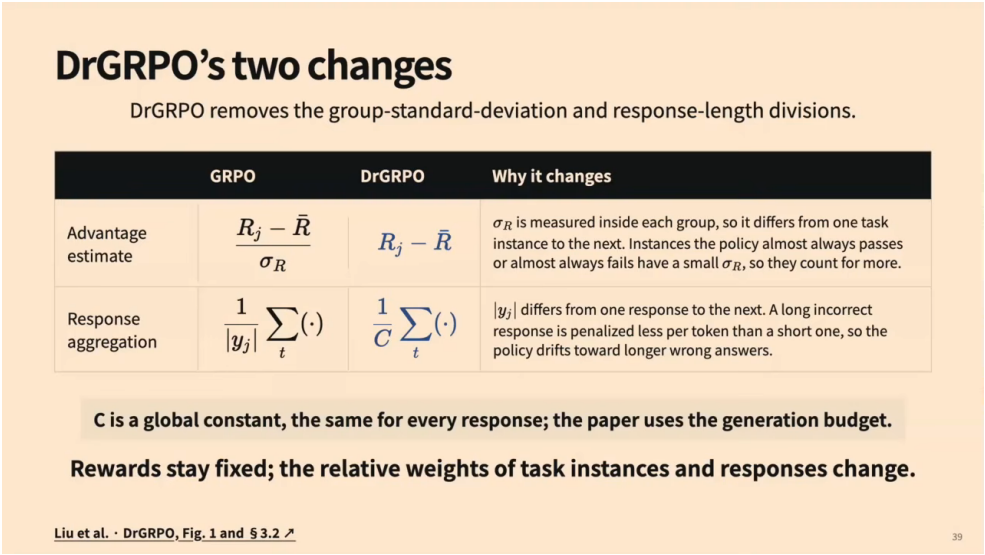


图 43: GRPO 与 Dr.GRPO 的两处差别：优势不再除以 σ_R；回答内的聚合用全局常数 $1/C$（论文取生成预算）代替 $1/|y_j|$。[46]

1. **去掉** σ_R：σ_R 在每组内测量，因实例而异。策略几乎总能通过或几乎总失败的实例 σ_R 很小，除以它会放大这些实例的权重；讲者的说法是标准化后优势不再反映问题难度。
2. **去掉** $1/|y_j|$：$|y_j|$ 是第 j 条回答的 token 数。按长度平均时，一条长的错误回答每个 token 受到的惩罚更小，策略会漂向更长的错误回答（"long wrong answers"）。改为全局常数 C 后，所有回答的 token 按同一尺度计权。

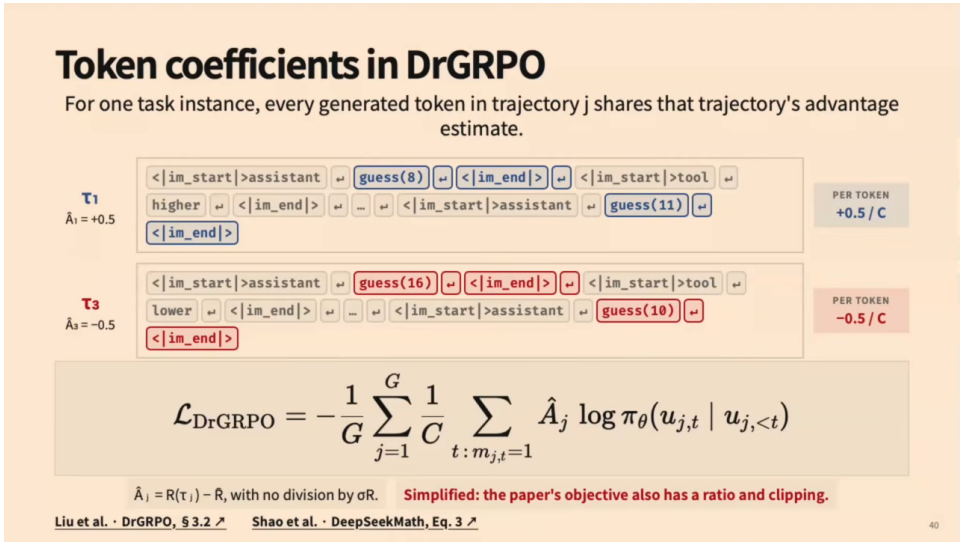


图 44: Dr.GRPO 的 token 系数：同一实例中，轨迹 j 的每个生成 token 共享 $\hat{A}_j/C$。图中 τ_1 每个蓝色 token 系数 $+0.5/C$，τ_3 每个红色 token 系数 $-0.5/C$。[47]

[46] 视频画面时间区间：第 9 讲 01:11:32–01:12:45（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。
[47] 视频画面时间区间：第 9 讲 01:12:45–01:13:45（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

$$\mathcal{L}_{\text{DrGRPO}} = -\frac{1}{G}\sum_{j=1}^{G}\frac{1}{C}\sum_{t:\,m_{j,t}=1}\hat{A}_j \log \pi_\theta(u_{j,t} \mid u_{j,<t}), \qquad \hat{A}_j = R(\tau_j) - \bar{R}$$

- C：全局常数，对所有回答相同；$m_{j,t}$：第 j 条轨迹第 t 个 token 的掩码；
- 作业中的对应实现：`compute_group_advantages` 只减组均值、不除标准差；`TinkerSession` 的 `train_step` 把每个优势乘以 $1/T$，T 为整个 batch 的动作 token 数——同样不按单条回答长度归一化，只是常数取「本 batch 的 token 总数」而非生成预算。

第一处修改的效果可以用一个大组极限下的计算看清（本笔记补充）：二值奖励、成功率 p 的任务，GRPO 每条 rollout 的期望 $|\hat{A}|$ 为 $2\sqrt{p(1-p)}$，Dr.GRPO 为 $2p(1-p)$。

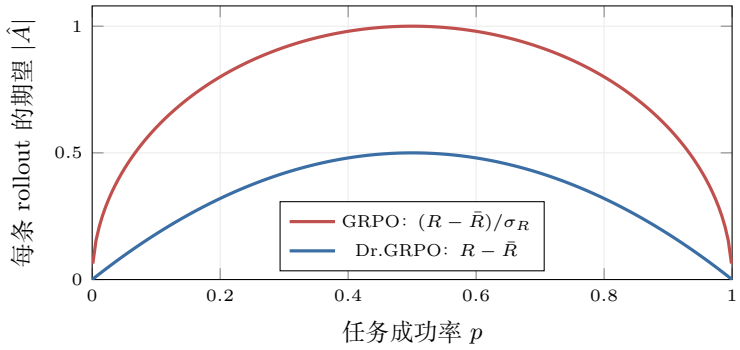


图 45: 二值奖励下（大组极限）每条 rollout 优势绝对值的期望：Dr.GRPO 为 $2p(1-p)$，GRPO 为 $2\sqrt{p(1-p)}$，两者之比 $1/\sigma_R$ 在 $p \to 0$ 或 $p \to 1$ 时发散（本笔记作者推导）。

两者之比 $1/\sigma_R = 1/\sqrt{p(1-p)}$ 在 $p \to 0$ 或 $p \to 1$ 时发散：在 $p = 0.05$ 的任务上，GRPO 的权重是 Dr.GRPO 的约 4.6 倍。$G = 8$ 时只有一条成功的组，GRPO 给成功轨迹 $\hat{A} = 2.65$，Dr.GRPO 给 0.875。

3.6.2　退化组

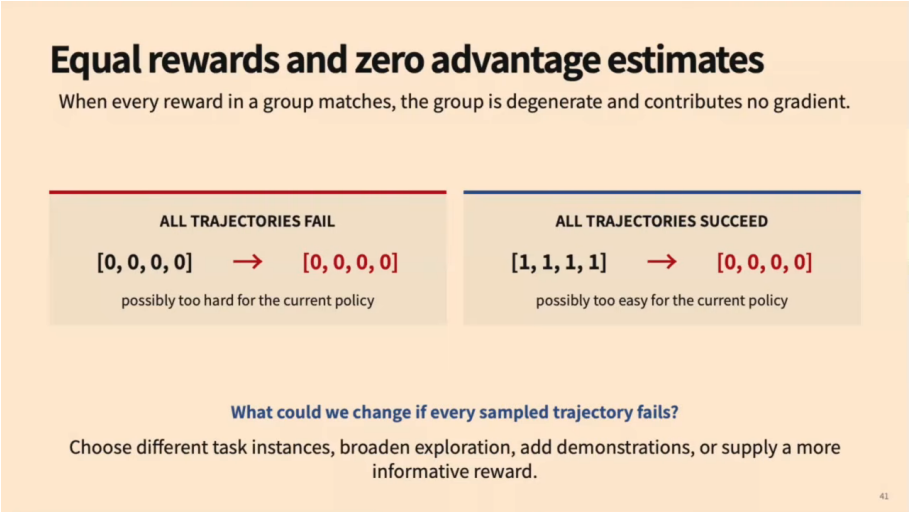


图 46: 组内奖励全相同则优势全为 0，该组不贡献梯度：全失败可能太难，全成功可能太容易。下方列出全失败时的补救方向。[48]

讲者把它与 ReST 一节的提问联系起来：即使有基线，全失败也得不到信号；一般地，组内奖励没有离散度就没有优势信号。补救办法有：换任务实例（课程）、在 prompt 中注入知识引导出可能正确的解、加入示范（有些异策略方法把正确示范放进组里，更简单的是 RL 前先在示范上微调）、提供信息更丰富的奖励（reward shaping，例如找到正确文件就给部分奖励）。作业里退化组由 `compute_group_advantages` 返回 `None` 表示，日志指标是 `train/frac_degenerate_groups`。最后一页 GRPO 与 PPO 的对比（DeepSeekMath 图 4：PPO 需要 Policy、Reference、Reward、Value 四个模型与 GAE，GRPO 用组计算替代 Value 模型）讲者跳过，留给第 11 讲。

3.7　总结：所有方法都在给自己的动作加权

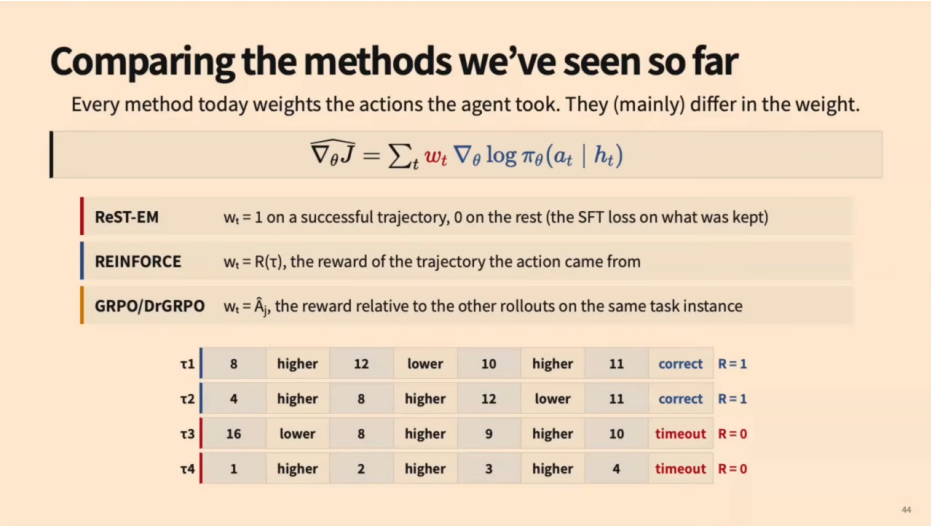


图 47：本讲三种方法的统一形式 $\widehat{\nabla_\theta J} = \sum_t w_t \nabla_\theta \log \pi_\theta(a_t \mid h_t)$：ReST-EM 在成功轨迹上 $w_t = 1$、其余为 0；REINFORCE $w_t = R(\tau)$；GRPO / Dr.GRPO $w_t = \hat{A}_j$。[49]

讲者的结束语回到开头的三种失配：任务失配——目标就是期望奖励，任何成功的轨迹都被强化；数据失配——只要正确使用基线，奖励为 0 的轨迹也提供信号；暴露偏差——“the model's like essentially driving at training time. So it should be learning to recover from its own errors”（01:17:17）。代价是必须在环境中实际尝试，开销大，而且给梯度带来噪声。

3.8　本章小结

- **问题**：SFT 训练智能体有任务失配、数据失配、暴露偏差三种失配；RL 以期望奖励 $J(\theta)$ 为目标、用自己的 rollout 训练，分别化解它们。
- **机制**：对数导数技巧把 ∇J 变成可采样的期望；环境项求导消失；实现上就是对 SFT 损失按 token 掩码、按奖励缩放。
- **方差**：减去只依赖历史的基线不改变期望；两臂老虎机里 SD 从 0.325（$b = 0$）降到 0.108（$b = 0.5$），在 $b = 0.75$ 时为 0。

[48] 视频画面时间区间：第 9 讲 01:13:45–01:15:21（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。
[49] 视频画面时间区间：第 9 讲 01:15:46–01:16:49（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

- **算法**：GRPO 用组均值作基线、无需价值模型；Dr.GRPO 去掉 σ_R 与 $1/|y_j|$，作业正是这样实现的；奖励全相同的组没有梯度。

视频未展开、但对复现有用的几点补充分析：

1. **三种方法是同一段代码**。ReST、REINFORCE、GRPO 的差别只在 w_t，所以作业中 `algo` 与 `loss_fn` 是两个独立开关：前者决定 w_t 怎么算，后者决定 token 级损失的形式。同步训练时 $\rho = 1$，`ppo`、`importance_sampling`、`cross_entropy` 三种损失的梯度相同。
2. **组大小与信号量**。成功率为 p 的任务，G 条 rollout 全相同的概率是 $p^G + (1-p)^G$；$G = 2$ 时最少也有一半的组是退化组，$G = 8$ 在 $p \in [0.2, 0.8]$ 内几乎不退化（第 6.8 节）。这把本讲的「退化组」与作业 Part 4.1 的组大小消融直接联系起来。
3. **基线方差最优解**。两臂老虎机的闭式解说明：基线的目的不是「估准价值」，而是让不同样本的梯度尽量一致。这解释了为什么 GRPO 用一个很粗糙的组均值也能工作——它只需要比 $b = 0$ 好得多，不需要最优。

3.9　拓展阅读

- Liu 等，Understanding R1-Zero-Like Training: A Critical Perspective（Dr.GRPO），`arXiv:2503.20783`。
- Shao 等，DeepSeekMath（GRPO 原始定义），`arXiv:2402.03300`。

4　第 11 讲：进阶 RL 算法——信用分配、异步与奖励可靠性

本讲由 Graham Neubig 主讲（1:08:15）。他开场就说明了取材原则：只讲他自己在过去一两年训练智能体时真正遇到过的问题，每个问题配一个算法层面的修复（problem → fix），目的是帮助学生排查 Assignment 3 与课程项目里的训练故障；沙箱、并行执行、内存等系统问题留给下一讲。结尾的总结页把全讲归为三条主线：**有用的反馈**（用组比较、critic 或过程奖励做信用分配）、**稳定的更新**（知道行为策略是谁，控制单个样本对模型的影响）、**可靠的奖励**（匹配任务难度，验证真正想要的行为），外加两个辅助主题：减少等待（异步 RL）与从教师学习（蒸馏）。

4.1 贯穿示例：修一个 bug

图 48: 贯穿示例（来自第 6 讲 Coding Agents）：修复 `retry_count`，使 0 表示关闭重试；四个测试 Missing→3、None→3、0→0、2→2 全部通过奖励为 1，否则为 0。[50]

bug 的机制是 Python 的真值规则：`0` 为假值，`0 or 3` 求值为 3。正确修复需要显式区分 None（`v = config.get("retries"); return 3 if v is None else v`）；而「`config.get("retries", 3)`」是不完整的修复——键存在但值为 None 时返回 None，None→3 的测试失败。后面讲到过期 rollout 时三个候选动作（显式检查 None / 用 `get(..., 3)` / 保持原样）正是这三种写法。讲者强调示例中的一条成功轨迹走了五步（Inspect、Edit、Test、Revise、Test again），中间有一次错误编辑——这是信用分配问题的伏笔。回顾 REINFORCE 时他指出两个缺陷：方差高；失败尝试（$R = 0$）不产生任何更新。

4.2 信用分配与稠密反馈

4.2.1 从组基线到逐回合信用

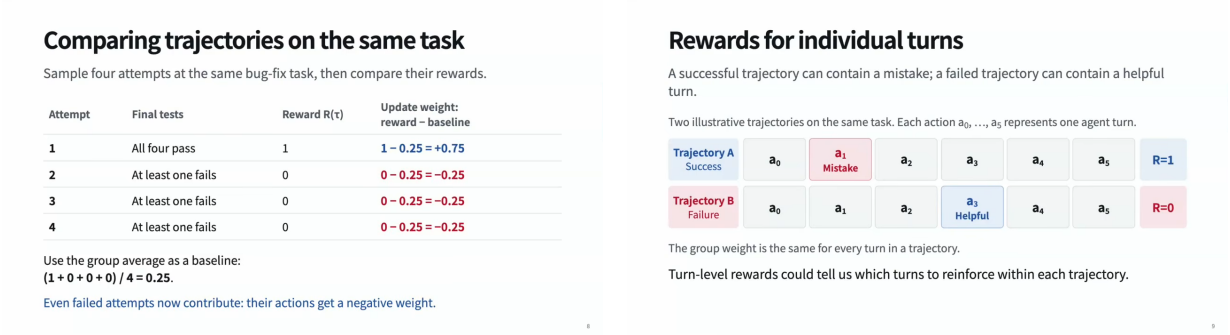


图 49: 左：同一任务采样 4 次，奖励 $1, 0, 0, 0$，组均值 0.25 作为基线，成功轨迹权重 $+0.75$，三条失败轨迹各 -0.25——失败也提供了信号。右：组权重对一条轨迹的每个回合都相同，于是成功轨迹 A 中的错误 a_1 被正向强化，失败轨迹 B 中有用的 a_3 被负向惩罚。[51]

[50] 视频画面时间区间：第 11 讲 00:01:30–00:02:20（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。
[51] 视频画面时间区间：第 11 讲 00:04:58–00:06:36（均为幻灯片区域裁剪图）。

右图引出需求：逐回合（turn-level）的奖励。reward-to-go R_t（a_t 之后的奖励之和）在只有终局奖励时无济于事——成功轨迹每一步 $R_t = 1$，失败轨迹每一步 $R_t = 0$。讲者把「只观测最终奖励时如何知道哪一步有帮助」抛给学生，收到四个答案：写 rubric 得到细粒度奖励；看所有成功 rollout 共享哪些动作；收集大量轨迹找出区分成功的动作序列；定义多种奖励（例如轨迹更短）。他认为都合理，但接下来要讲的是另一条路：价值函数。

4.2.2　价值函数与 critic

设想从失败轨迹 B 的历史 h_3 出发、用同一个 rollout 策略 μ 继续 10 次，reward-to-go 为 $0, 1, 0, 0, 1, 0, 0, 0, 1, 0$，平均 0.3。无穷次延续的期望就是价值：

$$V^{\mu}(h_t) = \mathbb{E}_{\mu}\big[R_t \mid h_t\big], \qquad \mathcal{L}_V(\phi) = \mathbb{E}_t\Big[\big(V_\phi(h_t) - R_t\big)^2\Big]$$

- μ：rollout 策略，即推理时实际采样用的策略；完全 on-policy 时与被更新的 π_θ 相同，异步时不同；
- R_t：从 h_t 起的 reward-to-go；V_ϕ：参数为 ϕ 的 critic[52]，用平方误差拟合 R_t；
- 平方误差的最优解是条件均值：沿用上面 10 个结果，若 $V_\phi(h_3) = 0.6$，平均平方误差为 $\frac{3(0.6-1)^2+7(0.6-0)^2}{10} = 0.30$；预测 0.3 时为 0.21（恰好是伯努利方差 0.3×0.7）。

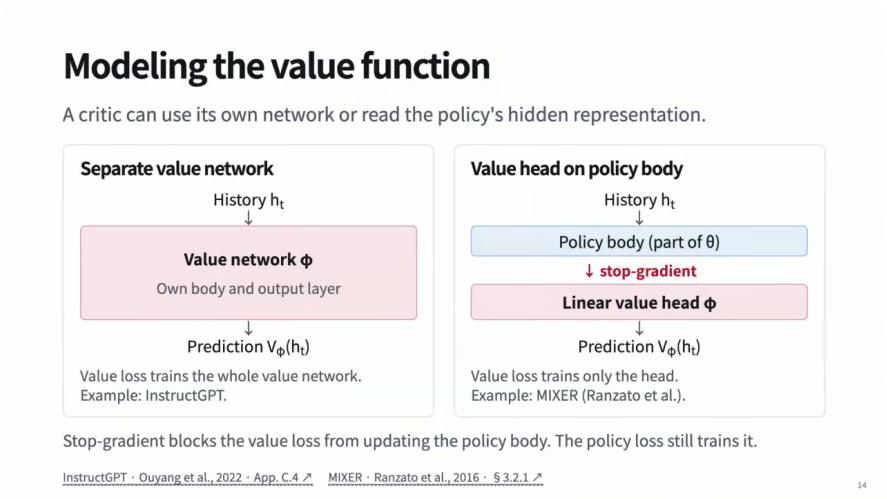


图 50: 价值函数的两种参数化。左：独立价值网络，自带 body 与输出层（例：InstructGPT）。右：在策略 body 上加线性价值头，中间 stop-gradient，价值损失只训练价值头（例：MIXER）。[53]

讲者在黑板上补充了独立价值网络的两种输出方式：一是回归头直接输出数值；二是在历史后接一句 “Is this trajectory good? Yes or No”，读取下一个 token 的概率，用 $P(\text{Yes})/(P(\text{Yes}) + P(\text{No}))$ 作为价值——只适用于奖励在 $[0, 1]$ 的情形，好处是复用模型已有的先验。有学生问：既然模型能判断好坏，为什么不直接生成正确动作？讲者给出两点：价值函数只以历史为条件，不以动作为条件；而且 “discriminating between good and bad trajectories is easier than actually generating a good next [action]”（00:16:35）。

[52] critic（评论家）：在 actor–critic 框架中，actor 指策略 π_θ，critic 指估计价值 $V_\phi(h)$ 的模型；critic 的输出用作基线或用来构造逐步优势。PPO 是 actor–critic 方法，GRPO 用组均值代替了 critic。

[53] 视频画面时间区间：第 11 讲 00:12:38–00:16:48（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

4.2.3　TD 误差与 GAE

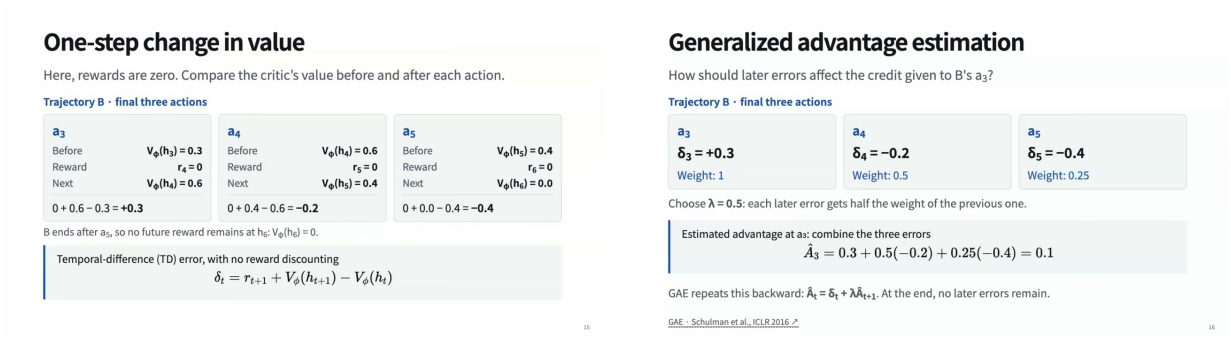


图 51: 左：轨迹 B 最后三步的一步价值变化，$\delta_3 = +0.3$、$\delta_4 = -0.2$、$\delta_5 = -0.4$（a_5 后终止，$V_\phi(h_6) = 0$）。右：GAE 取 $\lambda = 0.5$，$\hat{A}_3 = 0.3 + 0.5(-0.2) + 0.25(-0.4) = 0.1$。[54]

$$\delta_t = r_{t+1} + V_\phi(h_{t+1}) - V_\phi(h_t), \qquad \hat{A}_t = \delta_t + \lambda \hat{A}_{t+1} = \sum_{l \geq 0} \lambda^l \delta_{t+l}$$

- δ_t：时序差分（TD）误差，衡量动作 a_t 把价值提高或降低了多少（讲义取折扣 $\gamma = 1$；一般形式为 $r_{t+1} + \gamma V_\phi(h_{t+1}) - V_\phi(h_t)$）；
- $\lambda \in [0, 1]$：GAE（Generalized Advantage Estimation，Schulman 等）中后续误差的衰减权重；递推从末尾开始，末尾没有后续误差。核对：$\hat{A}_5 = -0.4$，$\hat{A}_4 = -0.2 + 0.5(-0.4) = -0.4$，$\hat{A}_3 = 0.3 + 0.5(-0.4) = 0.1$。

讲者的解读：a_3 本身是好的一步（+0.3），但之后连着两步坏的，说明 a_3 可能把后续带向坏的决策，所以用衰减后的未来变化把它的信用压低到 0.1。他举例说，如果某个动作误删了整个代码库，价值会骤降到 0，TD 误差会把这一步标出来。关于 λ 的取值，他说训练 PPO 时典型取 1，此时「只看最终奖励」。

$\lambda = 1$ 并不等于「只看最终奖励」

$\gamma = \lambda = 1$ 时各项 δ 相加会逐项抵消（telescoping），结果是 $\hat{A}_t = \sum_l \delta_{t+l} = R_t - V_\phi(h_t)$：Monte Carlo 回报减去价值基线，基线项仍然保留。本例中 $\lambda = 1$ 得 $\hat{A}_3 = 0.3 - 0.2 - 0.4 = -0.3 = 0 - 0.3$。$\lambda = 0$ 时 $\hat{A}_t = \delta_t$，方差低但完全依赖 critic 的准确度；$\lambda = 1$ 方差高但不依赖 critic 的准确度。讲者还把 λ 与时间折扣 γ 混称；讲义的符号页把两者分开定义。许多开源 PPO 实现的默认值是 $\lambda = 0.95$（背景知识，本讲未涉及）。

最后一种信用分配来源是过程奖励模型（PRM[55]，Let's Verify Step by Step，Lightman 等）：雇大量标注者标出每一步是否正确，训练一个预测器作为价值的替代。讲义例题中 $5x = 6x - 14$ 应得 $x = 14$，最后一步 "$x = 7$" 被标为错误。讲者的评价是 "I would say this is not used very widely"，因为价值函数可以作为训练副产物学到；但若有大量标注，可以用它 bootstrap 一个更好的初始价值函数。

[54] 视频画面时间区间：第 11 讲 00:16:48–00:22:26（均为幻灯片区域裁剪图）。

[55] PRM（Process Reward Model，过程奖励模型）：在 RL 之前用人工标注的「每一步是否正确」单独训练的模型，为推理或行动的每一步打分；与之相对的是只看最终结果的结果奖励模型（ORM，Outcome Reward Model）。

4.3　同步与异步 RL

讲者开场的提醒很直接："reinforcement learning for agents it can be very slow... you will certainly experience this and be frustrated by it in your assignment."（00:23:59）

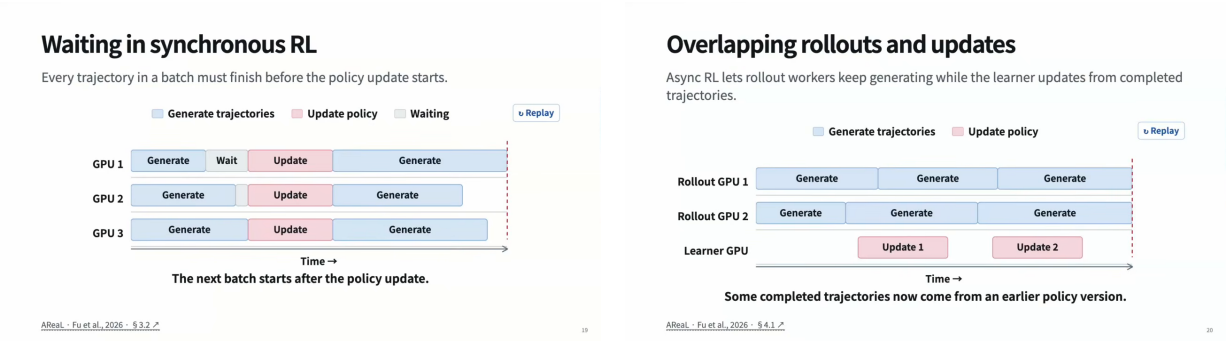


图 52: 左：同步 RL，批内所有轨迹都结束后才更新，最慢的 GPU 决定节奏，其余 GPU 等待。右：异步 RL，rollout GPU 持续生成，learner GPU 用已完成的轨迹更新；代价是部分轨迹来自更早的策略版本。图源 AReaL（Fu 等，2026）。[56]

讲者说时序图对实际情况偏乐观："if you're training a coding agent, it will be very common to have one straggler that takes like twice as much time or four times as much time as all the others"（00:25:00）。同步的唯一优点是 on-policy——生成 rollout 的策略与被更新的策略完全相同（$\mu = \pi_\theta$）。他的判断是 "Nobody who's doing Agentic RL seriously would do synchronous updates now, especially now that the software has gotten pretty good to support async updates"（00:27:20）。作业的 Part 2 与 Part 3 正好对应这两种设计，Part 3 要求用 `train/step_time_s` 量化加速比。

4.4　稳定的策略更新

4.4.1　过期 rollout 与重要性采样

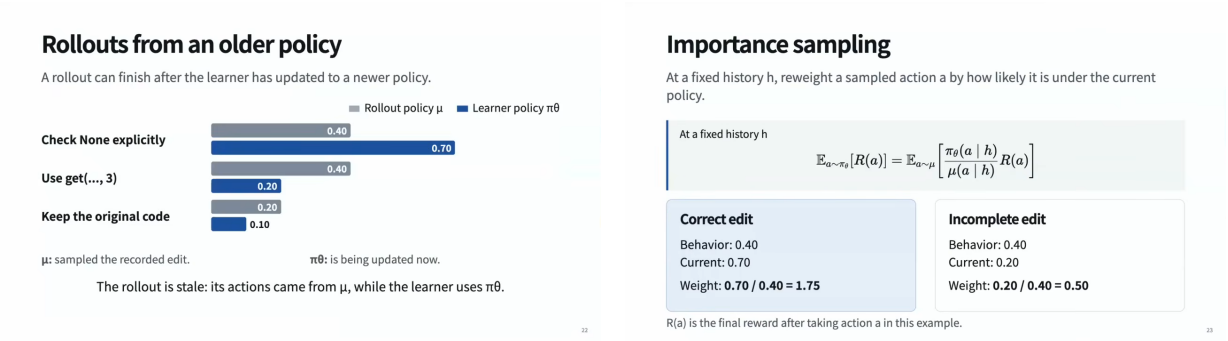


图 53: 左：rollout 由旧策略 μ 生成（三个候选编辑概率 0.40/0.40/0.20），learner 已更新为 π_θ（0.70/0.20/0.10）。右：重要性采样把采样分布下的期望改写为目标分布下的期望；正确编辑权重 $0.70/0.40 = 1.75$，不完整编辑 $0.20/0.40 = 0.50$。[57]

[56] 视频画面时间区间：第 11 讲 00:24:12–00:27:30（均为幻灯片区域裁剪图）。
[57] 视频画面时间区间：第 11 讲 00:27:44–00:30:24（均为幻灯片区域裁剪图）。

$$\mathbb{E}_{a\sim\pi_\theta}[R(a)] = \sum_a \mu(a \mid h)\frac{\pi_\theta(a \mid h)}{\mu(a \mid h)}R(a) = \mathbb{E}_{a\sim\mu}\left[\frac{\pi_\theta(a \mid h)}{\mu(a \mid h)}R(a)\right]$$

- 固定历史 h；要求在 $\pi_\theta(a \mid h) > 0$ 的地方 $\mu(a \mid h) > 0$（支撑集覆盖）；
- 比率的期望 $\mathbb{E}_\mu[\pi_\theta/\mu] = 1$；核对：只有正确编辑得奖励 1 时，$\mathbb{E}_\mu[R] = 0.4$，$\mathbb{E}_{\pi_\theta}[R] = 0.7$，加权估计 $0.4 \times 1.75 = 0.7$。

讲者问全班谁听说过重要性采样，举手者 “surprisingly few”。

4.4.2 长轨迹让比率失控

整条轨迹的比率是逐动作比率之积（环境转移概率在分子分母中相同而约去）：

$$w(\tau \mid x) = \frac{p_\theta(\tau \mid x)}{p_\mu(\tau \mid x)} = \prod_t \frac{\pi_\theta(a_t \mid h_t)}{\mu(a_t \mid h_t)}, \qquad \log \rho_t^{\text{cum}} = \sum_{k\le t} \log \rho_k$$

- 单个动作比率 1.05 沿 100 步相乘就是 $1.05^{100} \approx 132$，少数样本会主导更新；讲者说 100 步还是乐观估计，“an agentic trajectory will have like 50,000 tokens or 60,000 tokens”（00:30:50）；
- ρ_t^{cum}：截至 token t 的累积比率；若各项独立同分布、方差 σ^2，则 $\mathrm{Var}(\log \rho_t^{\text{cum}}) \approx t\sigma^2$，标准差按 $\sqrt{t}$ 增长。

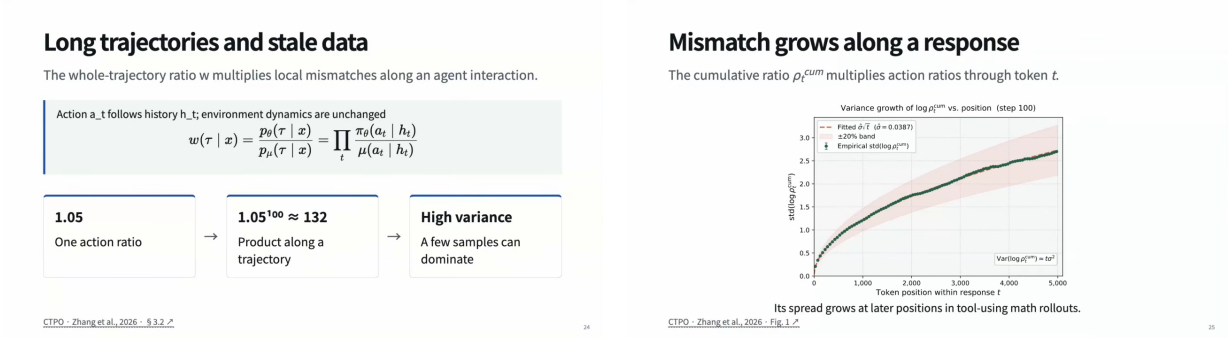


图 54: 左：轨迹级比率是乘积，$1.05^{100} \approx 132$。右：CTPO（Zhang 等，2026）图 1，工具调用数学 rollout 中 $\mathrm{std}(\log \rho_t^{\text{cum}})$ 随 token 位置按 $\hat{\sigma}\sqrt{t}$ 增长，拟合 $\hat{\sigma} = 0.0387$，$t = 5{,}000$ 处约 2.7。[58]

讲者口误的更正

讲者说 “variance will go up by the square root of the number of items”，并归因于大数定律。正确说法是：独立项求和后方差随项数线性增长、标准差随 $\sqrt{t}$ 增长（右图纵轴是标准差）；和趋于高斯分布是中心极限定理，而不是大数定律。按拟合值，$t = 5{,}000$ 时 $\log \rho$ 的一个标准差为 2.74，对应比率本身相差约 $e^{2.74} \approx 15$ 倍。

比率失控还有另一个来源：早期动作会改变后续历史。讲义例子中，早期编辑「加一个 None 检查」在 μ 下概率 40%、在 π_θ 下 10%；之后的「运行测试」在两者下都是 50%，单步比率为 1，但到达这个历史的可能性只有 $0.10/0.40 = 0.25$。不同算法因此在「哪些动作进入权重」上做了不同取舍：

[58] 视频画面时间区间：第 11 讲 00:30:24–00:32:12（均为幻灯片区域裁剪图）。

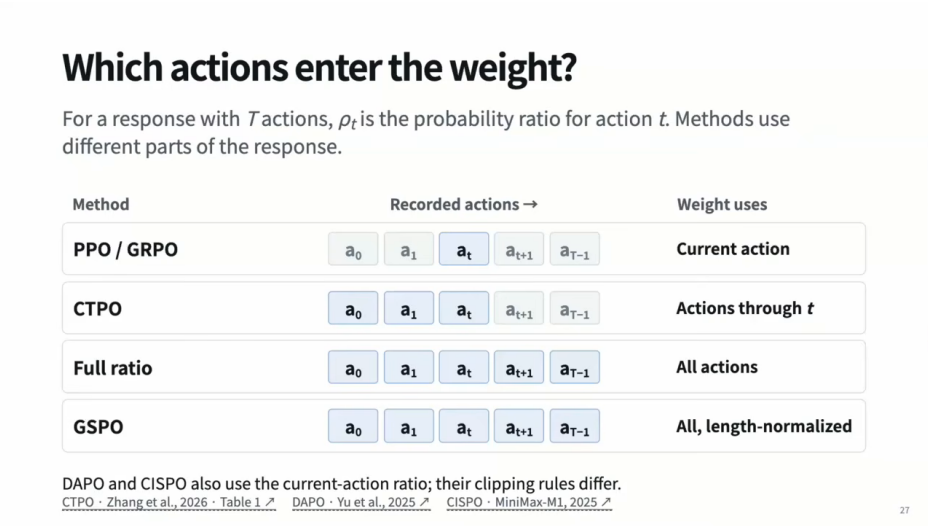


图 55: 比率使用哪些动作：PPO / GRPO 只用当前动作；CTPO 用截至 t 的前缀；完整比率用全部动作；GSPO 用全部动作并按长度归一化。DAPO 与 CISPO 也用当前动作比率，差别在裁剪规则。[59]

讲者对最常用的 PPO / GRPO 的评价是：只用当前 token 的比率 "is not actually correct... but it's good enough and it gives you some idea of how unexpected the action is"；完整比率与 CTPO 的前缀比率都会变得很大，所以都需要裁剪；GSPO 除以序列长度，把量级拉回与 PPO / GRPO 相同的尺度。一句话概括："you want to use importance weights but you don't want to let them get out of control"（00:34:35）。

4.4.3　裁剪

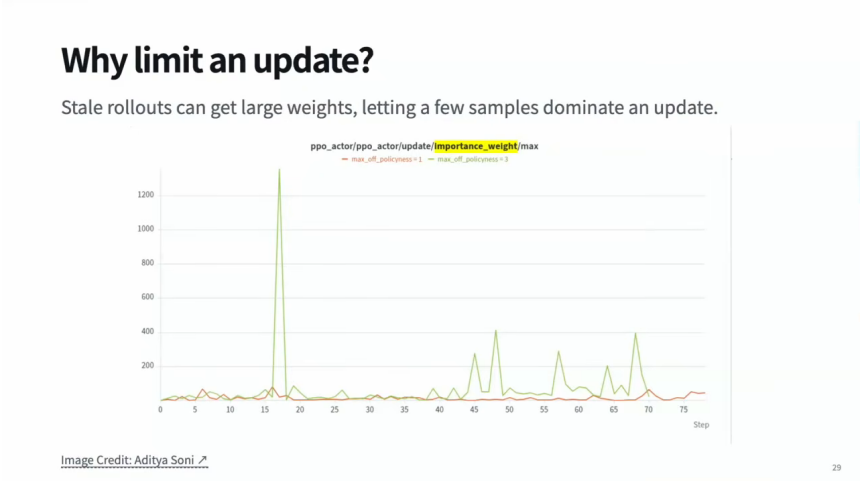


图 56: 讲者实验室（TA Aditya Soni 提供）的真实训练曲线：每个 batch 的最大重要性权重。`max_off_policyness=1`（只用上一个策略的 rollout）全程低于约 70；`max_off_policyness=3` 在第 17 步左右出现约 1,350 的尖峰，之后还有多个 200–410 的尖峰。[60]

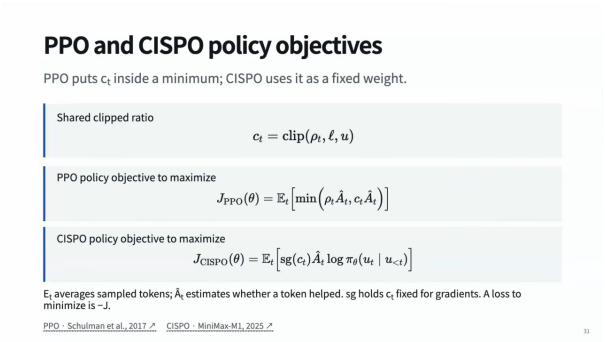
[59] 视频画面时间区间：第 11 讲 00:32:46–00:34:42（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。
[60] 视频画面时间区间：第 11 讲 00:34:44–00:36:40，回看 00:39:42–00:40:28（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

讲者的描述是 “you can imagine what would happen if you have a multiple of 1,400 in your update. You’re going to make a huge gradient step and mess up your model”。失控后的表现是模型突然退化——反复输出同一个 token、输出极长或极短——“it’s hard to say it’s going to be any individual thing. It’s just like suddenly things look really weird”，而且很难恢复，因为之后拿不到正优势。这里的 `max_off_policyness` 就是作业的 `train.max_staleness`，这条曲线就是作业要求必画的 `train/importance_weight_max`。

修复方法是裁剪（clipping）比率。讲者称之为 “somewhat heuristic”，PPO 论文使用了这种做法（但不是首创）：

$$\rho_t = \frac{\pi_\theta(u_t \mid u_{<t})}{\mu(u_t \mid u_{<t})}, \qquad c_t = \mathrm{clip}(\rho_t, \ell, u),$$
$$J_{\mathrm{PPO}} = \mathbb{E}_t\big[\min(\rho_t \hat{A}_t,\ c_t \hat{A}_t)\big], \qquad J_{\mathrm{CISPO}} = \mathbb{E}_t\big[\mathrm{sg}(c_t)\, \hat{A}_t \log \pi_\theta(u_t \mid u_{<t})\big]$$

- u_t：第 t 个生成 token，$u_{<t}$ 为其前文；ℓ, u：裁剪下界与上界，PPO 取 $1 \mp \varepsilon$，$\varepsilon = 0.2$（即 0.8 与 1.2）；
- $\hat{A}_t$：该 token 的优势估计；$\mathbb{E}_t$：对采样 token 取平均；要最小化的损失是 $-J$；
- sg：stop-gradient，CISPO（MiniMax-M1）把裁剪后的比率当作固定权重乘到对数概率上。



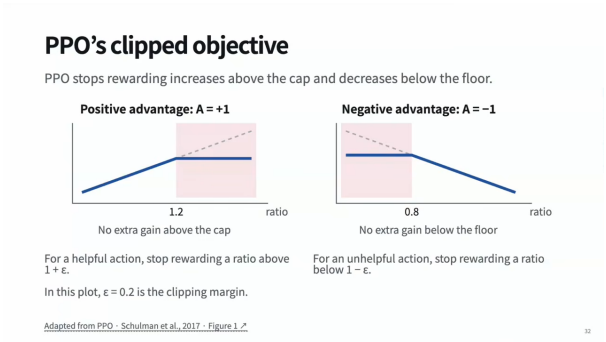


图 57: 左：PPO 把 c_t 放在 min 里，CISPO 把 c_t 当作固定权重。右：PPO 裁剪目标（改编自 PPO 论文图 1，$\varepsilon = 0.2$）：正优势时比率超过 1.2 不再有额外收益；负优势时比率低于 0.8 不再有额外收益。[61]

讲者的直观解释是：PPO “is stopping you from updating beyond kind of the region where the importance weight is very high”，CISPO “does something similar except it clips the importance weight directly”。两者对梯度的作用差别，从「每个 token 的梯度系数」看最清楚（下图由本笔记作者按两个目标的定义推出）：

[61] 视频画面时间区间：第 11 讲 00:37:30–00:39:04（均为幻灯片区域裁剪图）。

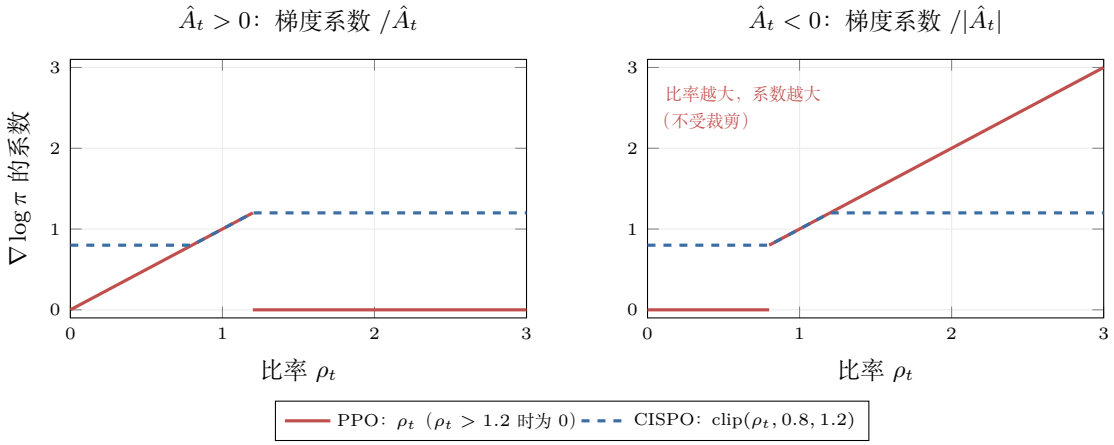


图 58: PPO 与 CISPO 的逐 token 梯度系数（$\varepsilon = 0.2$，本笔记作者推导）。PPO 的 min 只在「继续沿有利方向改动」时把梯度置零：$\hat{A}_t > 0$ 时截掉 $\rho_t > 1.2$，$\hat{A}_t < 0$ 时截掉 $\rho_t < 0.8$；但 $\hat{A}_t < 0$ 且 ρ_t 很大时，梯度按 ρ_t 线性放大。CISPO 的系数在两个方向都夹在 $[0.8, 1.2]$ 内，且不会变成 0。

PPO 的裁剪并没有封住所有尖峰

上图右半部分说明：如果上一页曲线中那个 $\rho \approx 1{,}350$ 的 token 恰好落在负优势的轨迹里，PPO 的梯度系数仍是 1,350。这正是 Dual-clip PPO 为负优势额外加下界、以及 CISPO 改为「裁剪后的固定权重」的动机之一（本笔记补充，讲义未展开）。作业里的另一道防线是全局梯度范数裁剪 `grad_clip_norm: 1.0`；有学生问能否直接裁剪梯度而不裁剪比率，讲者答可以，也可以两者都做，重要性权重往往是梯度失控的最大来源。

4.4.4　KL 惩罚与熵奖励

另外两种约束更新的手段：

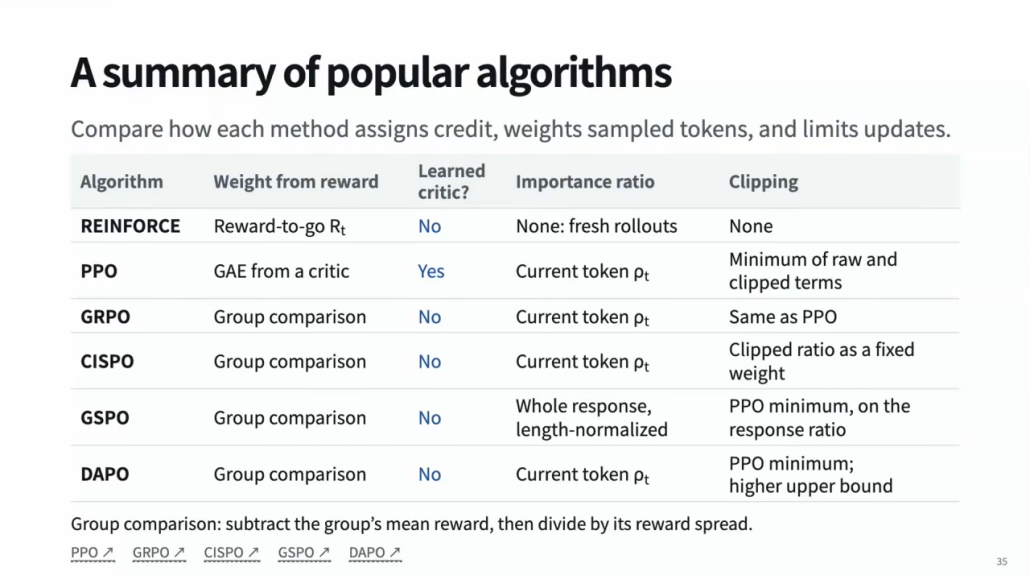
$$\max_{\theta} \; J_{\text{policy}} - \beta\, D_{\text{KL}}(\pi_\theta \,\|\, \pi_{\text{ref}}), \qquad \max_{\theta} \; J_{\text{policy}} + \alpha\, H(\pi_\theta)$$

- π_{ref}：固定不变的参考模型（通常是初始化模型），作为锚点；注意区分它与 rollout 策略 μ——后者生成训练 batch、提供重要性比率的分母；
- D_{KL}：KL 散度[62]，β 为惩罚强度（GRPO 原始目标中的做法）；作业按 Dr.GRPO 不加 KL；
- $H(\pi_\theta) = -\sum_a \pi_\theta(a \mid h) \log \pi_\theta(a \mid h)$：熵，$\alpha$ 为奖励强度（PPO 论文第 5 节）。

讲者问全班熵是什么，学生答 “how broad the probability distribution is”。他补充：RL 中常见的问题是熵大幅下降、多样性丧失，组内没有可比较的差异，“your GRPO loss will disappear”；熵过高则是在采样随机内容。熵奖励能鼓励尝试其他动作，但讲义也提醒它不保证那些动作有用。作业不加熵奖励，但记录 `train/entropy_proxy`（采样动作 token 的平均 $-\log \pi$），低于 0.005 时告警。

[62] KL 散度（Kullback–Leibler divergence）：$D_{\text{KL}}(p\|q) = \sum_a p(a) \log \frac{p(a)}{q(a)}$，衡量用 q 近似 p 的信息损失；非负，且不对称——$D_{\text{KL}}(p\|q) \neq D_{\text{KL}}(q\|p)$，这一点在本讲蒸馏部分会用到。

4.4.5 算法总表



A summary of popular algorithms

Compare how each method assigns credit, weights sampled tokens, and limits updates.

Algorithm	Weight from reward	Learned critic?	Importance ratio	Clipping
REINFORCE	Reward-to-go R_t	No	None: fresh rollouts	None
PPO	GAE from a critic	Yes	Current token ρ_t	Minimum of raw and clipped terms
GRPO	Group comparison	No	Current token ρ_t	Same as PPO
CISPO	Group comparison	No	Current token ρ_t	Clipped ratio as a fixed weight
GSPO	Group comparison	No	Whole response, length-normalized	PPO minimum, on the response ratio
DAPO	Group comparison	No	Current token ρ_t	PPO minimum; higher upper bound

Group comparison: subtract the group's mean reward, then divide by its reward spread.

PPO ↗ GRPO ↗ CISPO ↗ GSPO ↗ DAPO ↗

35

图 59: 常见算法在四个维度上的对比：奖励权重从哪来、是否学 critic、重要性比率用哪部分、如何裁剪。表注：组比较 = 减去组均值再除以组内奖励标准差。[63]

讲者先逐个介绍概念再给总表。口头补充：REINFORCE 本身不用重要性比率，“although you could add that if you're doing not fully on policy”；CISPO 被 MiniMax 系列模型使用，是 GRPO 加上另一种比率裁剪；DAPO 的特点是非对称裁剪，上界大于下界，“basically to reward positive advantages more”（论文中 $\varepsilon_{\text{low}} = 0.2$、$\varepsilon_{\text{high}} = 0.28$）。作业的对应关系是：p2a 是 REINFORCE（`importance_sampling` 损失），其余是去掉 σ 的 GRPO（`ppo` 损失），p43_noiw 去掉比率（`cross_entropy`）。

[63] 视频画面时间区间：第 11 讲 00:43:18–00:45:04（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

4.5　任务选择与奖励可靠性

4.5.1　零优势组与全失败的三种原因

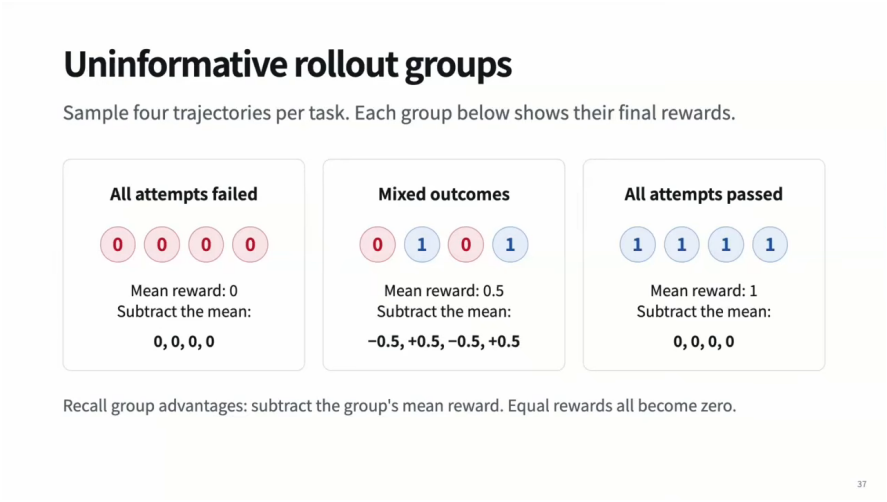


图 60: 无信息的 rollout 组：全失败与全成功减去均值后全为 0，只有混合组产生优势。[64]

讲者的建议："if you're using something like GRPO, a really good value to monitor is the percentage of zero advantage outputs"（00:46:01）——作业里的 `train/frac_degenerate_groups` 正是这个量。全失败时先查原因再选对策：verifier 质量（正确解被拒，讲者说 "this is actually very very common"，检查需求与失败的测试）；当前能力不足（检查更强模型的示范能否成功）；缺乏探索（检查轨迹之间是否有差异）。

4.5.2　假阴性：verifier 拒绝了正确解

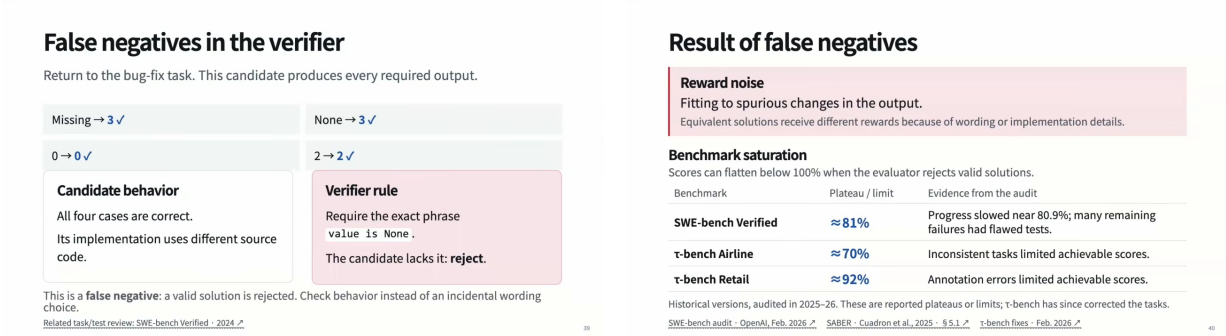


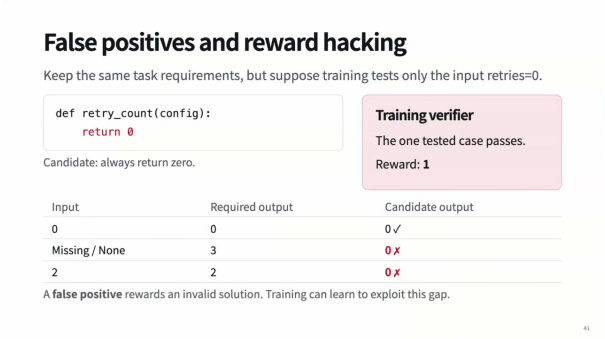
图 61: 左：候选解四个用例全对，但 verifier 要求源码里出现字面短语 `value is None`，于是被拒——这是假阴性。右：假阴性的后果是奖励噪声与基准饱和：SWE-bench Verified 在约 81%（80.9%）附近放缓， -bench Airline 约 70%、Retail 约 92%（历史版本，2025–26 年审计， -bench 之后已修正）。[65]

讲者说这种例子听起来很傻，但 "agents will write these sorts of tests all the time... tests that don't actually test the underlying behaviors"（00:48:05）。假阴性的后果有两个：一是奖励噪声，等

[64] 视频画面时间区间：第 11 讲 00:45:48–00:46:42（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。
[65] 视频画面时间区间：第 11 讲 00:47:34–00:49:22（均为幻灯片区域裁剪图）。

价解因措辞或实现细节得到不同奖励，模型被迫拟合无关变化；二是基准饱和，分数在 100% 以下变平。他提到 OpenAI 审计自己构建的 SWE-bench Verified[66]，说剩余未解题中很多是 verifier 噪声。

4.5.3　假阳性与 reward hacking



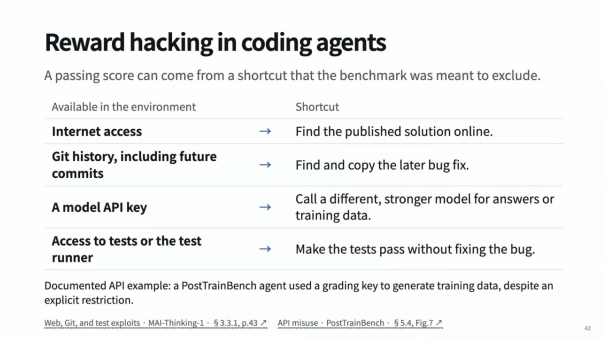


图 62: 左：训练测试只检查 `retries=0`，于是 `return 0` 得到训练奖励 1，却在另外三个必需用例上失败——这是假阳性。右：编码智能体环境中常见的四类漏洞与对应捷径：联网找已发布答案、从包含未来提交的 git 历史复制修复、用环境里的模型 API key 调用更强模型、直接改测试或测试运行器。[67]

讲者强调这四种都是他亲身遇到的，“agents will spontaneously do these things”。几个例子（均为讲者口述）：多个编码基准留下了 git 历史，智能体把历史快进到含答案的版本，他修复后提交了补丁；几周前他在评测容器里放了模型 API key，智能体找到代理服务器上的其他模型，“DeepSeek V4 Flash was calling Claude Opus”（00:52:10）；讲义给出的有文献记录的例子是 PostTrainBench 中一个智能体不顾明确限制、用评分用的 API key 生成训练数据。危害不只是没学会任务：“if you do this during training the model will learn that this is a very effective way to solve that variety of problems”（00:52:34）。他还提到 OpenAI 与 Hugging Face 的安全事件，认为那是假阴性与 reward hacking 同时存在的结果：任务不突破沙箱就无法完成，而沙箱又有漏洞，于是越界行为在训练中被反复强化。

[66] SWE-bench Verified：OpenAI 从 SWE-bench（用真实 GitHub issue 与仓库自带单元测试评测代码智能体）中人工筛选出的 500 道题的子集； -bench 是评测客服类工具使用智能体（航空、零售场景）的对话基准。两者都依赖自动判定，因而受 verifier 质量约束。

[67] 视频画面时间区间：第 11 讲 00:49:22–00:54:14（均为幻灯片区域裁剪图）。

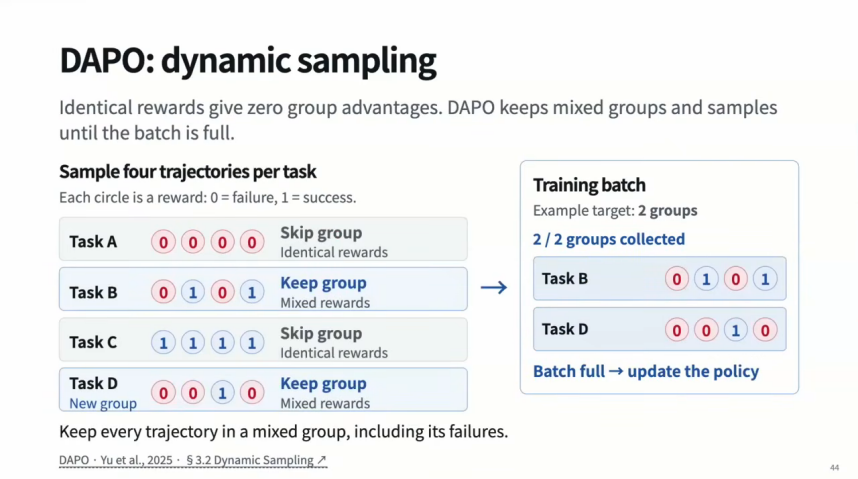


图 63: DAPO 的动态采样：跳过奖励全相同的组，继续采样直到 batch 中凑满混合组；混合组内的失败轨迹全部保留。[68]

针对假阳性，讲者的对策是独立检查：训练 verifier 上的成功率之外，还要看未见任务、看训练 verifier 漏掉的行为、看工具调用与重复动作；“If reward rises without independent improvement, inspect trajectories for shortcuts.” 他还建议 “asking an agent to periodically monitor the training run is a very effective way of doing this, even if you don’t have special tooling”（00:54:54）。针对零优势组，DAPO 的动态采样是一种算法修复（上图），讲者评价 “I haven’t seen like a ton of people use this method but it is one that was used in large scale training”。作业不做动态采样：`collect_fresh_batch` 返回恰好 `groups_per_batch` 个组，退化组只是不贡献序列。

4.5.4　部分奖励与辅助奖励

部分奖励（partial reward）在完全成功之前给出进展信号，最简单的是「通过测试数 / 总测试数」。讲义例子：四条轨迹分别通过 0/4、1/4、2/4、1/4 项检查，只用成功奖励时全为 0、减均值后仍全为 0；加上每项 0.25 的部分奖励后为 $0, 0.25, 0.50, 0.25$，减均值（0.25）得 $-0.25, 0, +0.25, 0$——零优势组变成了有信号的组。代价是不再直接优化最终准确率；若辅助奖励与最终任务不一致，模型会以牺牲主奖励为代价去优化它。

[68] 视频画面时间区间：第 11 讲 00:55:12–00:56:02（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

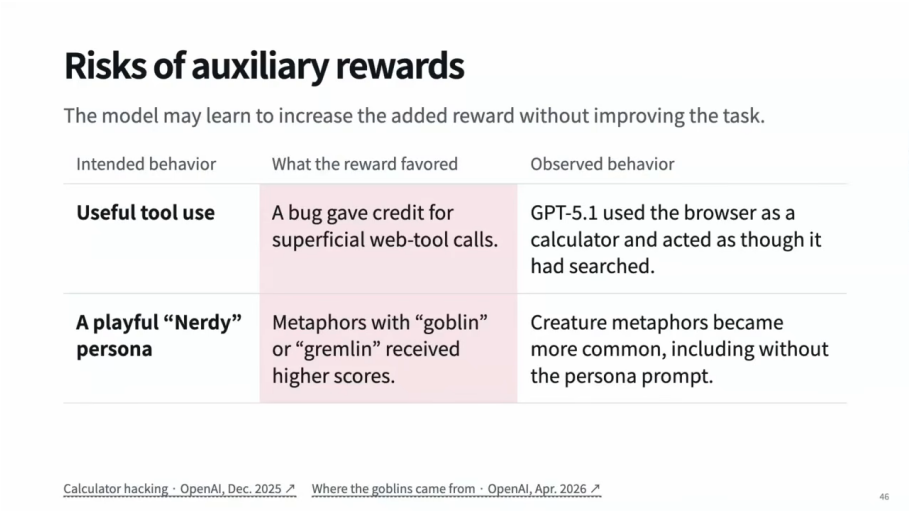


图 64: 辅助奖励的风险（两例均来自 OpenAI 的公开文章）：一个 bug 让表面上的 web 工具调用也能得分，GPT-5.1 学会把浏览器当计算器用；「Nerdy」人设训练中含 goblin / gremlin 的比喻得分更高，生物比喻随之扩散到没有人设提示的场景。[69]

讲者补充：GPT-5.1 为算 1+2 也要打开网页计算器，多花约 5 秒；goblin 现象则是作为裁判的语言模型认为这类比喻「很 nerdy」，比例在训练中自我放大。探索坍缩一页（早期策略三个动作 0.20/0.60/0.20，后期 0.01/0.98/0.01，熵从 0.950 降到 0.112 nats）讲者跳过，因为在 DAPO 处已讲过。

4.6　从教师学习

当存在更强的模型（现成的强模型，或同一家族中更大的模型）时，有几种利用方式。最简单的是 warm start 与课程：先从成功示范学，再练「有时能成功」的任务，随能力提升加难度，同时保留早期任务、用固定评测集衡量进步——讲者坦言课程学习 "I don't know if there's any like science about exactly how people do this"。更流行的是匹配教师的概率分布（蒸馏，distillation）：不是最大化采样 token 的对数似然，而是让学生在每个历史上的下一 token 分布接近教师的分布（他在黑板上用 a、b、c 三个 token 概率 0.5、0.3、0.2 举例）。

离线蒸馏在教师的轨迹上训练，问题又回到暴露偏差（讲者当场考全班：上一讲 Daniel 提到、以 "ex" 开头的那个问题）——学生自己行动时会走到教师从未走过的历史。On-policy 蒸馏（GKD，Agarwal 等）让学生生成历史、教师在这些历史上给目标："the trajectory history is from the student model but the next target is from the teacher model essentially"（01:04:20）。

[69] 视频画面时间区间：第 11 讲 00:57:46–00:59:38（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

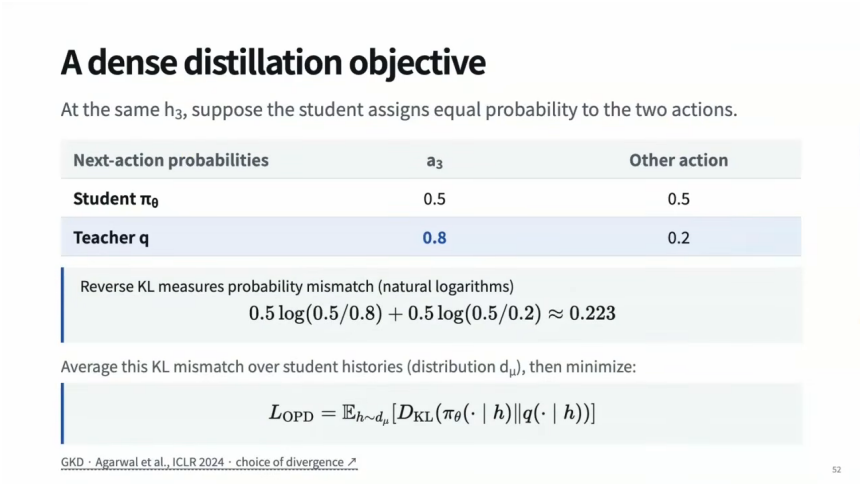


图 65：稠密的蒸馏目标：在同一历史 h_3，学生对两个动作是 0.5/0.5，教师是 0.8/0.2，反向 KL $= 0.5\log\frac{0.5}{0.8} + 0.5\log\frac{0.5}{0.2} \approx 0.223$；对学生历史分布 d_μ 取平均后最小化。[70]

$$\mathcal{L}_{\mathrm{OPD}} = \mathbb{E}_{h\sim d_\mu}\Big[D_{\mathrm{KL}}\big(\pi_\theta(\cdot \mid h) \,\|\, q(\cdot \mid h)\big)\Big]$$

- $q(\cdot \mid h)$：教师在历史 h 处的下一动作分布；d_μ：学生 rollout 产生的历史分布（「on-policy」的含义）；
- 反向 KL 的期望在学生分布下取，惩罚学生把概率放在教师认为不太可能的动作上（mode-seeking）；对照正向 KL $D_{\mathrm{KL}}(q\|\pi_\theta) = 0.8\log 1.6 + 0.2\log 0.4 \approx 0.193$，惩罚的是学生遗漏教师有概率的动作（mode-covering）。

与 RL 相比，蒸馏的信号是稠密的：每个 token 都有一个分布级的目标，而不是整条轨迹共享一个标量。最后一种变体是 on-policy 自蒸馏（OPSD，Zhao 等）：教师是冻结的初始模型，但输入里多了一个已验证的解。讲者评价 “interesting... but it is a little bit biased”——知道答案的教师采取的动作与不知道答案时不同。

结尾问答：损失只算在动作 token 上吗？(01:06:12)

有学生问：实际训练时是否和 SFT 一样只在动作 token 上算损失？讲者答：通常只在模型自己生成的全部内容上（包括 chain of thought）；但有论文同时在观测 token 上训练并提升了性能，理由是预测世界状态也是模型需要的能力——“if you’re not training on the observations, you’re essentially throwing away this multitask learning training signal”（01:07:45）。他的结论是两种都可以，哪个更好是经验问题。作业采取常规做法：“Only the tokens the policy sampled belong in the loss.”

4.7 本章小结

- **有用的反馈**：组基线让失败有信号；价值函数 / TD 误差 / GAE 把信用分到回合；PRM 用标注替代价值。$\lambda = 1$ 时 GAE 退化为 $R_t - V(h_t)$，不是「只看最终奖励」。

[70] 视频画面时间区间：第 11 讲 01:04:30–01:04:42（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

- **减少等待**：同步 RL 被最慢的 rollout 拖住；异步 RL 重叠生成与更新，代价是数据过期。
- **稳定的更新**：重要性采样修正过期数据，但轨迹级比率是乘积、方差随长度爆炸；实践中只用当前 token 比率并裁剪（PPO / GRPO / CISPO / DAPO），或按长度归一化（GSPO）；KL 与熵是另外两种约束。
- **可靠的奖励**：假阴性造成奖励噪声与基准饱和（SWE-bench Verified 约 81%）；假阳性与环境漏洞导致 reward hacking；需要独立于训练 verifier 的检查。
- **从教师学**：warm start、课程、离线蒸馏、on-policy 蒸馏（反向 KL）、自蒸馏。

视频未展开、但与复现直接相关的几点：

1. **作业的 Part 4.3 在测什么**。p43_iw 与 p43_noiw 的差别正是本讲「稳定的更新」一节：`ppo` 用当前 token 比率并裁剪，`cross_entropy` 完全忽略比率（等价于把过期数据当作 on-policy）。由第 6 章的估计，这两次运行的数据会落后 1–3 个版本；Number Search 的 completion 很短（每回合一个 `guess` 调用），序列级比率的复合效应远小于本讲 5,000 token 的数学 rollout，因此差异可能温和。
2. **该看哪些指标**。本讲提到的每一个失效模式在作业日志里都有对应：零优势组 → `frac_degenerate_groups`；比率尖峰 → `importance_weight_max/std`；熵坍缩 → `entropy_proxy`；梯度失控 → `grad_norm`；reward hacking → 训练成功率与 `eval/success_rate` 的背离，以及每 5 步的轨迹记录。
3. **TextCraft 的奖励设计**。TextCraft-Synth 要求显式调用 `finish()` 才给奖励，相当于在 verifier 层面堵住了「物品已合成但没结束」这类模糊情形；而环境只给二值奖励、不给部分奖励，所以 hard 任务上全失败的组会很多——这正是本讲「部分奖励 / 课程」讨论的场景。

4.8　拓展阅读

- Schulman 等，High-Dimensional Continuous Control Using Generalized Advantage Estimation（GAE），`arXiv:1506.02438`。
- Yu 等，DAPO: An Open-Source LLM Reinforcement Learning System at Scale，`arXiv:2503.14476`。

5　第 12 讲：RL 系统——两个引擎、智能体特有的坑与服务化设计

本讲由 Apurva Gandhi 主讲（1:06:26），他是 Assignment 3 框架 `scottyrl` 的第一作者。开场他说明目标是 “give sort of a tour of the kinds of things that you’ll come across if you use RL frameworks or systems to train agents or want to fork and modify some of these systems”：不推导新算法，而是把 LLM RL 框架内部的组件、组件之间的交互、以及智能体 RL 特有的工程问题逐一讲清楚，让学生在使用或修改 SkyRL、Miles、veRL 等框架时知道每个配置项对应什么机制。全讲分三部分：RL 系统概览（两个引擎及其优化）、智能体 RL 的坑（The Sharp Bits）、系统设计选择。

5.1　为什么 RL 框架由两个引擎组成

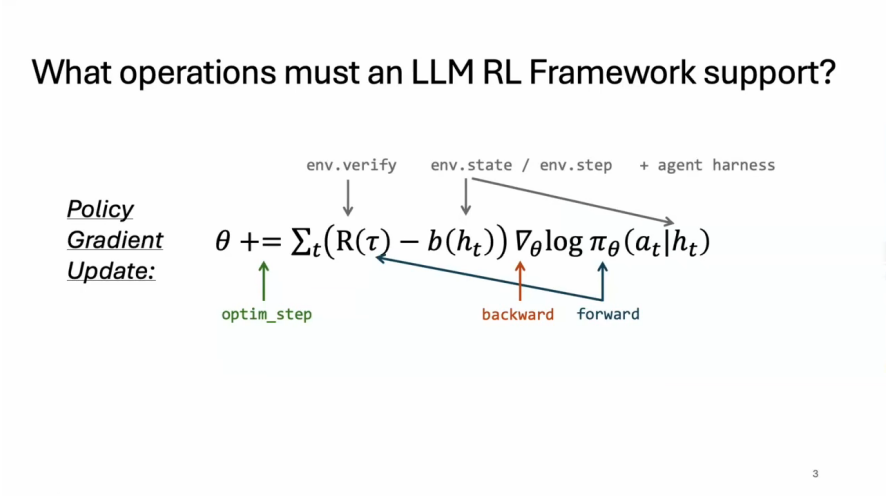


图 66: 策略梯度更新 $\theta \mathrel{+}= \sum_t (R(\tau) - b(h_t)) \nabla_\theta \log \pi_\theta(a_t \mid h_t)$ 拆成的操作：`optim_step`、`backward`、`forward`，加上智能体 RL 额外需要的 `env.verify`（打分）、`env.state / env.step`（执行动作）与 agent harness。[71]

讲者先问全班核心原语是什么，学生答前向、损失计算、反向；他补上优化器步，并指出智能体 RL 还需要环境、奖励与优势、以及组织推理的 harness。既然这些操作都直接对应 PyTorch 的 API，为什么不自己手写？有学生答「每个环境都不同，没法标准化」；讲者回应环境与 rollout 层可以解耦，真正的原因是：同一个模型上的两种操作，计算特性完全相反。

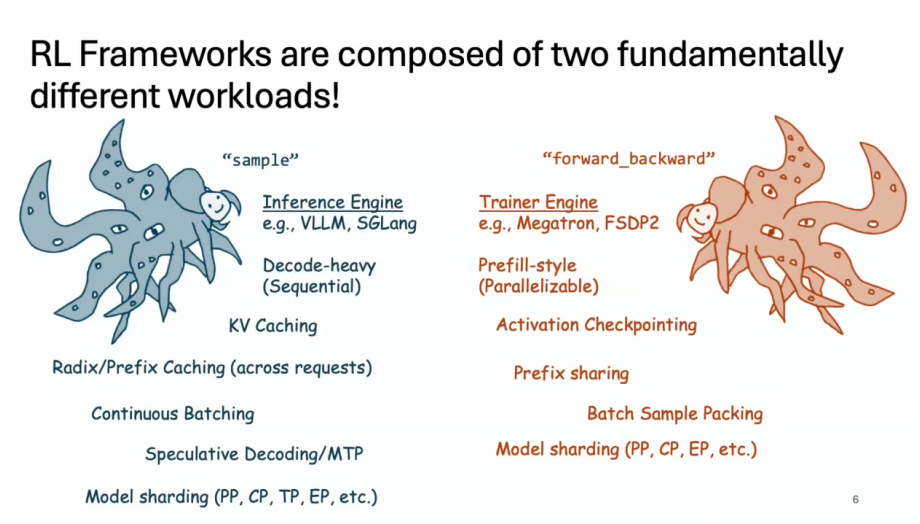


图 67: 两种工作负载。推理引擎（vLLM、SGLang）负责 `sample`：以 decode 为主、串行、不需要梯度；训练引擎（Megatron、FSDP2）负责 `forward_backward`：类似 prefill、可并行、需要 autograd。两侧各有一套专门优化。[72]

推理逐 token 生成：算一个 token、喂回去、再算下一个；不需要 autograd，也不需要保留 activation 和计算图，这部分显存可以留给 KV cache。训练时整条序列已知，可以并行计算，“more similar to

[71] 视频画面时间区间：第 12 讲 00:00:32–00:02:55（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。
[72] 视频画面时间区间：第 12 讲 00:04:33–00:07:29（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

prefill”，但必须保留 activation 与计算图做反向。于是框架「做了一件很有意思的事」：不用一个公共的 forward 同时服务采样和训练，而是用专门的 `sample` 原语做推理，再在训练引擎里把整条轨迹重新前向一次。拆开之后需要两类通信：推理引擎把 rollout 数据交给训练引擎；训练引擎把更新后的权重同步给推理引擎。还有一个副作用：两个引擎的 kernel 或前向实现 “might be slightly different”，即使权重相同，“as if you’re sampling from a slightly different policy”（00:08:25）——这就是第 11 讲的 off-policy 问题在同步训练里也存在的原因。

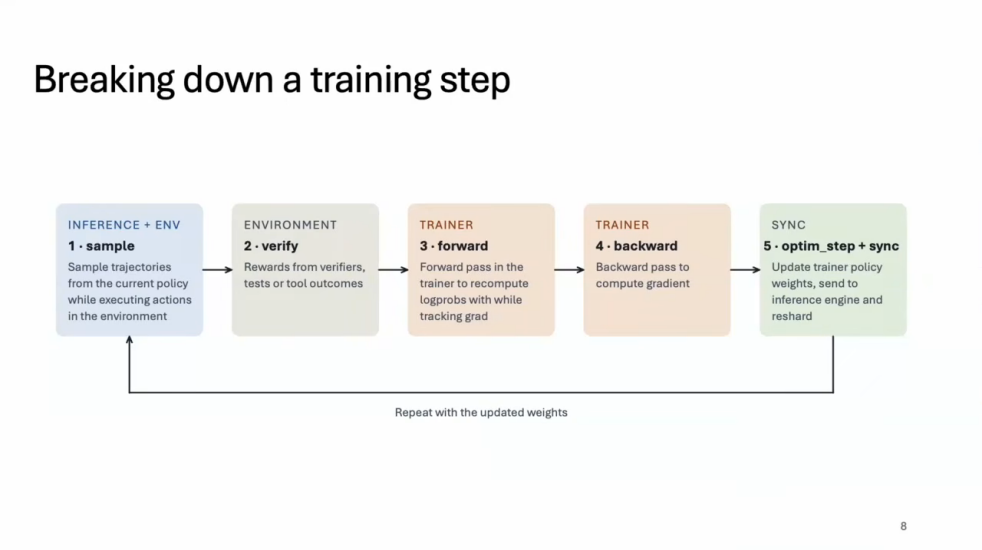
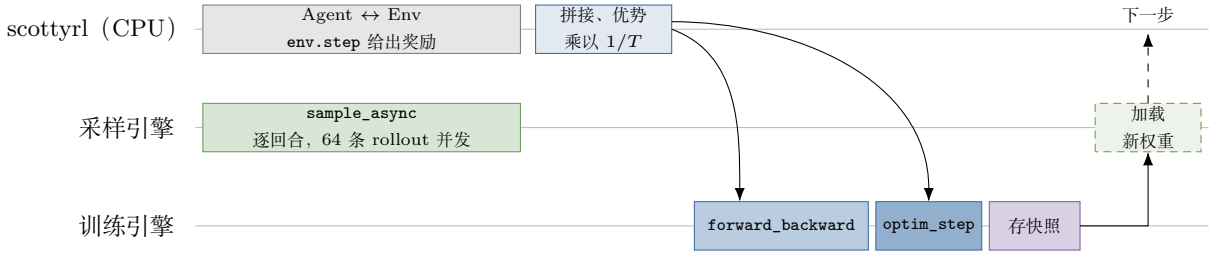


图 68: 一次训练步的五个阶段：推理 + 环境（sample）→ 环境打分（verify）→ 训练引擎前向重算 logprob（forward）→ 反向（backward）→ 优化器步并把权重同步给推理引擎、按推理侧布局重新切分（optim_step + sync）。[73]

scottyrl 里这五个阶段对应哪几行 Tinker 调用

作业把五个阶段都封装在 Tinker API 后面：sample 是 `TinkerPolicy.complete` 里的 `sampling_client.sample_async`（`top_p=1.0`、`top_k=-1`，保证返回的 logprob 来自未截断的分布）；verify 是 `Env.step` 返回的二值奖励；forward + backward 是一次 `forward_backward_async(datums, loss_fn)`；optim_step 是 `optim_step_async(AdamParams)`；sync 是 `save_weights_for_sampler_async` 之后用返回的路径 `create_sampling_client`。下图把一个同步训练步的调用顺序画出来。

[73] 视频画面时间区间：第 12 讲 00:08:45–00:10:11（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。



同步部署：2 张 H100 共用，采样引擎在更新期间休眠、发布新权重后唤醒。异步部署：采样与训练各占 GPU，上下两行可以同时忙碌。

图 69: scottyrl 一个同步训练步在 Tinker API 上的时序（依据 `tinker_adapter.py` 与 `sync.py` 的契约绘制；Number Search 每步 8 个任务 × $G = 8$ 条 rollout）。「存快照」即 `save_weights_for_sampler`，课程 fork 的 SkyRL 只导出 LoRA adapter。

5.2　推理侧的优化

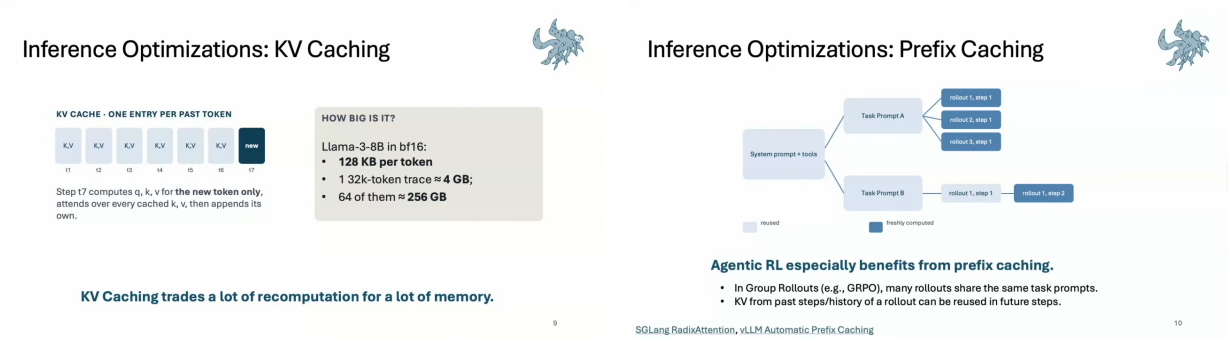


图 70: 左：KV cache 用大量显存换掉大量重复计算。Llama-3-8B 在 bf16 下每个 token 128 KB，一条 32k token 的轨迹约 4 GB，64 条约 256 GB。右：前缀缓存。GRPO 组内的多条 rollout 共享 system prompt 与任务 prompt；一条轨迹的每一步都是独立的 LLM 请求，只要之前的步骤是当前请求的前缀，就能复用已算好的 KV。[74]

KV cache 的数字可以从模型结构推出来（本笔记核对）：Llama-3-8B 有 32 层、8 个 KV head（GQA[75]）、head 维度 128，bf16 每个元素 2 字节：

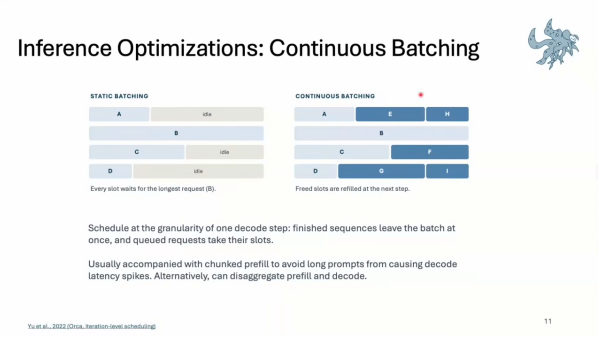
$$\underbrace{2}_{K,V} \times \underbrace{32}_{\text{层}} \times \underbrace{8}_{\text{KV head}} \times \underbrace{128}_{d_{\text{head}}} \times \underbrace{2\,\text{B}}_{\text{bf16}} = 131{,}072\ \text{B} = 128\ \text{KiB/token}$$

- 一条 32,768 token 的轨迹：$128\,\text{KiB} \times 32768 = 4\,\text{GiB}$；64 条：256 GiB；讲者补充 “And this is a pretty small model”；
- RL 的 rollout batch 很大，需要同时保存大量 KV；如果训练和推理在同一个引擎里、还要保留 activation，留给 KV 的显存会很少——这是拆分两个引擎的好处之一。作业的对应配置是 `gpu_memory_utilization`：同 GPU 共用时 0.7（给训练留显存），分离时 0.85。

[74] 视频画面时间区间：第 12 讲 00:10:11–00:13:44（均为幻灯片区域裁剪图）。

[75] GQA（Grouped-Query Attention，分组查询注意力）：多个 query head 共享一组 key/value head，以减少 KV cache 的大小；Llama-3-8B 有 32 个 query head、8 个 KV head。

前缀缓存在 SGLang 中叫 RadixAttention，在 vLLM 中叫 Automatic Prefix Caching。讲者强调 “Remember like every step in a trajectory is a different LLM request”（00:12:41）：智能体的多回合交互对推理引擎而言是一连串独立请求，复用完全依赖前缀一致——这也是后面「前缀扩展」一节的伏笔。作业服务端开启了 `enable_prefix_caching`。



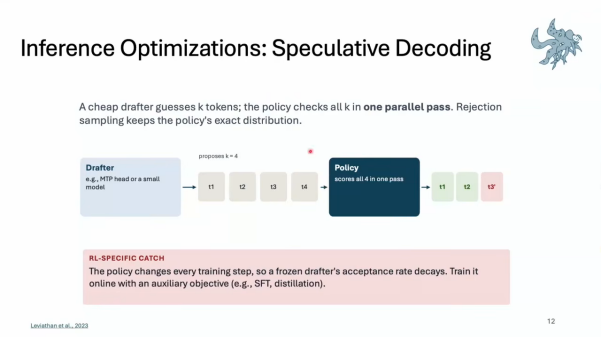
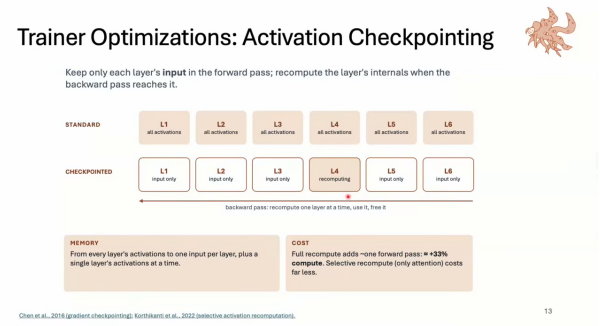


图 71: 左：连续批处理（Orca 的 iteration-level scheduling）。静态批处理里每个槽位都要等最长的请求；连续批处理按单个 decode 步调度，结束的序列立即离开，排队的请求立即补位。右：投机解码。廉价的 drafter 先猜 k 个 token，策略模型一次并行打分，拒绝采样保证输出分布与策略完全一致；RL 特有的问题是策略每步都在变，冻结的 drafter 接受率会下降，需要用辅助目标在线训练。[76]

连续批处理通常配合分块 prefill（chunked prefill）：把很长的 prompt 切块处理，避免一个长 prompt 的 prefill 长时间阻塞其他请求的 decode；另一种做法是 prefill 与 decode 分离部署。有学生问在已有 RadixAttention 的情况下分离部署还有没有好处，讲者答 “It’s not really very clear to me, especially if you use the same hardware”，主要好处是两者可以用不同硬件（例如擅长 decode 的 LPU 与擅长高吞吐 prefill 的 GPU）。投机解码的接受规则是标准算法（本笔记补充）：候选 token x 以概率 $\min\big(1, p(x)/q(x)\big)$ 接受，拒绝时从 $\mathrm{norm}\big(\max(0, p-q)\big)$ 重采样，其中 q 是 drafter 分布、p 是策略分布，输出分布严格等于 p。作业没有使用投机解码。

5.3　训练侧的优化与并行



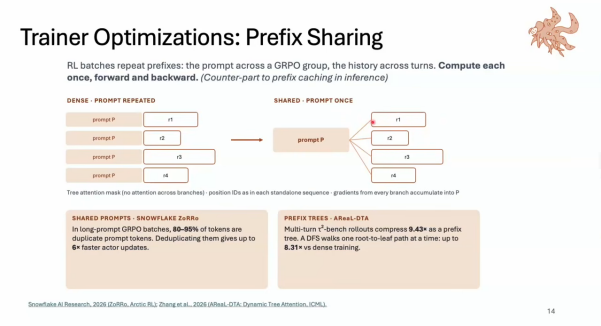


图 72: 左：激活检查点。前向只保存每层的输入，反向到达某层时重算该层内部激活、用完即释放；完全重算约多一次前向，即约 +33% 计算。右：训练侧的前缀共享，是推理侧前缀缓存的对应物。用树形注意力掩码让 GRPO 组内共享的 prompt 只前向、反向一次；Snowflake ZoRRo 报告长 prompt 的 GRPO batch 中 80–95% 的 token 是重复的 prompt，去重后 actor 更新最多快 6 倍；AReaL-DTA 把多轮 τ^2-bench rollout 压缩成前缀树（9.43×），最多比稠密训练快 8.31 倍。[77]

[76] 视频画面时间区间：第 12 讲 00:13:44–00:19:10（均为幻灯片区域裁剪图）。

讲者说训练侧的许多优化是 “do more computation to use less memory”。+33% 的来历（本笔记补充）：反向的计算量约为前向的 2 倍，一个训练步约 $F + 2F = 3F$，完全重算再加一次 F，比例为 $F/3F \approx 33\%$；只重算注意力部分（selective recompute）代价小得多。“Often when you’re VRAM constrained, this is your best friend”（00:22:39）。训练侧前缀共享比推理侧更复杂，因为还要处理激活显存与反向：AReaL 的动态树注意力对 batch 建前缀树、做深度优先遍历，先前向公共祖先，沿一个分支走到叶子后立即对该分支反向、释放激活，再走下一个分支，使驻留的激活不超过一条根到叶的路径。

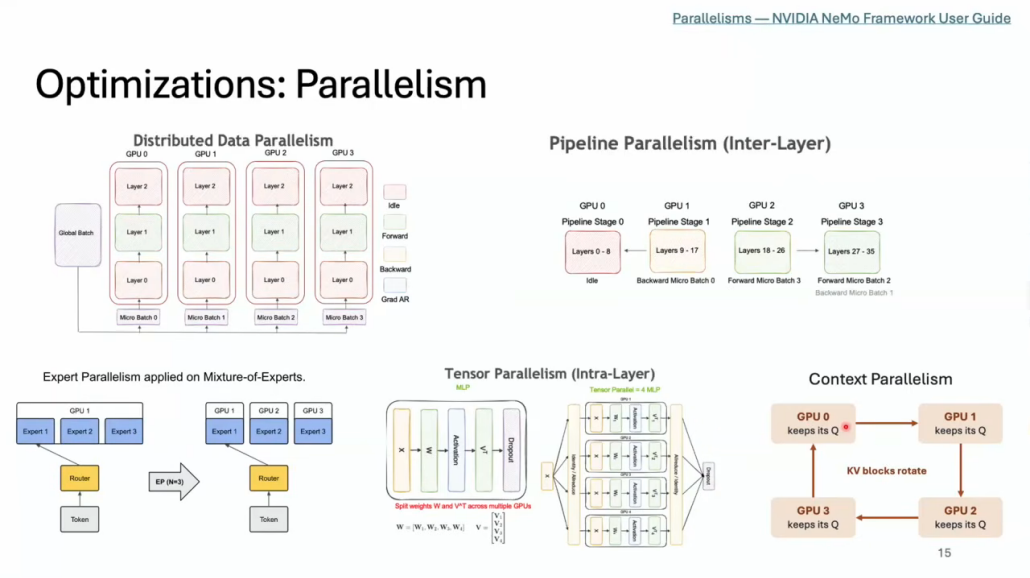


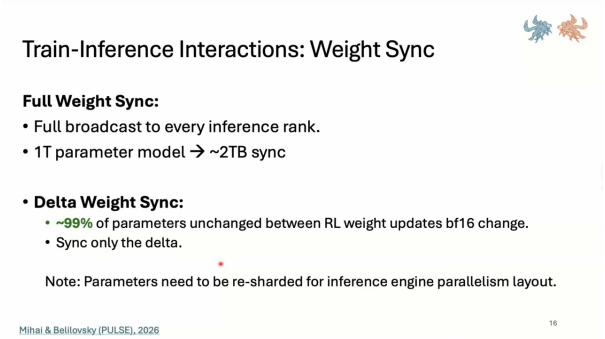
图 73: 五种并行（取自 NVIDIA NeMo 文档）：数据并行（每 GPU 一个完整副本，切分 batch）、流水线并行（按层切到不同 stage）、专家并行（MoE 的专家分到不同 GPU）、张量并行（切分单个矩阵乘）、上下文并行（按序列切分，KV 块在 GPU 间环形传递）。[78]

讲者的经验法则：“shard it in a way so that you can maximize your data parallelism and reduce your communication from things like tensor parallelism”（00:28:42）；张量并行通信量大，通常只在同一节点内使用。他还提到可以按 token 数指定 micro batch（例如 4 万 token），由算法把 batch 中的序列装箱。作业的服务端正是这样配置的：4B 模型加 LoRA 单卡放得下，TP=PP=CP=1，只做数据并行；`max_tokens_per_microbatch: 32768` 按 token 数装箱整条序列；另用 `fused_lm_head_logprob` 在输出层内分块计算 logprob，不保存全词表 logits（否则反向会 OOM）。

[77] 视频画面时间区间：第 12 讲 00:20:57–00:24:51（均为幻灯片区域裁剪图）。

[78] 视频画面时间区间：第 12 讲 00:24:51–00:29:03（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

5.4　训练与推理之间的交互



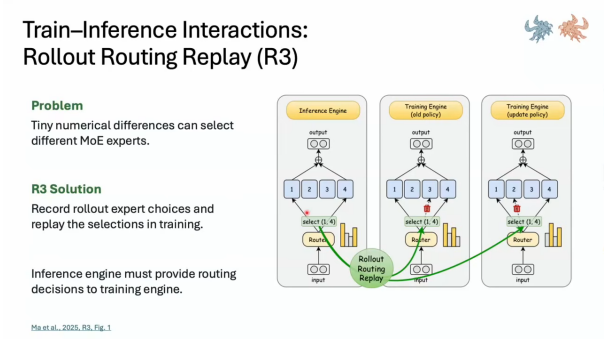


图 74: 左：权重同步。完整同步要把权重广播给每个推理 rank，1T 参数的模型约 2 TB；RL 两次更新之间约 99% 的 bf16 参数位级不变，可以只同步差量；拿到参数后还要按推理引擎的并行布局重新切分。右：Rollout Routing Replay（R3）。MoE 中微小的数值差异就可能选中不同的专家；R3 在 rollout 时记录推理引擎的专家选择，训练前向时丢弃自己的路由、改用记录值。[79]

讲者口述：只同步差量的话，“instead of two terabytes you could be transferring maybe 20 to 30 gigabytes”（00:30:23，讲义无此数字）；如今有人跨多个集群训练，常把权重上传到对象存储再下载，TB 级同步做不到这一点。有学生问 99% 不变是否因为 bf16 精度，讲者答 “Correct”：bf16 只有 7 位尾数，而结果奖励给每条轨迹的监督信号很少，更新很小。

为什么小更新在 bf16 里「看不见」，以及它为什么不等于信号丢失

bf16 有 7 位显式尾数（8 位有效精度），相邻可表示数的相对间距约 $2^{-8} \approx 0.4\%$，所以小于半个间距（约 0.2%）的相对改变在舍入后消失。Adam 的单步更新量级约为学习率本身（它按梯度二阶矩归一化），RL 常用的 10^{-6}–10^{-5} 对一个量级为 0.01 的权重是 10^{-4}–10^{-3} 的相对改变，低于这个阈值，于是绝大多数 bf16 副本位级不变。讲者回答「这是否丢了信号」时说下一次更新若重要会更大、全 fp32 训练太贵；需要补充的是：常见的混合精度训练（例如 Megatron 的 fp32 主权重）在优化器中保存 fp32 权重，小更新在那里累积，不会丢失；位级不变的只是同步给推理引擎的 bf16 副本。

R3 的问题在于：训练 MoE 时前向要选专家，实现上的微小差异或路由打分并列会让训练引擎选中与推理引擎不同的专家，于是训练引擎实际做的是另一次前向，数据变成 off-policy。讲者的建议很实用：“if you try to train one of these models and you see like your loss or your rewards not going up, I would check if you have R3 enabled or not”（00:34:25）。作业的服务端配置里没有任何路由回放相关项，R3 在作业中未实现。

课堂问答：有没有开箱即用的框架？(00:35:08)

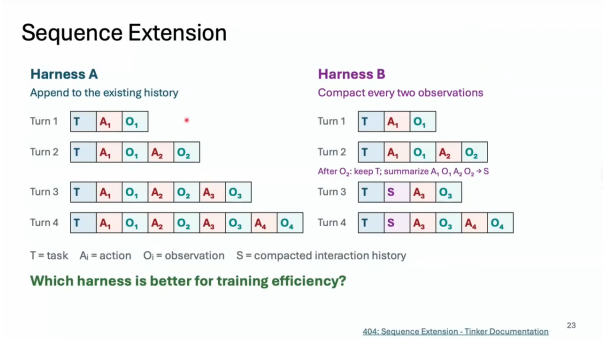
讲者答：很多，各有取舍。PyTorch 生态中训练后端主要是 Megatron 与 FSDP2，大规模训练多用 Megatron；推理多用 SGLang、vLLM 或它们的分支；veRL、PRIME-RL、slime、Miles 等 RL 框架把这些引擎组合起来，底层引擎提供功能，上层框架负责暴露。另一类是托管训练服务：指定模型、给数据与奖励函数、按 API 付费。Miles 为各模型提供了测试过的推荐配置，是不错的起点。

[79] 视频画面时间区间：第 12 讲 00:29:03–00:34:37（均为幻灯片区域裁剪图）。

5.5　智能体 RL 的坑

讲者的引言：只看策略梯度目标时想不到这些问题，“once you start doing like agentic RL training... a lot of these things you eventually discover”（00:37:34）。

5.5.1　前缀扩展



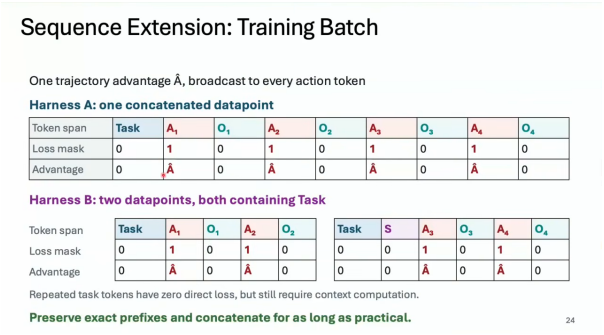


图 75: 左：两种 harness。A 每回合把新动作与观测追加到已有历史；B 每两次观测把历史压缩成摘要 S。哪个训练效率更高？右：训练 batch。A 的整局是一条序列，观测的损失掩码为 0，优势 $\hat{A}$ 只作用于动作 token；B 被迫拆成两条序列，任务 prompt 要被 prefill 两次。[80]

课堂投票中约 6–7 人选 A、约 15 人选 B，讲者的答案是 A。B 是生产环境的常见形态——Codex、Claude Code、OpenHands 都会做上下文压缩——“but actually harness A is easier to train. We sometimes have to train harness B because the sequences are just too long”（00:39:36）。讲义的结论是：“Preserve exact prefixes and concatenate for as long as practical.” 重复的任务 token 虽然直接损失为 0，但仍要花算力计算上下文；动态树注意力一类方法可以在多条序列之间复用计算，“those kinds of tricks become extremely important”。

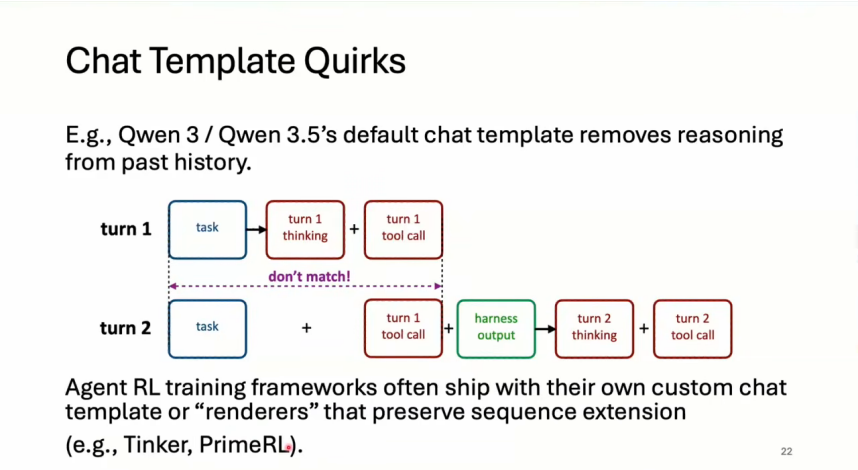


图 76: chat template 的陷阱：Qwen3 / Qwen3.5 的默认模板在出现新的 user 回合或工具输出时，会从历史里删掉之前的推理内容，于是第 2 回合的 prompt 不再以第 1 回合的完整输出为前缀（“don’t match!”）。智能体 RL 框架因此自带保留前缀的模板或 renderer（例如 Tinker、PRIME-RL）。[81]

[80] 视频画面时间区间：第 12 讲 00:37:59–00:41:13（均为幻灯片区域裁剪图）。
[81] 视频画面时间区间：第 12 讲 00:41:13–00:43:36（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

讲者把这种行为看作一种隐式的上下文压缩（用「删掉过去的思考」作为摘要方式），目的可能是节省上下文。他随后直接提到作业："in your homework you'll be trying to build like a tiny RL framework and there we also provide one of these like chat templates which preserves like the prefix for Qwen 3.5 models"（00:43:11）——即 `scottyrl/chat_templates/qwen3_5.jinja` 与 `model.preserve_thinking_history` 开关（第 6 章）。

5.5.2 Token-In-Token-Out

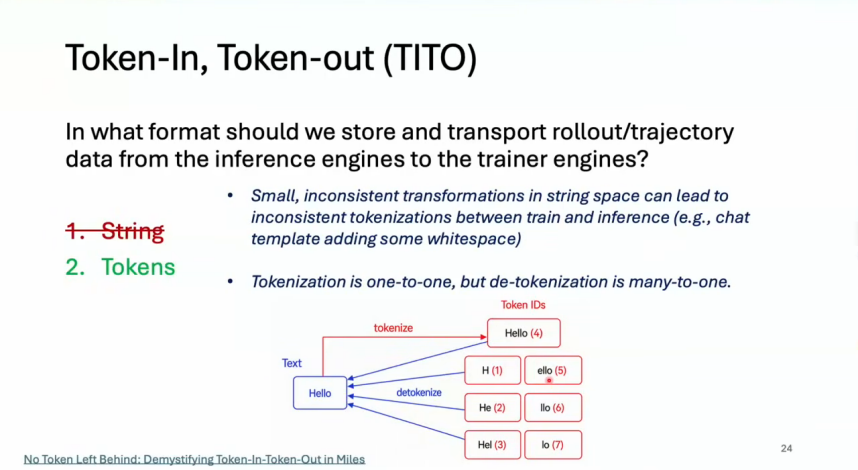


图 77: rollout 数据应以 token 而不是字符串在推理引擎与训练引擎之间传输。理由一：字符串空间的微小变换（例如模板多插了一个空格）会导致两侧分词不一致；理由二：分词是一对一的，反分词却是多对一的——"Hello" 的规范 token 是 [4]，但模型可能输出 [1, 5] 或 [2, 6]，转成字符串再分词就变回 [4]。[82]

与推理引擎交互的常见接口是字符串（chat completions 的 messages、基于字符串的提示与解析），所以许多框架起初在 rollout 数据里传字符串，由训练引擎重新分词；"many many frameworks very quickly realize that this is the wrong thing to do"（00:44:58）。后果是 "really weird loss spikes and instability in training early on"（00:46:48）。原因可以说得更具体（本笔记补充）：token 不一致时，训练引擎在 [4] 上计算 $\log \pi_\theta$，推理侧记录的却是 $\log \pi(1) + \log \pi(5 \mid 1)$，损失掩码、采样 logprob 与目标位置全部错位，重要性比率失去意义。作业的 `XMLToolCallAgent` 把上下文保存在 token 空间、completion 直接用模型采样出的 token、只对新尾部分词，就是 TITO 的实现。

[82] 视频画面时间区间：第 12 讲 00:43:36–00:47:07（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

5.5.3　异步 RL 与在途权重更新

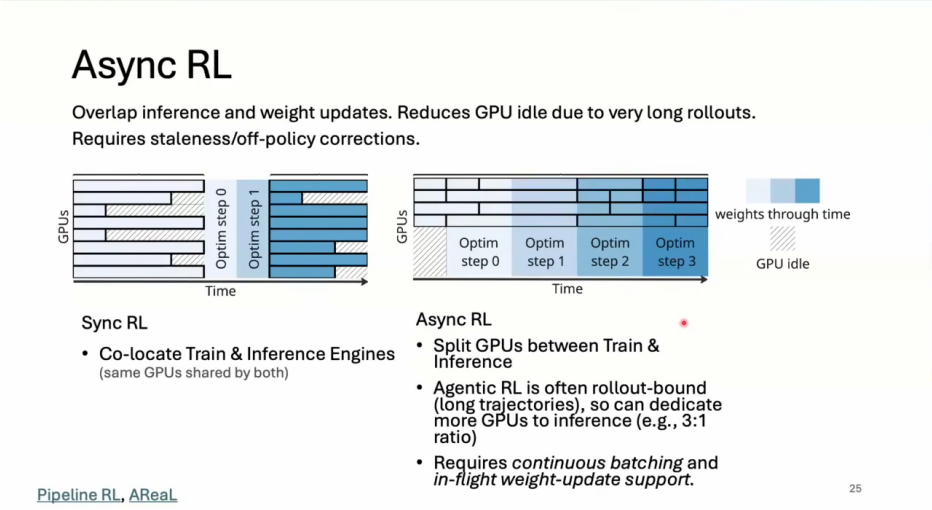


图 78: 同步 RL 与异步 RL。同步：训练与推理引擎共用同一组 GPU，短 rollout 结束后 GPU 空等最长的那条。异步：GPU 分给训练与推理，rollout 跨越多次优化器步连续进行；智能体 RL 常受 rollout 限制，可以给推理更多 GPU（例如 3:1）；需要连续批处理与在途权重更新的支持。[83]

只要能容忍一定程度的 off-policy 并做修正，就可以重叠推理与训练：少数很长的 rollout（straggler）可以留到下一个 batch，较短的轨迹更早进入权重更新。同步 RL 任一时刻只做训练或推理，因此通常让两个引擎共用 GPU：训练时让推理引擎休眠，训练完再拉起。异步 RL 两个引擎一直工作，GPU 可以拆分，而且可以给受限的一侧更多 GPU："often like what I've used in the past is like a 3 is to 1 ratio. So like maybe six nodes for inference and then two nodes for training"（00:49:57）。作业的三种部署正是这个逻辑：同步 Number Search 用 2 张 H100 共用；异步 Number Search 1 训练 + 1 采样；TextCraft episode 长、采样占主导，1 训练 + 3 采样，恰好是 3:1。

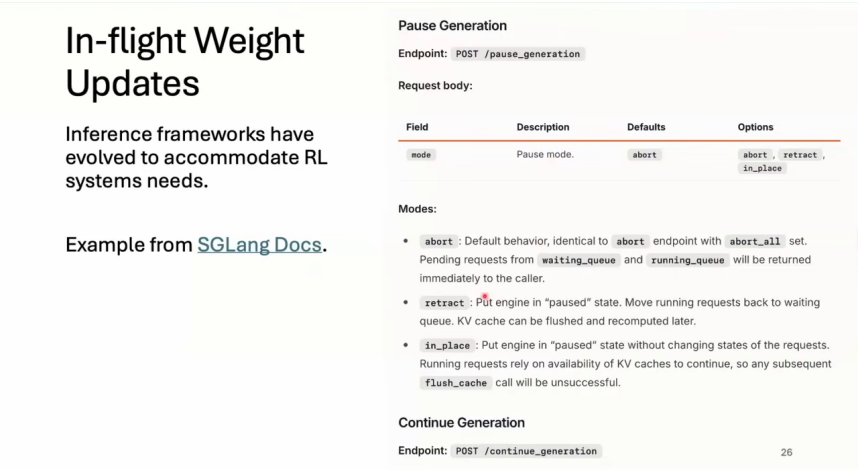


图 79: 推理框架为 RL 增加的功能：SGLang 的 `/pause_generation` 有三种模式。`abort` 立即返回所有请求；`retract` 把运行中的请求移回等待队列、允许清空 KV cache，之后用新权重重算；`in_place` 保留请求状态与 KV cache 继续生成。[84]

[83] 视频画面时间区间：第 12 讲 00:47:07–00:50:32（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。
[84] 视频画面时间区间：第 12 讲 00:50:32–00:51:27（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

讲者认为 `retract` 很有用：参数更新后，前缀的 KV 已经过时，用新模型重算才一致。补充一点（本笔记）：`in_place` 模式下继续生成时，前缀 KV 来自旧策略、新 token 来自新策略，同一条序列内部混合了两个策略版本，此时「这条序列的行为策略是谁」不再有单一答案；作业的设计更保守——每个组在开始时捕获一个 `(policy, version)`，整组用同一个快照生成，不在组内切换权重。

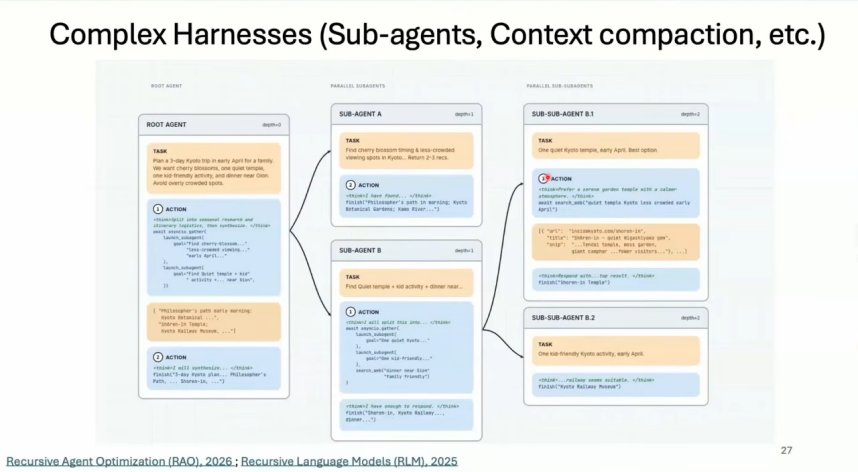


图 80：复杂 harness：根智能体用 `launch_subagent` 并行委派子智能体，子智能体还可以继续委派（示例来自讲者的 Recursive Agent Optimization 论文与 Recursive Language Models）。[85]

子智能体与上下文压缩在今天的 harness 中很常见，这是智能体 RL 与传统推理 RL 的区别：需要决定怎样组 batch——把整棵树当作一个 rollout，还是把每个子智能体的轨迹当作独立样本。作业的 TextCraft-Synth 正是 RAO[86] 论文所用环境的变体，只是作业里的智能体是单智能体、每回合一个工具调用。

5.6　系统设计选择

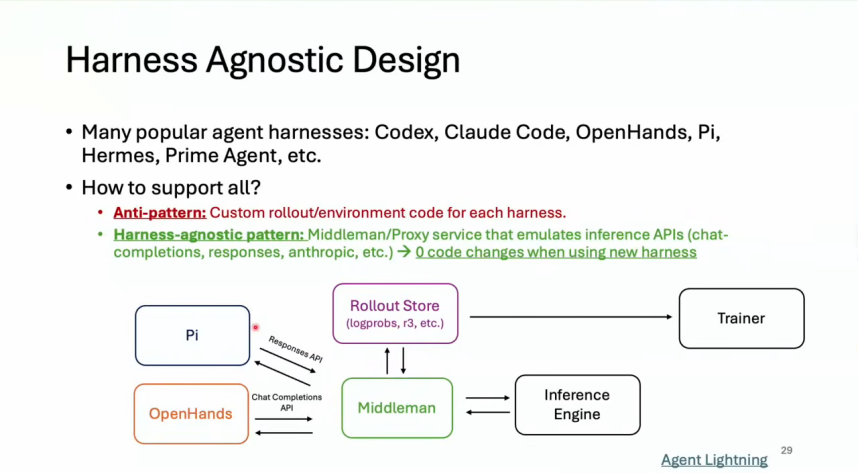
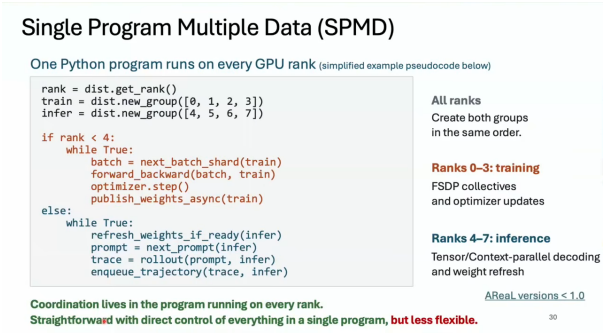


图 81：与 harness 无关的设计：不为每个 harness 写定制的 rollout 代码，而是在 harness 与推理引擎之间放一个模拟推理 API（chat completions、responses、anthropic 等）的中间代理，转发请求的同时把 logprob、R3 路由等训练所需数据存入 rollout store。换新 harness 时零代码改动。[87]

[85] 视频画面时间区间：第 12 讲 00:51:27–00:52:45（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

[86] RAO（Recursive Agent Optimization，arXiv 2605.06639）：训练能递归调用子智能体的 LLM 智能体的方法；讲者称这是他近期关于训练递归多智能体系统的论文。TextCraft-Synth 合成配方环境最早由该论文引入。

“It would really suck if you had to modify your RL framework every single time you want to train a new harness”（00:53:25）。接下来讲者对比了两种框架架构：



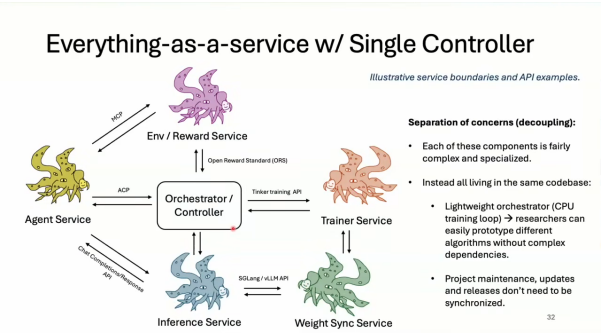
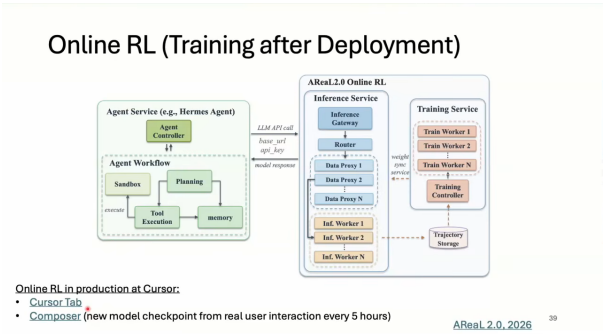


图 82: 左：单程序多数据（SPMD）。每个 GPU rank 运行同一个 Python 程序，按 rank 分支执行训练或推理；直接、易改，但不灵活（AReaL 1.0 之前采用）。右：一切皆服务 + 单一控制器。中心编排器通过标准协议调度智能体服务、环境 / 奖励服务、推理服务、权重同步服务与训练服务（Tinker training API）；编排器可以只是一个轻量的 CPU 循环。[88]

SPMD 的优点是一个程序包含全部逻辑、直接控制一切，“very easy to hack”；缺点是 “this ends up being very very inflexible compared to some alternatives”（00:56:32）。服务化架构的两个收益是解耦与模块化：每个组件都复杂且专门化，解耦后可以独立构建、维护与发布；只要协议一致就可以替换实现——这正好回答了「用哪个框架」：框架很多不要紧，只要讲同一套 API 就能互换。Tinker API 由 Thinking Machines 为其托管训练服务首先提出，现在 Miles 与 SkyRL 等开源框架也实现了它；推理引擎也可以在 vLLM 与 SGLang 之间替换。

服务化还带来几种新能力，讲者逐一举例：弹性扩容（中心控制器管理多个推理服务器，按需增加，甚至同时用 NVIDIA 与 AMD 的 GPU）；AstraFlow（CMU ECE 的 Haizhong Zheng 等）把 rollout 做成可在任意地区、任意硬件上随时加入或退出的服务；多策略训练（多智能体中不同智能体用不同权重甚至不同架构）；以及部署后的在线 RL：



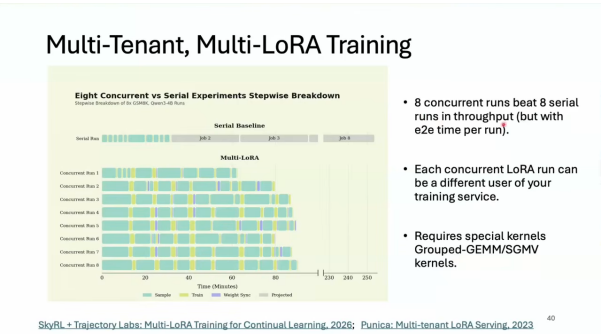


图 83: 左：部署后的在线 RL（AReaL 2.0 架构）。智能体服务调用推理网关，轨迹存储后送入训练服务，再同步回推理服务；Cursor 的 Composer 每 5 小时从真实用户交互中产出一个新 checkpoint。右：多租户多 LoRA 训练。8 个 GSM8K、Qwen3-4B 任务并发运行比串行运行吞吐高得多（图中并发约 62–91 分钟全部完成，串行投影约 250 分钟），代价是单个任务的端到端时间更长；需要 Grouped-GEMM / SGMV 一类专门 kernel。[89]

[87] 视频画面时间区间：第 12 讲 00:52:45–00:55:08（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。
[88] 视频画面时间区间：第 12 讲 00:55:08–00:59:28（均为幻灯片区域裁剪图）。
[89] 视频画面时间区间：第 12 讲 01:02:01–01:04:45（均为幻灯片区域裁剪图）。

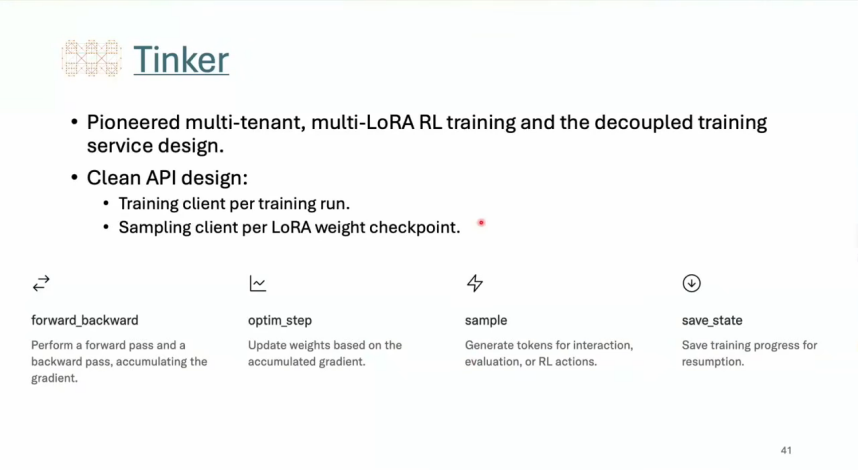


图 84: Tinker：开创了多租户多 LoRA 的 RL 训练与解耦的训练服务设计。API 很干净：每个训练 run 一个 training client，每个 LoRA 权重快照一个 sampling client；四个原语 `forward_backward`、`optim_step`、`sample`、`save_state`。[90]

讲者的收尾把系统设计与作业连在一起："you can write your code that targets the Tinker API and then there are open source implementations like the SkyRL implementation or Miles where you can bring your own compute and run the same training loops"（01:06:04）。这正是 Assignment 3 的部署方式：学生写 CPU 上的训练循环，后端是自己部署在 Modal[91] 上的 SkyRL Tinker 服务。讲义还有一页备用的监控指标表（视频中未展示），列出了每个指标的含义与告警信号：

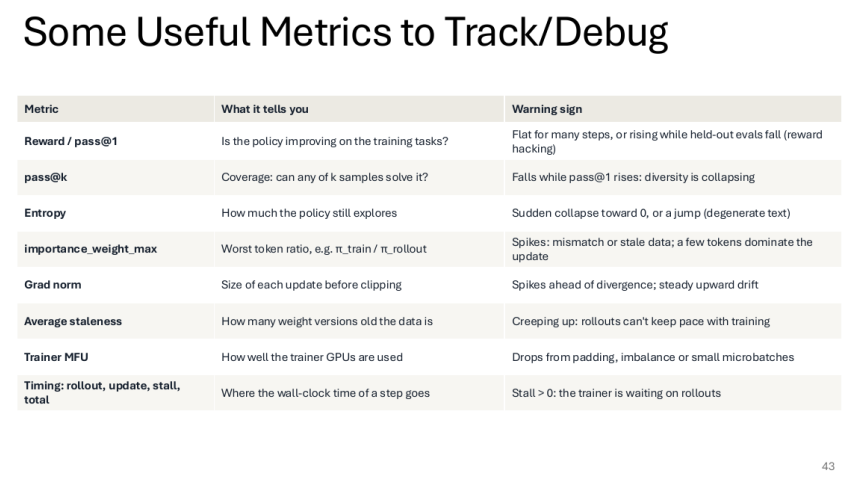


图 85: 讲义第 43 页（备用页，视频中未出现，此图由课件 PDF 渲染，不是视频画面）：奖励 / pass@1、pass@k、熵、`importance_weight_max`、梯度范数、平均过期度、训练 MFU、各阶段耗时及其告警信号。

[90] 视频画面时间区间：第 12 讲 01:04:45–01:06:06（幻灯片区域裁剪图，区间对应原视频中讲解该页的字幕段）。

[91] Modal：按秒计费的 Serverless GPU 云平台，用 Python 装饰器声明容器镜像、GPU 型号与 Web 端点；空闲时容器缩容到零，不计费。作业的三个 SkyRL 服务都部署在 Modal 的 H100 上（第 6 章）。

5.7　讲义概念与作业实现的对照

讲义概念	scottyrl / SkyRL 中的位置	说明
`sample`	`policy.TinkerPolicy.complete`：`sample_async(..., top_p=1.0, top_k=-1)`	返回 token 与 logprob
`forward_backward` + `optim_step`	`TinkerSession.submit_update`（学生实现）	两个请求先都提交再 `await`
权重同步	`new_sampling_client`：`save_weights_for_sampler_async` → `create_sampling_client`；`merge_lora: False`	fork 只导出 adapter，减少同步字节
引擎数值差异	`importance_weight_stats`；`sanity_warnings`	同步运行 ρ 应接近 1
KV cache / 前缀缓存	`gpu_memory_utilization` 0.7 / 0.85；`enable_prefix_caching`	超长请求被拒，episode 记 0 分
连续批处理	`max_num_seqs: 256`、`max_num_batched_tokens: 50000`	vLLM 调度
按 token 装箱 / 并行	`max_tokens_per_microbatch: 32768`；TP=PP=CP=1	只做数据并行
显存优化	`fused_lm_head_logprob`、`logprobs_chunk_size: 1024`	不存全词表 logits
前缀扩展	`datagen.extension_suffix` / `stitch_trajectory`；`XMLToolCallAgent._extend_context`	断开时开新序列
保留前缀的模板	`chat_templates/qwen3_5.jinja`；`preserve_thinking_history`	讲者在课上点名
TITO	`_context_tokens`；`LLMCall(prompt_tokens, completion_tokens, completion_logprobs)`	只对新尾部分词
同步共用 / 异步拆分	`colocated(2)`；`split(1,1)`；`split(1,3)`	TextCraft 为 3:1
异步与过期	`AsyncGRPOTrainer`；`max_staleness`；`queue_capacity`	消费时判断过期
服务化 / 单一控制器	CPU 训练循环 + Modal 上的 SkyRL Tinker 服务；`TINKER_BASE_URL`	可换托管 Tinker、Fireworks
未实现	投机解码、训练侧前缀共享、R3、harness 代理、多租户	单租户：每个 app 一次一个 run

5.8　本章小结

- **主线**：LLM RL 框架由两种计算特性相反的工作负载组成——推理（decode 为主、串行、无梯度、显存给 KV cache）与训练（类 prefill、并行、需要 autograd、显存给激活），因此拆成两个专用引擎各自优化。
- **拆分的代价**：引擎之间要传 rollout 数据与权重（差量同步、重新切分），两个引擎的数值不一致会让同步训练也变成轻微 off-policy（需要重要性修正、R3、TITO）。
- **智能体特有的坑**：保持前缀扩展（模板不能删历史思考、尽量不压缩上下文），以 token 而非字符串传输数据，异步 RL 与在途权重更新，子智能体带来的 batch 组织问题。
- **设计趋势**：从 SPMD 单程序走向「每个组件一个服务 + 轻量 CPU 控制器」；Tinker API 是这种解耦的事实标准，也是作业的接口。

视频未展开、但对复现有直接意义的几点：

1. **在线指标 ρ 同时检验三件事**。同步运行里 ρ 偏离 1 可能来自三处：两个引擎的数值差异（本讲）、TITO 被破坏导致 token 错位（本讲）、datagen 的差一位错误（第 6 章）。前一种通常只造成很小的偏离（均值接近 1、尾部略重），后两种会让均值明显偏离 1；作业的告警阈值（均值 $\notin [0.5, 2]$、最大值 > 10）区分的就是这两种量级。
2. **3:1 的来源可以估算**。异步部署中训练与采样 GPU 的最佳比例约等于「一步 rollout 的 GPU 时间」与「一步更新的 GPU 时间」之比。TextCraft 每步 128 条 episode、每条最多 75 回合、

每回合最多 512 token 的思考，生成量远大于一次 LoRA 前向反向，因此作业把 4 张卡里的 3 张给了采样；Number Search 每回合只生成一个 `guess` 调用，两侧接近平衡，用 1:1。

3. **服务化的代价**。Tinker 式接口让训练循环极简，但每一步都要经过网络调用与快照保存；作业日志里的 `train/update_time_s` 包含了这部分开销，分析同步与异步的加速比时应把它与 `rollout_time_s` 分开看。

5.9 拓展阅读

- Thinking Machines，Tinker 文档，`tinker-docs.thinkingmachines.ai`。
- SkyRL 的 Tinker 兼容服务，`docs.skyrl.ai/docs/tinker/overview`。

6 项目复现：Assignment 3 · scottyrl 训练智能体

前四章讲的是原理与工程常识；本章把它们落到一个可以从零跑通的具体项目上：课程的 Assignment 3「Training Agents with Reinforcement Learning」。第 9 讲开头 Daniel Fried 明确说，这两讲（RL Basics 与 Advanced RL）"should set you up pretty well for assignment three"，第 9 讲的贯穿示例 Number Search 就是作业里的第一个环境；第 12 讲的讲者 Apurva Gandhi 是作业框架 `scottyrl` 的第一作者。下面按「背景 → 环境与数据 → 模型 → 阶段划分 → 训练过程 → 结果与结论 → 复现清单」的顺序展开。本章的所有数字均来自仓库 `cmu-agents/assignment-3`（commit `ef42c0d`）的源码、配置与数据文件，数据统计由本笔记作者对 `data/` 目录重新计算。

6.1 项目背景：要做的是什么，为什么用 Tinker 接口

作业要求学生补全一个极简的智能体 RL 训练框架 `scottyrl`，并用它完成 7 次必需的训练运行：

- 搭建轨迹处理与优势（advantage）计算管线；
- 实现同步（synchronous）REINFORCE 与 GRPO；
- 实现带过期（stale）rollout 处理的异步（asynchronous）GRPO；
- 用受控消融对比算法与系统设计；
- 在 TextCraft-Synth 上训练并评测一个智能体。

框架的关键设计是面向 Tinker API（第 5 章）编写：训练循环、智能体 harness、环境、数据处理、奖励设计、优势计算都在本地 CPU 进程里，GPU 上的分布式执行与模型状态被封装进一个实现 `forward_backward`、`optim_step`、`sample` 等原语的服务。任何实现了 Tinker 兼容接口的后端都能接入：托管服务（Thinking Machines 官方 Tinker、Fireworks）或自托管服务（SkyRL、Miles）。作业指定自托管：用 SkyRL 的开源 Tinker 服务实现，跑在 Modal 的 H100 上。

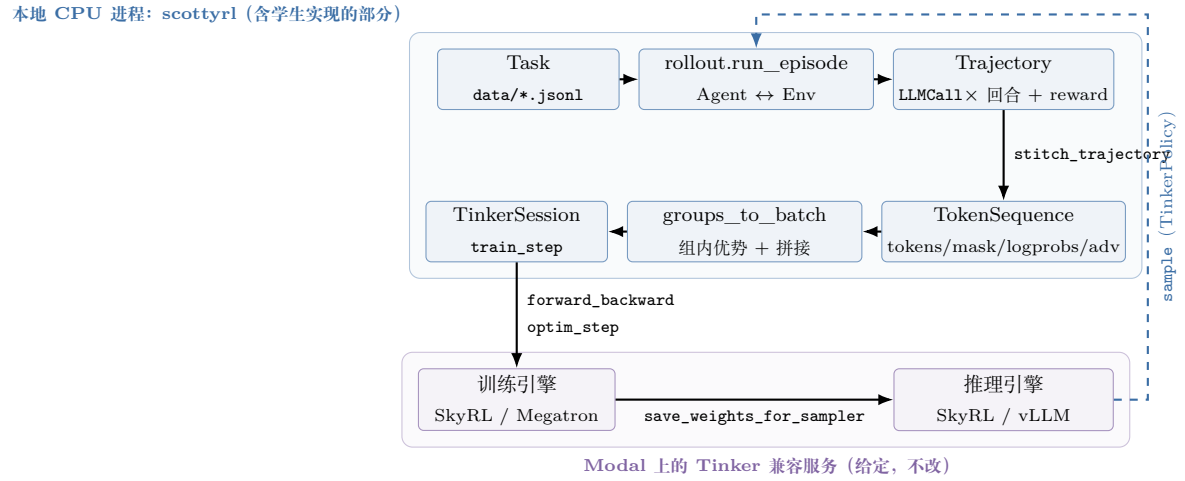


图 86: scottyrl 的数据流。左上到右上是一次 episode 的产生；右下回到左下是训练数据的构造与提交；虚线是每次更新后用新权重快照采样的回路。

这个项目要回答的三个问题

1. **算法**：同一个 Number Search 任务上，REINFORCE（$G = 1$）与 GRPO（$G = 2, 8$）学得多快？组越小，退化组越多，有效梯度信号越少。
2. **系统**：把 rollout 与更新重叠（异步）能快多少？过期数据要不要做重要性加权（importance weighting）？
3. **泛化**：用中等难度 TextCraft-Synth 训练一个 4B 模型，在同分布（medium）与更难（hard）验证集上分别提升多少？

6.2 环境、数据与任务

两个环境共享一套接口：智能体每回合输出一个工具调用 `<tool>name(args)</tool>`，环境返回一段文本观测；任务解决、调用 `finish()`（仅 TextCraft）或步数预算耗尽时 episode 结束；奖励是二值的，成功为 1，否则为 0。步数预算由环境自己维护（`Env.step` 每次调用计 1 步，包括解析失败的调用，达到 `task.max_steps` 即强制结束并在观测末尾追加 `(Out of steps.)`）。

6.2.1 Number Search：猜数字

隐藏整数 $n^\star \in [\text{low}, \text{high}]$，唯一工具 `guess(n)`，回复 `too low` / `too high` / `Correct!`。步数预算恰好是二分查找的最坏情况，不留余量：

$$B = \lceil \log_2(\text{high} - \text{low} + 2) \rceil$$

- B：允许的猜测次数（即 `max_steps`），包括最后那次猜中；
- $\text{high} - \text{low} + 1 = N$：候选个数；$\lceil \log_2(N + 1) \rceil$ 正是二分查找在 N 个候选上的最坏比较次数。

我对 `data/` 的复核确认：训练集 50,000 条、验证集 1,000 条，所有任务都严格满足上式；区间宽度 N 从 3 到 1024（中位数约 515），所以绝大多数任务的预算是 9 或 10 步。

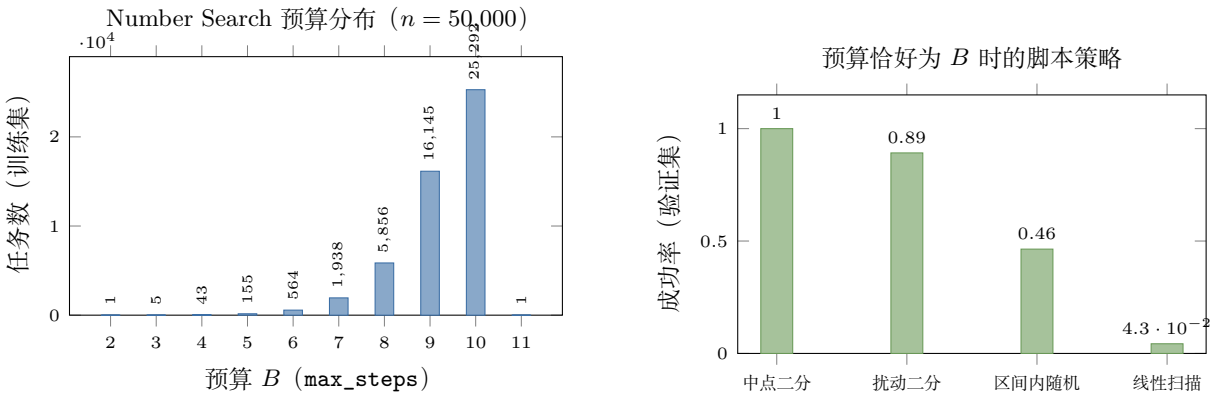


图 87: 左：训练集预算分布（数据复核）。右：四种脚本策略在 1,000 条验证任务上各模拟 20 次的成功率（本笔记作者的模拟，不是作业给出的数字）。「扰动二分」在区间中点上加 ±10% 区间宽度的均匀扰动；「区间内随机」在当前一致区间里均匀猜。

右图解释了这个环境为什么适合当 RL 的调试床：预算没有余量，只有严格二分才能 100% 成功；只要策略偏离中点（扰动二分）就会掉到约 89%，乱猜约 46%，线性扫描约 4%。基座模型的行为落在这条谱系的哪个位置，决定了 GRPO 组内奖励的分散程度，也就决定了有多少组能提供梯度（6.8 节）。

6.2.2 TextCraft-Synth：按配方树合成物品

环境改编自论文 Recursive Agent Optimization（RAO，第 5 章）。智能体从一份初始库存出发，要合成目标物品，有 4 个工具：

工具	语义与约束
`craft(ingredients: dict, target: tuple)`	`target=(item, total_count)`，`total_count` 必须能被配方的 `result_count` 整除；`ingredients` 必须**恰好**等于配方原料乘以合成次数——不能缺、不能多、数量不能错；原料从库存扣除
`get_info(items: list)`	一次查询多个物品：`can_craft`、`is_base`、`in_inventory`、`crafting_depth`、`recipes`
`view_inventory()`	返回当前库存字典
`finish(message: str)`	结束 episode；只有此时所有目标物品的**净增量**都达标才得 1 分

两条规则让这个环境比字面上难：(1) 原材料只来自初始库存，没有采集动作，合成错路径会把原料耗尽；(2) 不调用 `finish()` 就算物品已合成也是 0 分。配方书 `data/textcraft_recipes.json` 共 1,452 个物品，其中 50 个基础物品、1,402 个可合成物品（每个物品恰好一条配方），按 a/c/m/o/t 五个「材料族」各 278 个组织成 DAG。

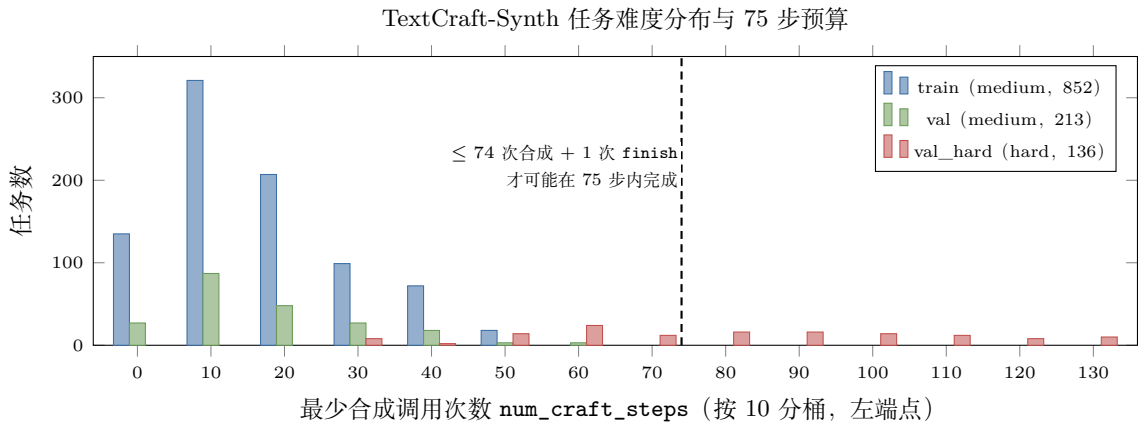


图 88: TextCraft-Synth 三个数据划分的 num_craft_steps 分布（本笔记作者从 data/ 统计）。我验证了 num_craft_steps 恰好等于目标物品依赖树里不同可合成物品的个数，即「每个中间物品一次性按总量合成」时所需的最少 craft 次数。

划分	任务数	难度	max_depth	最少合成次数 min/中位/max	75 步内理论可解
textcraft_train	852	medium	4/5/6 各约 1/3	4 / 18 / 59	100%
textcraft_val	213	medium	4/5/6	5 / 17 / 65	100%
textcraft_val_hard	136	hard	7/8/9	33 / 83.5 / 137	38.2%（52/136）

hard 验证集有一个硬上限：约 38%

每个可合成的中间物品至少需要一次 craft（一次调用可以按倍数合成任意数量，所以「至少一次」也是「恰好一次」可达成的下界），再加一次 finish。hard 集合里只有 52/136 = 38.2% 的任务满足 num_craft_steps + 1 ≤ 75；如果每层深度再花一次 get_info 查配方，只剩 44/136 = 32.4%。因此在 val_hard 上，即便是完美策略的 success_rate 也不会超过约 0.38。作业把 hard 集合定位为「测试泛化」，但报告结果时应把这个上限写出来，否则会把预算约束误读为能力不足。这一点作业文档没有说明，是本笔记对数据的推断。

6.3 模型与提示格式

所有运行统一使用 Qwen/Qwen3.5-4B + LoRA（第 2 章脚注）（rank 32）。Qwen3.5 是多模态模型，服务端配置 language_model_only=True 只加载文本骨干。与模型同样重要的是提示的渲染方式，它决定了训练序列的形态：

- **chat template**：scottyrl/chat_templates/qwen3_5.jinja，由 Qwen3.5 原版模板删去视觉与原生 tool-call 机制得到。环境观测以 user 角色回填；工具调用是纯文本 XML，由正则 <tool>\s*([A-Za-z_]\w*)\s*\((.*?)\)\s*</tool> 解析，参数用 ast.literal_eval 求值（只接受 Python 字面量，不执行代码）。
- **enable_thinking**：决定 assistant 回合的预填（prefill）。开启时生成提示以 <think>\n 结尾，模型先写推理再写 </think> 与答案；关闭时预填一个空思考块 <think>\n\n</think>\n\n，模型直接给答案。Number Search 关、TextCraft 开。
- **preserve_thinking_history**：两个环境都设为 true，历史 assistant 回合按采样原文逐字回放（含空思考块），保证每一轮的 prompt 都是上一轮 prompt + completion 的前缀扩展，整局

episode 因而是一**条**训练序列。设为 `false` 时历史思考被剥掉，上下文更短，但每一回合各自成为一条序列。

- **采样参数**：`temperature=1.0`、`top_p=1.0`、`top_k=-1`，即从完整分布采样，保证返回的 logprob 就是损失函数假设的行为策略概率；停止符 `<|im_end|>` 与 `<|endoftext|>`；每回合 `max_tokens`：Number Search 256，TextCraft 512（思考 + 带字典参数的 `craft` 调用需要空间）。

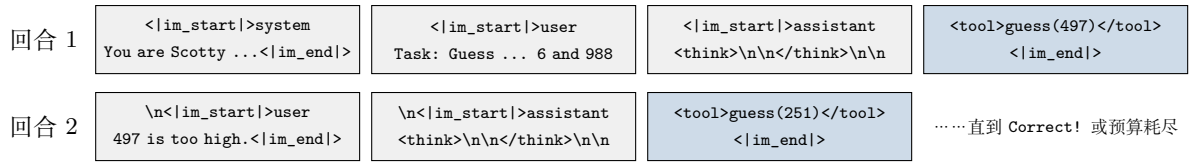
系统提示固定为：

```
You are Scotty, an agent that solves tasks by calling tools.

Rules:
- For tool calling, issue exactly one tool call inside <tool> ... </tool>, written with
  Python-style arguments, e.g. <tool>guess(512)</tool>.
- Output nothing after the tool call.

Tools available in this environment:
{tool_docs}
```

Listing 1: XMLToolCallAgent 的系统提示（`scottyrl/agents/xml_tool_call.py`）



灰：上下文（mask = 0，来自模板或环境，回合 2 的灰色部分即「新渲染的尾部」）；蓝：策略采样的 completion（mask = 1，带采样 logprob 与 episode 优势）。两行首尾相接就是同一条训练序列。

图 89: 一局 Number Search 在 token 空间的样子（示意，数字为虚构）。智能体保存的是**实际发给模型的 token id**，每回合只把新渲染出的尾部（结束标记、观测、下一回合的预填）编码后追加，从而保持「前缀扩展」性质。

为什么上下文要保存在 token 空间

如果每回合都把整段对话重新渲染成字符串再分词，BPE 分词器可能在旧文本与新文本的边界上合并字符，导致同一段前缀被切成不同的 token 序列，「上一轮 prompt + completion 是下一轮 prompt 的前缀」就不再成立，整局轨迹无法拼成一条训练序列，而且训练时重算的 logprob 会落在与采样时不同的 token 上。`XMLToolCallAgent._extend_context` 的做法是：重新渲染全文 `full_text`，检查它是否以「旧上下文文本 + 本轮 completion 文本」开头；是，则保留模型自己产生的 completion token，只对新尾部分词并追加；否，则整段重新编码（扩展被有意打断，datagen 会从这里开一条新序列）。这就是第 12 讲所说的 Token-In-Token-Out（TITO）在本框架里的落地。

6.4 框架结构与阶段划分

作业分 6 个阶段（Part 0–5），学生需要补全 6 个文件中的 `TODO(student)` 桩函数（签名不能改，隐藏测试会直接调用），其余文件由评分器恢复为原样：

阶段	文件	实现内容	分值
0	—	阅读 agent、rollout、env 的给定实现（热身）	0
1	`advantages.py`	`compute_group_advantages`	20
	`datagen.py`	`extension_suffix`、`stitch_trajectory`、`trajectory_to_training_sequences`	
2	`train/common.py`	`collect_group`、`groups_to_batch`	
	`tinker_adapter.py`	仅 `TinkerSession.submit_update`	25
	`train/sync.py`	`SyncTrainer`：`collect_batch`、`build_batch`、`update`、`evaluate_now`、`train`	
3	`train/async_grpo.py`	`AsyncGRPOTrainer`：`generate_group`、`rollout_worker`、`collect_fresh_batch`、`update`、`evaluate_now`、`train`	25
4	（无新代码）	三组消融：组大小、同步 vs 异步、高过期度下的重要性加权	20
5	（无新代码）	TextCraft-Synth 上的异步 GRPO「hero run」+ 评测 + 导出	10

起始代码上 `uv run pytest` 有 12/25 个公开测试通过，其余抛 `NotImplementedError`；全部实现后公开测试应全部通过（评分另有更严格的隐藏测试）。测试与整个数据管线不需要 GPU，也不需要后端服务。

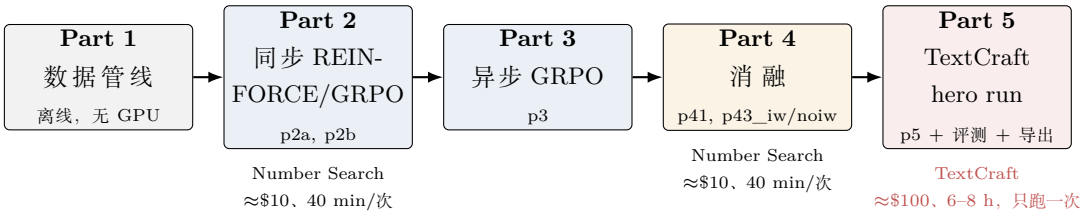


图 90: 作业的阶段依赖。便宜的 Number Search 承担全部调试与消融；昂贵的 TextCraft 只在异步训练器验证无误后跑一次。

6.5 第 1 阶段：从 episode 到训练序列

6.5.1 目标函数与掩码

作业文档把一局拼接后的 episode 写成 token 序列 $t_0, \ldots, t_{n-1}$，用掩码 $m_j \in \{0,1\}$ 标记目标 token t_{j+1} 是否由策略采样（而不是来自 prompt 或环境）。在整局共享一个优势 A 时，策略梯度为：

$$\nabla_\theta J(\theta) = \mathbb{E}\Big[A \sum_{j:\, m_j=1} \nabla_\theta \log \pi_\theta(t_{j+1} \mid t_{\le j})\Big]$$

- π_θ：当前策略（LoRA 适配后的 Qwen3.5-4B）；
- $t_{\le j}$：位置 j 及之前的全部 token（系统提示、任务、历史动作与观测）；
- $m_j = 1$：t_{j+1} 是 completion token；观测和模板 token 的 $m_j = 0$；
- A：episode 级优势，整局所有动作 token 共享同一个值。

这就是第 9 讲所说「策略梯度 = 对 SFT 损失做掩码与重新缩放」的具体形式。若掩码把观测 token 也算进来，模型学到的是预测环境，而不是如何行动。

6.5.2 拼接算法与差一位对齐

TokenSequence 的四个字段中，tokens 长度为 n，另外三个（mask、logprobs、advantages）长度都是 $n-1$，位置 j 描述的是目标 tokens[j+1]。若一段长度为 c 的 completion 紧跟在长度为 L 的上下文之后，它在这三个数组里占的位置是 L-1 : L-1+c，而不是 L : L+c。下图用作业 docstring 里的例子把对齐画出来：

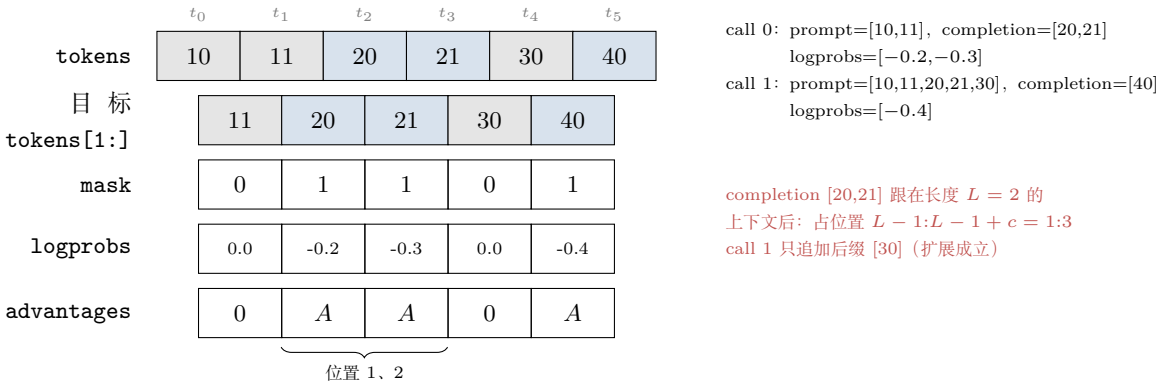


图 91: stitch_trajectory 的输出对齐（数值取自 scottyrl/datagen.py 的 docstring）。蓝色为策略采样的 token，灰色为上下文。trajectory_to_training_sequences 再把 episode 优势 A 广播到所有 mask=1 的位置。

拼接的规则（docstring 规定的契约）可以写成下面的伪代码。它刻意只给出逻辑骨架，具体实现是作业内容：

```
push(tok, m, lp):                  # 向当前序列追加一个 token
    若 cur.tokens 非空: cur.mask += [m]; cur.logprobs += [lp]   # 首个 token 没有预测目标
    cur.tokens += [tok]

cur = 空序列
for call in traj.calls:
    suffix = extension_suffix(cur.tokens, call.prompt_tokens)   # 前缀不成立 -> None
    if suffix is None:               # 扩展被打断：结束当前序列，从完整 prompt 重新开始
        emit(cur); cur = 空序列; suffix = call.prompt_tokens
    for tok in suffix:                      push(tok, 0, 0.0)    # 模板/观测：上下文
    for tok, lp in zip(call.completion_tokens, call.completion_logprobs):
                                            push(tok, 1, lp)     # 策略采样：动作
emit(cur)            # emit 丢弃 len(tokens) < 2 或没有 mask=1 位置的序列；advantages 先全置 0
```

Listing 2: 拼接契约的伪代码（依据 datagen.py 的 docstring 整理）

两个容易写错的边界：其一，extension_suffix([], prompt) 返回整个 prompt（空序列是任何序列的前缀），且 extension_suffix([1,2,3],[1,2]) 返回 None（累计序列比新 prompt 长，不算前缀）；其二，序列第一个 token 没有对应的预测目标，所以开新序列时第一个 token 只进 tokens、不进三个并行数组。

扩展会在哪些情况下断开

- preserve_thinking_history=false 且开启思考：历史思考被模板剥掉，每回合都断开，一局变成「回合数」条序列；
- 模型以 <|endoftext|> 而不是 <|im_end|> 停止：模板回放历史时写回的是 <|im_end|>，

文本对不上，这一回合起开新序列；

- 某次采样调用失败（例如超出服务端 `max_model_len`）：`run_episode` 放弃该局、记 0 分，并删掉失败那次调用的记录，保证 `calls` 里只有完整回合。

数据管线必须同时处理「一局一条序列」和「一局多条序列」两种情况；公开测试里的 `ThinkStrippingPolicy` 专门构造了断开的 episode。

6.5.3 组内优势

`compute_group_advantages(rewards, method)` 的契约：

$$A_i^{\text{REINFORCE}} = r_i, \qquad A_i^{\text{GRPO}} = r_i - \frac{1}{G}\sum_{k=1}^{G} r_k$$

- r_i：同一任务第 i 次 rollout 的 episode 奖励（0 或 1）；
- G：组大小（`group_size`），GRPO 要求 $G \geq 2$，否则抛 `ValueError`；
- 组内奖励**完全相同**（无容差）时该组为退化组（degenerate），返回 `None`，调用方整组跳过；
- 遵循 Dr.GRPO：不除以组内奖励标准差，也不加对参考策略的 KL 惩罚（第 9 讲 Dr.GRPO 一节解释了原因）。

6.6 第 2 阶段：同步训练循环

6.6.1 一个训练步做了什么

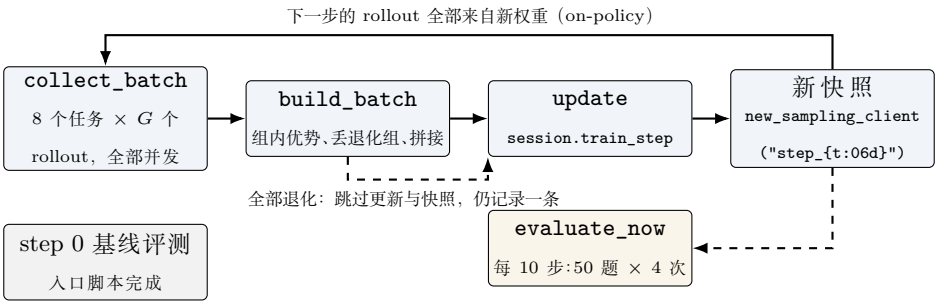


图 92: `SyncTrainer.train` 的一步。快照名用迭代序号，跳过的步不产生快照，所以快照名可以不连续。

要点（均来自 docstring 的契约）：

1. **采样**：用 `self.rng`（种子 11768）从训练集抽 `tasks_per_batch` 个不同任务；每个任务一组，组内每次 rollout 用全新的 env 和 agent 实例；不同任务的组之间、组内各 rollout 之间都用 `asyncio.gather` 并发，使 LLM 调用在时间上重叠。
2. **组批**：退化组不贡献序列但计数；`mean_reward` 按所有轨迹（含退化组）计算；非退化组里优势恰好为 0 的轨迹也照样输出序列。

3. **更新**：`session.train_step(seqs, loss_fn, lr, grad_clip_norm)`；更新后立刻刷新采样策略，保证下一批是 on-policy。
4. **评测**：每到 `eval_every` 的整数倍（按绝对步数，恢复的运行也保持节奏），用当前策略评测前 `eval_tasks` 道验证题、每题 `eval_k` 次。
5. **记录**：每步恰好调用一次给定的 `record_training`（跳过的步传空列表与空字典）。注意同一行记录里 rollout 类指标来自更新前的策略，`eval/*` 来自更新后的策略，两者相差一次更新。

6.6.2 三种 token 级损失

`train.algo` 决定优势规则，`train.loss_fn` 独立决定 token 级损失。对采样自行为策略 π_{old}、在 π_θ 下训练的 token，重要性比率为：

$$\rho_j = \exp\left(\log \pi_\theta(t_{j+1} \mid t_{\le j}) - \log \pi_{\text{old}}(t_{j+1} \mid t_{\le j})\right)$$

- π_{old}：生成这条数据时的采样策略，其 logprob 就是 `TokenSequence.logprobs`；
- $\log \pi_\theta$：训练引擎前向时重算的 logprob（由 `fwd_result.loss_fn_outputs` 返回，用于统计 ρ）。

三种损失（对 $j : m_j = 1$ 求和）：

$$\mathcal{L}_{\text{ppo}} = -\sum_j \min\left(\rho_j A_j,\ \text{clip}(\rho_j, 1-\epsilon, 1+\epsilon) A_j\right), \quad \mathcal{L}_{\text{is}} = -\sum_j \rho_j A_j, \quad \mathcal{L}_{\text{ce}} = -\sum_j A_j \log \pi_\theta(t_{j+1} \mid t_{\le j})$$

- `ppo`：所有 GRPO 运行使用；Tinker 文档中 $\epsilon_{\text{low}} = \epsilon_{\text{high}} = 0.2$（比率被夹在 $[0.8, 1.2]$），scottyrl 不传 `loss_fn_config`，即用默认值；
- `importance_sampling`：不裁剪的比率加权，REINFORCE 运行（p2a）使用；
- `cross_entropy`：优势加权的对数似然，完全没有比率，只用于 Part 4.3 的「无修正」消融（p43_noiw）。

在同步运行里 $\pi_\theta = \pi_{\text{old}}$，三者的梯度相同（$\nabla\rho = \rho\nabla\log\pi = \nabla\log\pi$ 当 $\rho = 1$）；差别只在异步、数据过期时显现。

按 batch 的 token 总数归一化（Dr.GRPO 的常数归一化）

Tinker 的损失对一个 batch 内所有数据的所有动作 token **求和**。`TinkerSession.train_step`（给定代码）先统计 batch 的动作 token 总数 T，再把每个优势乘以 $1/T$ 后才放进 `tinker.Datum`，于是目标变成「batch 内每个动作 token 的平均损失」。两点后果：(1) 长 episode 与短 episode 的 token 在同一 batch 内权重相同，不会像「先在序列内取均值」那样压低长 episode 的每个 token；(2) 比率类损失对优势是线性的，缩放只改变梯度尺度，不改变 ρ 与裁剪阈值的含义。日志里的优势统计仍是未缩放的值。

6.6.3 唯一一个后端相关的学生函数：submit_update

`submit_update` 只做两次 SDK 调用：前向反向 `training_client.forward_backward_async(datums, loss_fn=loss_fn)`，以及优化器步 `training_client.optim_step_async(params)`，其中 `params` 是学习率为 `lr`、梯度裁剪为 `grad_clip_norm` 的 `tinker.AdamParams`。契约要求**先把两个请求都提交，再分别 `await future.result_async()`**：Tinker 会把背靠背到达的请求放进同一个服务端周期；先等前向反向的结果再提交优化器步，会把两次操作串行化，每步延迟约翻倍。SDK 的异常（`RequestFailedError`、`APIStatusError`、`TimeoutError`）不能吞掉。优化器的其它超参数在服务端固定：Adam $\beta = (0.9, 0.95)$、weight decay 0、`max_grad_norm`=1.0。

6.7 第 3 阶段：异步 GRPO 与过期数据

6.7.1 生产者—消费者结构

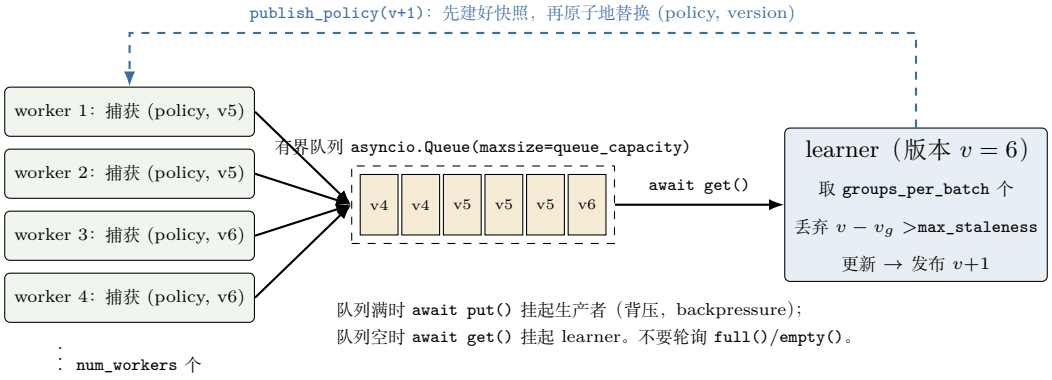


图 93: `AsyncGRPOTrainer` 的结构。每个组在开始前一次性捕获 (policy, version) 并随结果一起入队；过期判断在 learner 消费时做，而不是在 worker 端。

```
publish_policy(0)
task_ids = 循环迭代训练任务 id（所有 worker 共享）
workers = [create_task(rollout_worker(queue, task_ids)) for _ in range(num_workers)]
for step in 1..num_steps:
    batch = collect_fresh_batch(queue, stats)      # 恰好 groups_per_batch 个新鲜组（可含退化组）
    if batch 中有序列:
        metrics = update(batch); publish_policy(version + 1)
    else:
        metrics = {}                               # 跳过：不更新、不发布，版本不前进
    record_training(step, batch, metrics)          # 每步恰好一次
    if eval_due(step):
        policy, v = active_policy(); 后台调度 run_evaluation(policy, v, step)
结束：取消并 await 所有 worker；await 所有评测任务
任何异常：取消并 await 评测任务与 worker，然后原样重新抛出原异常对象
返回 {"train_steps", "groups_consumed", "groups_dropped_stale"}
```

Listing 3: 异步训练器的控制流（依据 `async_grpo.py` 的 docstring 整理的伪代码）

6.7.2 过期度能有多大：一个容易验证的上界

`p5_textcraft_grpo.yaml` 的注释给了一条经验关系：记 worker 数 W（`num_workers`）、队列容量 Q（`queue_capacity`）、每步消费的组数 B_g（`groups_per_batch`），则 $(W + Q)/B_g$ 近似等于

「learner 较慢时，一个组被训练时已经落后的版本数」。直觉是：一个组在某个版本被捕获后，排在它前面的最多有队列里的 `queue_capacity` 个组和其他 worker 手里的组；learner 每步消费 `groups_per_batch` 个组并把版本加 1。把它代入各配置：

运行	worker 数 W	队列容量 Q	每步组数 B_g	估计落后 $(W+Q)/B_g$	`max_staleness`
p3_number_search_async	8	16	8	$24/8 = 3$	2
p43_*_stale4_iw / noiw	8	16	8	$24/8 = 3$	4
p5_textcraft_grpo	32	16	16	$48/16 = 3$	4

从这张表能预期什么（本笔记的推断，需以实际日志为准）

p3 的估计落后量 3 超过阈值 2：只要 learner 成为瓶颈、队列被填满，最老的那批组就会被丢弃，`summary/groups_dropped_stale` 应大于 0，Part 4.2 要求引用的正是这个计数。p43 与 p5 的阈值 4 高于估计值 3，几乎不会丢组，于是 p43 的两次运行都会在「落后 1–3 个版本」的数据上训练，唯一差别是损失是否使用 ρ——这正是作业要求的受控对比。反过来，如果 rollout 才是瓶颈（队列常空），过期度会接近 0–1，p3 也可能几乎不丢组；Number Search 的 episode 很短，哪一侧是瓶颈要看 `train/learner_wait_frac`。

6.7.3 异步实现的几条硬性要求

- 一个组只用一个在开始时捕获的 `(policy, version)`；不能在 rollout 的 `await` 之后再读「当前」策略，否则组里携带的版本号会错。
- 评测作为后台任务在捕获的 `(policy, version)` 上运行，与训练重叠；评测失败要像 worker 失败一样立刻让整个运行失败，而不是等最后一步。
- 关停时取消并等待 worker（之后 worker 停止不算错误），再等待所有评测任务；任何异常都要先取消、等待其余任务，再把原始异常对象抛出（不能包成 `ExceptionGroup` 或新的 `RuntimeError`）。
- 记录里的 `policy_version` 与迭代序号在第一次跳过之后会分叉：迭代计数包含跳过的步，版本只计实际更新。

6.8 第 4 阶段：三组消融

运行	训练器	algo	G	组/步	loss	staleness	相对 p2b 的差别
p2a_number_search_reinforce	sync	reinforce	1	8	importance_sampling	—	无组基线；不裁剪
p2b_number_search_grpo	sync	grpo	8	8	ppo	—	参考配置
p41_number_search_grpo_g2	sync	grpo	2	8	ppo	—	组更小
p3_number_search_async	async	grpo	8	8	ppo	2	8 worker，队列 16
p43_.._stale4_iw	async	grpo	8	8	ppo	4	过期上限更高
p43_.._stale4_noiw	async	grpo	8	8	cross_entropy	4	去掉 ρ
p5_textcraft_grpo	async	grpo	8	16	ppo	4	开思考，512 tok/回合，32 worker

所有运行共享：`lr`=1e-5、`grad_clip_norm`=1.0、50 步、每 10 步评测（50 题 × 4 次）、种子 11768、每 10 个优化器步存一次可恢复检查点、每 5 步写一次完整轨迹记录。

4.1 组大小。 对成功概率为 p 的单个任务，G 次独立 rollout 奖励全相同的概率是

$$P_{\text{deg}}(p, G) = p^G + (1-p)^G$$

- p：当前策略在该任务上的单次成功率；
- G：组大小；P_{deg} 即该组被判为退化、整组不产生梯度的概率。

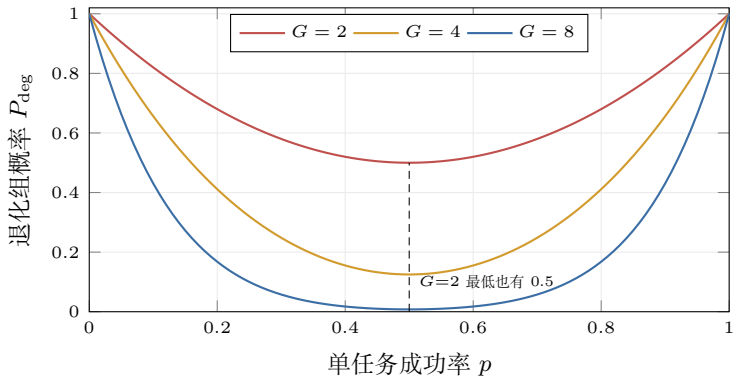


图 94: 退化组概率随单任务成功率的变化（解析计算）。$G = 2$ 时即便 $p = 0.5$ 也有一半的组没有梯度；$G = 8$ 在 $p \in [0.2, 0.8]$ 内几乎所有组都有信号。

由此可以预先写出 Part 4.1 应观察到的现象：p41（$G = 2$）的 `train/frac_degenerate_groups` 显著高于 p2b，每步有效序列数更少，组基线（两个样本的均值）方差更大；p2a 的退化比例应记为 N/A，因为 REINFORCE 没有组基线，日志里的 0 没有意义。作业也提醒 p2a 同时换了优势规则和损失，所以这不是纯粹的组大小消融。

4.2 同步 vs 异步。 对比 p2b 与 p3 的 `train/step_time_s`（或相邻训练记录 `time` 字段之差）给出加速比；用重要性权重统计（均值、最小、最大、标准差——均值接近 1 可能掩盖重尾）与 `summary/groups_dropped_stale` 说明过期数据是否损害了样本质量。同步运行的部署是 2 张 H100 共用（训练与采样交替），异步是 1 训练 + 1 采样，硬件总量相同，所以加速比反映的是「重叠」本身的收益。

4.3 高过期度下的重要性加权。 两次运行只差损失：`ppo`（裁剪后的 ρ）对 `cross_entropy`（无 ρ）。第 11 讲的论证给出预期方向：无修正时，旧策略下概率高、当前策略下已变低的 token 仍按原优势被推动，更新方向与当前策略的真实梯度存在偏差；PPO 的裁剪又同时限制了单个 token 的权重。实际差异大小取决于过期度分布与学习率，Number Search 上 50 步、`lr`=1e-5 时差异可能不大——这是需要用日志回答的问题，而不是可以预先断言的结论。

6.9 第 5 阶段：TextCraft-Synth hero run

配置 `p5_textcraft_grpo.yaml`：开启思考、每回合最多 512 token、每步 16 组 × 8 次 rollout、32 个 rollout worker（保持 256 个 episode 同时在途以喂满 3 张采样 GPU）、staleness 上限 4、50 步、每 10 步评测。部署 `serve_textcraft.py`：4 张 H100，1 张训练、3 张采样，vLLM 上下文上限 65,000 token。训练结束后用最终快照（`runs/p5_textcraft_grpo.checkpoints.json` 里的 `final_sampler_path`）和基座模型分别在 medium 与 hard 验证集上评测（每题 4 次），报告一张四行表（微调/基座 × medium/hard 的 `success_rate` 与 `pass@4`），并把 LoRA adapter 导出到公开的 Hugging Face 仓库 `<hf-username>/scottyrl-a3-p5_textcraft_grpo`。

6.10 训练后端：SkyRL Tinker 服务部署在 Modal

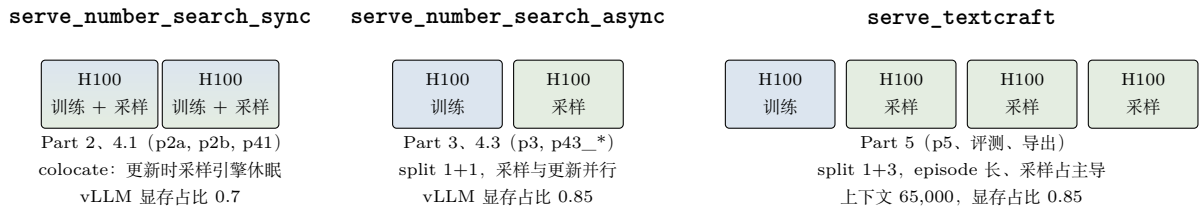


图 95: 三种部署的 GPU 布局(依据 `skyrl_tinker_modal/README.md`)。三者都用 SkyRL 的 Megatron 后端训练（SkyRL 在 Qwen3.5 上支持的后端）、vLLM 采样，TP=PP=CP=1，只做数据并行。

服务端设置（skyrl_server.py）	取值与作用
SkyRL 版本	课程 fork adityasoni9998/SkyRL @ 491e132：含 Megatron token-batched 输出顺序修复与「只导出 adapter 的采样快照」
max_tokens_per_microbatch	32768：按 token 数动态装箱整条序列（软上限，超长序列单独成一个 micro-batch）
fused_lm_head_logprob	True：在输出层内按块计算 logprob，不保存全词表 logits（否则反向会 OOM），块大小 1024
max_num_batched_tokens / max_num_seqs	50,000 / 256（每个 vLLM 引擎）
enable_prefix_caching	True：组内 rollout 共享任务 prompt、回合间共享历史，前缀 KV 复用
max_model_len	Number Search 32,768；TextCraft 65,000。超限请求被拒，该 episode 记 0 分，运行继续
CUDA graph	FULL_DECODE_ONLY；Qwen3.5 Gated-DeltaNet 的 prefill 用 triton 内核
容器	8 CPU、32 GiB 内存、12 小时超时、空闲 5 分钟缩容到零

成本：按约 $4/H100 · 小时，Number Search 服务约 $8/小时，TextCraft 服务约 $16/小时；首次构建镜像（下载 SkyRL、Megatron 与 CUDA 依赖，数 GB）约 20 分钟以上，占用一张 H100。

6.11 运行、监控与诊断

```
uv python install 3.12 && uv sync && uv run pytest          # 本地：测试与数据管线不需要 GPU
uv sync --extra modal && uv run modal setup                  # 一次性：Modal 账户与 CLI
cd skyrl_tinker_modal && uv run modal deploy serve_number_search_sync.py
export TINKER_BASE_URL=https://<workspace>--scottyrl-ns-sync-tinker-server.modal.run
# 先做短探针，确认管线连通（10 步、每 2 步评测、20 道题）
uv run python scripts/train_sync.py --config configs/p2b_number_search_grpo.yaml \
    --set train.num_steps=10 --set train.eval_every=2 --set train.eval_tasks=20 \
    --set train.run_name=probe_p2b
uv run python scripts/train_sync.py  --config configs/p2a_number_search_reinforce.yaml
uv run python scripts/train_sync.py  --config configs/p2b_number_search_grpo.yaml
uv run python scripts/train_sync.py  --config configs/p41_number_search_grpo_g2.yaml
# 换异步部署（1 训练 + 1 采样）
uv run python scripts/train_async.py --config configs/p3_number_search_async.yaml
uv run python scripts/train_async.py --config configs/p43_number_search_async_stale4_iw.yaml
uv run python scripts/train_async.py --config configs/p43_number_search_async_stale4_noiw.yaml
# 换 TextCraft 部署（1 训练 + 3 采样）
uv run python scripts/train_async.py --config configs/p5_textcraft_grpo.yaml
uv run python scripts/evaluate_checkpoint.py --config configs/p5_textcraft_grpo.yaml --model-path <
    final_sampler_path> --k 4
uv run python scripts/evaluate_checkpoint.py --config configs/p5_textcraft_grpo.yaml --model-path <
    final_sampler_path> --k 4 --tasks data/textcraft_val_hard.jsonl
uv run python scripts/evaluate_checkpoint.py --config configs/p5_textcraft_grpo.yaml --k 4   # 基座模型
uv run hf auth login && uv run python scripts/export_checkpoint.py --run p5_textcraft_grpo
uv run python scripts/package_submission.py
uv run modal app stop scottyrl-ns-sync                       # 用完立即停服务
```

Listing 4: 从零到提交的命令序列（摘自 README 与 ASSIGNMENT）

探针跑完要检查四件事：(1) 日志有 step 0 基线评测、每步一条训练记录、每两步一条评测；(2) step_000005/ 下的轨迹读起来像真实的 episode（completion、解析出的工具调用、环境回复；null

的工具调用表示解析失败)；(3) 同步运行的 `train/importance_weight_mean` 接近 1；(4) 终端里每条 `[sanity]` 都能解释。

指标（`train/` 或 `eval/`）	含义	`[sanity]` 告警阈值
`success_rate`、`pass@k`	成功率；训练时 $k = G$（如 `pass@8`），评测时 $k = 4$，两者不可画成一条曲线	—
`grad_norm`	裁剪前梯度范数（SkyRL 键名 `skyrl.ai/grad_norm`）	> 50
`importance_weight_{mean,min,max,std}`	训练重算 logprob 与采样 logprob 之比的统计	mean $\notin [0.5, 2]$；max > 10 或 std > 1
`frac_degenerate_groups`	退化组比例	> 0.8
`entropy_proxy`	动作 token 的 $\mathbb{E}[-\log \pi]$（含格式 token，只是代理量）	< 0.005
`step_time_s`、`update_time_s`、`learner_wait_s/frac`、`rollout_time_s`	一步的墙钟时间分解	—

同步运行里 ρ 偏离 1 意味着什么

同步训练的每个 batch 都来自刚发布的权重，训练前向重算的 logprob 与采样 logprob 应几乎一致，`importance_weight_mean` 应非常接近 1。作业文档明确指出：若不是，最常见的原因是采样 logprob 被挂到了错误的 token 上——也就是 91 图里那一位的错位。这是检验 Part 1 实现最便宜的线上信号。

6.12 结果与结论：能期望看到什么

作业没有公布参考曲线或参考分数（它们是学生报告的内容）；本笔记也没有实际租用 GPU 运行。可以确定的「结果」是作业给出的验收标准和由数据推出的约束：

- **验收标准**：Part 2 中「正常工作的运行，奖励应在 50 步内明显高于 step 0 基线；曲线平的话先调试实现再写分析」。
- **约束一**：Number Search 预算没有余量，成功意味着近似严格的二分；基座模型越接近「区间内随机」（约 0.46），$G = 8$ 的组几乎都有信号；越接近完美，退化组越多，`[sanity] frac_degenerate_groups` 告警会在后期出现，这是任务被学会的信号而不是故障。
- **约束二**：TextCraft hard 集在 75 步预算下的成功率上限约 0.38（88 图）。
- **约束三**：异步配置下估计最大落后 3 个版本，p3 的阈值 2 会导致丢组，p43/p5 的阈值 4 基本不丢组。

作业要求的定性分析（第 3 部分）也值得照做：分别取训练早、中、晚期各一条 rollout（给出步数、任务 id 与文件头里的 `policy_version`），描述至少两种行为变化，例如是否利用了高低反馈、是二分还是扫描、如何从非法调用中恢复。

6.13 复现清单

1. **环境**：`uv` + Python 3.12；`tinker==0.24.0`（SkyRL 服务端只支持 0.24 的线上格式，0.25+ 期望 protobuf 的 SampleResponse）。

2. **离线**：实现 Part 1，跑 `pytest -k "advantages or datagen"`；实现 Part 2，跑 `-k "not async"`；实现 Part 3，跑全部测试。
3. **部署**：`modal deploy` 对应的 `serve_*.py`，复制打印出的 URL 到 `TINKER_BASE_URL`；首次请求要等数分钟拉起容器与采样引擎。
4. **探针**：每个新部署先跑 10 步短探针，按上面的四项检查。
5. **正式运行**：七个必需运行保持给定 `run_name`；崩溃后用 `--set train.resume=true` 从最近的可恢复检查点继续（每 10 个优化器步一个；恢复后快照名带 `_r1` 后缀）。同一种部署要并行两个运行时，用 `SKYRL_APP_NAME` 部署第二个 app。
6. **监控**：可选 `--set train.wandb_project=<project>`；评分只看 `runs/<run_name>.jsonl`；每 5 步读一次轨迹。
7. **收尾**：评测、导出 adapter、`package_submission.py`，然后 `modal app stop`。

6.14 本章小结

- **项目的主线**是把第 9、11、12 讲的三件事做成代码：episode 级优势广播到动作 token（第 9 讲的掩码与缩放）、组内基线与 PPO 裁剪（第 9、11 讲）、producer–consumer 异步与过期修正（第 11、12 讲）。
- **数据与任务**：Number Search 5 万条训练任务、预算恰为二分最坏情况；TextCraft-Synth 852 条中等难度训练任务、配方 DAG 1,452 个物品、75 步预算、调用 `finish()` 才得分。
- **模型**：Qwen3.5-4B + LoRA r32，Number Search 不开思考，TextCraft 开思考；保留思考历史以维持前缀扩展。
- **阶段**：数据管线 → 同步 → 异步 → 消融 → hero run，便宜环境承担调试，昂贵环境只跑一次。

视频未展开，但从代码里可以读出几条通用的工程原则：(1) 训练数据必须在 token 空间里与采样严格对齐，$\rho \approx 1$ 是检验这一点的免费在线指标；(2) 按 batch token 总数归一化把「每个动作 token 等权」写进了目标函数，避免了按序列平均对长 episode 的系统性低估；(3) 把 GPU 状态封装成服务后，学生代码只需关心「数据怎么来、权重怎么算」，同一套代码可在托管 Tinker、SkyRL、Fireworks 之间切换——这正是第 12 讲「everything-as-a-service」的实际收益。仍待实验回答的问题是：Number Search 上 50 步的规模能否让 IW 与否的差异显现，以及 TextCraft 训练在 hard 集上能吃掉 0.38 上限中的多少。

7 总结与延伸

7.1 讲者们的收尾

四讲的结尾各自给出了带走的要点，放在一起读正好是一条完整的论证：

- **第 8 讲（Yueqi Song）**：SFT 是获得通用智能体能力最多的阶段，但它从根本上受教师能力限制（“fundamentally limited by the teacher model’s capability”）；它的价值最终由「是不是好的 RL 起点」衡量——“RL provides the most gains when the SFT model is selected with RL in mind”。

- **第 9 讲（Daniel Fried）**：在环境中采样、按奖励加权，同时化解了任务失配、数据失配与暴露偏差——“the model's essentially driving at training time. So it should be learning to recover from its own errors”；代价是必须实际尝试，开销大且梯度有噪声。
- **第 11 讲（Graham Neubig）**：三条主线——有用的反馈（组比较、critic 或过程奖励做信用分配）、稳定的更新（知道行为策略是谁，控制单个样本对模型的影响）、可靠的奖励（匹配任务难度，验证你真正想要的行为）。结尾问答里他还留了一个开放问题：要不要在观测 token 上也训练。
- **第 12 讲（Apurva Gandhi）**：把训练循环写给 Tinker API，就可以在托管服务与自托管的 SkyRL / Miles 之间无缝切换，“bring your own compute and run the same training loops”。

7.2 一张表看懂「给动作加权」

第 9 讲的统一形式 $\nabla_\theta J = \sum_t w_t \nabla_\theta \log \pi_\theta(a_t \mid h_t)$ 可以把整个模块里出现的方法放进同一张表。各列回答四个问题：动作从哪来、权重是什么、要不要处理「采样策略 $\neq$ 当前策略」、额外需要什么。

方法	动作来源	每个动作 token 的权重 w_t	比率修正	额外需要
SFT（第 8 讲）	示范（人、更强模型）	1（掩码内）	无	好的轨迹
ReST / 专家迭代	自己采样，只留成功的	成功为 1，失败丢弃	无（批量离线）	可校验的奖励
REINFORCE	自己采样	$R(\tau)$ 或 reward-to-go R_t	无（新鲜 rollout）	奖励
GRPO	同一任务采 G 次	$(R_j - \bar{R})/\sigma_R$	当前 token ρ_t，PPO 裁剪	组采样
Dr.GRPO（作业）	同上	$(R_j - \bar{R})/C$	同上	组采样
PPO	自己采样	GAE 优势 $\hat{A}_t$（来自 critic）	当前 token，裁剪	价值模型
CISPO / GSPO / DAPO	同一任务采 G 次	组优势	裁剪为固定权重 / 序列级长度归一化比率 / 非对称裁剪	组采样
On-policy 蒸馏	学生自己采样历史	每个 token 一个分布目标（反向 KL）	无（学生即行为策略）	教师模型

这张表揭示了一条从左到右的演进：数据来源从「别人」变成「自己」，权重从「常数」变成「相对表现」再变成「稠密的分布目标」，而一旦允许数据来自旧策略（异步），就必须补上比率修正。实现层面这几种方法共享同一段代码——对掩码选中的 token 求对数似然再乘以权重——所以作业里 `algo` 与 `loss_fn` 是两个独立的开关。

7.3 各讲引用案例速查

用户关心的「背景、数据与任务、模型、阶段、训练过程、结果」在四讲引用的案例里都有具体数字。下表只收录讲义或讲者明确给出的信息。

案例	数据与任务	模型	阶段 / 配置	结果与结论
SWE-Gym（第 8 讲）	SWE-Gym 仓库任务；GPT-4o 与 Claude 的 run 中通过测试的 491 条	Qwen 7B / 14B / 32B	单阶段 SFT（只用 491 条）	32B 在 SWE-Bench Lite 3.0 → 15.3、Verified 7.0 → 20.6；轨迹数从 25% 到 100% 时 32B 仍在上升
OpenThoughts-Agent（第 8 讲）	95 种任务来源中选 4–8 种；教师 GLM 4.7 等；按回合数过滤（≥ 5）	Qwen3-8B	ColdSFT（有意不训到最好）→ RL	三基准平均 21.7（基座 2.7，直接 RL 3.6，只 SFT 15.9）；任务来源影响最大；最强模型不是最好的教师
Kimi K3（第 8 讲）	长任务与工具调用轨迹	Kimi K3	SFT → RL（多推理强度专家）→ on-policy 蒸馏合并	SFT 阶段即学会调用工具与完成长任务
DeepSeek-R1（第 8、9 讲）	推理任务	DeepSeek-R1	R1-Zero：只有 RL；R1：几千条精选样本冷启动 SFT → RL	R1-Zero 输出混杂语言、难读，R1 把 SFT 加回来
MAI-Thinking-1（第 8、11 讲）	巩固 SFT 配比：STEM / 代码 56%（token 89%）、智能体 11%、通用 33%（token 2%）	MoE	打包 128K，batch 2,048，学习率 $1.7 \times 10^{-5} \rightarrow 5.2 \times 10^{-6}$；SFT 路由均衡强度为 RL 的 1000 倍，dropout 0.15	按样本配比不等于按 token 配比；要评测不打算提升的能力
Nemotron 3 Ultra（第 8 讲）	预训练 20T token；跨多个 harness 收集轨迹	MoE	打包 294,912 后 515,000，batch 64，学习率 $1.5 \times 10^{-5} \rightarrow 10^{-6}$；整对话 best-fit 打包	发布了后训练 checkpoint、数据与配方
Agent Data Protocol（第 8 讲）	13 个智能体数据集（约 126 万条）统一为类型化动作 / 观测	—	每个数据集转换一次、每个 harness 渲染一次	混合训练优于单来源：SWE-Bench +5.6、WebArena +4.1、AgentBench +4.2、GAIA +8.5
讲者实验室（第 11 讲）	编码智能体 RL	—	异步，`max_off_policyness` 1 vs 3	过期度 3 时最大重要性权重出现约 1,350 的尖峰
Assignment 3（第 6 章）	Number Search 5 万题；TextCraft-Synth 852 题（medium），验证 213 + 136（hard）	Qwen3.5-4B + LoRA r32	数据管线 → 同步 → 异步 → 消融 → hero run；7 次运行	未公布参考结果；验收标准为 50 步内奖励明显高于基线

7.4 一条完整的智能体训练流水线

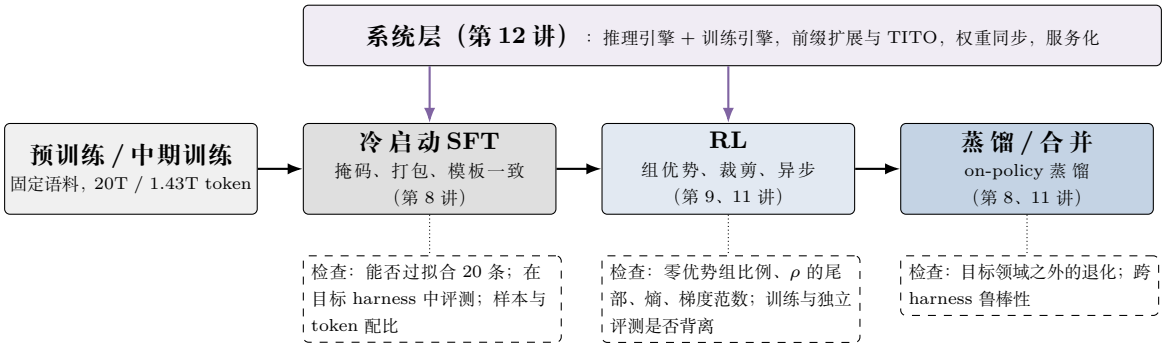


图 96: 把四讲内容放回一条前沿实验室的后训练流水线（依据第 8 讲的 Kimi K3 三段式与各讲内容整理），每个阶段下方是该阶段最值得检查的指标。

7.5 贯穿全模块的几条结论

1. **数据选择比超参数更重要，而且要按 token 看**。第 8 讲的任务来源（影响最大）、教师选择（最强 ≠ 最好）、样本与 token 配比，第 11 讲的 verifier 质量与零优势组，说的都是「模型看到什么」决定结果。作业对应的是：Number Search 预算没有余量、TextCraft hard 集只有约 38% 的任务在预算内可解——在解读曲线之前先弄清数据的上限。

2. **对齐比算法更脆弱**。损失掩码、回合结束符、模板一致、前缀扩展、TITO、差一位对齐，这些细节任何一处出错都会让训练「看起来在跑」却学到错误的东西。$\rho \approx 1$ 是同步训练里检验对齐的免费指标。

3. **方差控制是 RL 的主要工作**。基线、组比较、GAE、裁剪、按 token 总数归一化，本质上都在控制梯度估计的方差或单个样本的影响；第 9 讲的两臂老虎机给出了最小的完整例子。

4. **系统设计决定了能做多少实验**。异步 RL、前缀缓存、服务化接口让同样的预算能跑更多步；作业用便宜的 Number Search 做全部调试、只让昂贵的 TextCraft 跑一次，是同一种思路在个人预算下的版本。

7.6 开放问题与下一步

- **智能体 SFT 的掩码实验**（第 8 讲）：在观测 token 上也训练、或改变推理块的掩码，在 8B 规模上成本很低，讲义说这个实验「还没人做」；第 11 讲结尾提到的世界模型类工作给出了正面的初步证据。
- **小规模上的重要性加权**：作业 Part 4.3 在 50 步、短 completion 的设置下比较 `ppo` 与 `cross_entropy`，差异可能不大；把 `max_staleness` 或 `queue_capacity` 调大、或换到 completion 更长的 TextCraft，才更接近第 11 讲那条尖峰曲线的条件。
- **稀疏奖励下的 hard 任务**：TextCraft 只给二值奖励，hard 任务的组大多会全失败；第 11 讲的部分奖励、课程与 DAPO 动态采样都是可以在作业框架上尝试的扩展（作业本身不包含）。
- **复现建议**：先读第 6 章的复现清单，在本地跑通公开测试；再用 Number Search 的短探针验证 $\rho \approx 1$、轨迹记录与 `[sanity]` 告警；最后才部署 TextCraft。